

Міністерство освіти і науки України
Вінницький національний технічний університет

Алгоритми та структури даних

Навчальний посібник

Вінниця
ВНТУ
2024

УДК 004.415.2043+004.421
К65

Рекомендовано до видання Вченою радою Вінницького національного технічного університету Міністерства освіти і науки України (**протокол № 13 від 30.06.2024 р.**)

Рецензенти:

Р. М. Бабаков, доктор технічних наук, доцент

Н. В. Добровольська, кандидат технічних наук, доцент

Є. А. Паламарчук, кандидат технічних наук, доцент

Коваленко, О. О.

К65 Алгоритми та структури даних: навчальний посібник [Електронний ресурс] / О. О. Коваленко, О. М. Ткаченко, Чехместрук Р.Ю. Вінниця: ВНТУ, 2024. 113 с.

Посібник присвячений матеріалам лекційного курсу з дисципліни «Алгоритми та структури даних» для студентів, які навчаються за спеціальністю 121 «Інженерія програмного забезпечення» денної та заочної форм навчання.

Мета посібника – надати студентам можливість більш детально вивчити аудиторний матеріал, опрацювати теми відведені на самостійну роботу і підготуватися до іспиту, а також застосувати отримані знання для подальшої фахової роботи. Перелік та зміст тем відповідає програмі вказаної вище дисципліни.

УДК 004.415.2043+004.421

2
ЗМІСТ

ВСТУП.....	
4 1 ОСНОВНІ ПОНЯТТЯ АЛГОРИТМІВ ТА ПРИКЛАД ПЕРШОГО АЛГОРИТМУ.....	6
2 АНАЛІЗ АЛГОРИТМІВ.....	11
3 ОСНОВНІ СТРУКТУРИ ДАНИХ.....	15
4 МЕТОД ДЕКОМПОЗИЦІЇ. РЕКУРЕНТНІ СПІВВІДНОШЕННЯ.....	23
5 МЕТОД ЗАМІТАЮЧОЇ ПРЯМОЇ.....	32
6 ХЕШУВАННЯ ДАНИХ.....	37
7 БІНАРНІ ДЕРЕВА ПОШУКУ.....	45
8 ОЦІНЮВАННЯ СКЛАДНОСТІ АЛГОРИТМІВ. АЛГОРИТМИ СОРТУВАННЯ.....	53
9 ГРАФИ.....	60
10 АЛГОРИТМИ ПОШУКУ НАЙКОРОТШОГО ШЛЯХУ У ГРАФІ.....	72
11 ДИНАМІЧНЕ ПРОГРАМУВАННЯ.....	80
12 ЖАДІБНІ АЛГОРИТМИ ТА НР-ПОВНІ ЗАДАЧІ.....	88
13 КЛАСТЕРИЗАЦІЯ ЗА МЕТОДОМ К-СЕРЕДНІХ.....	96
14 ПОШУК НАЙБЛИЖЧИХ СУСІДІВ.....	102
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	111

ВСТУП

Основні поняття алгоритмізації, базові алгоритми та структури даних є основою для професійної роботи фахівців з програмної та комп'ютерної інженерії. Представлені навчальні ресурси сформовані на основі досвіду авторів та студентів спеціальності «Інженерія програмного забезпечення» робити та впровадження інноваційних та навчальних ІТ-проектів [1].

Для чого вивчати алгоритми? Це перше питання, на яке ми хочемо відповісти у вступі до навчального посібника. Відповіді на ці питання можна знайти в опитуваннях досвідчених програмістів на багатьох професійних форумах в мережі Інтернет, в шкільних та вузівських підручниках з інформатики, програмування та теорії алгоритмів. Такі дискусії часто піднімаються серед початківців та менеджерів ІТ-проектів. Але головна відповідь полягає в тому, що алгоритми дозволяють чітко формалізувати задачу, виявити критичні точки зміни стану системи, яку ви реалізуєте, використати існуючі оптимальні рішення реалізації різноманітних підпрограм, оптимізації вхідних та вихідних даних. Алгоритми є життєво необхідними складовими для рішення будь-яких задач і особливо в сфері комп'ютерних наук. Алгоритми відіграють ключову роль у сучасному розвитку технологій, в таких напрямках як:

- розв'язання математичних рівнянь різної складності, знаходження до бутку матриць, обернених матриць;
- знаходження оптимальних шляхів транспортування товарів та людей;
- знаходження оптимальних варіантів розподілення ресурсів між різними вузлами та системами);
- знаходження послідовностей, які співпадають;
- пошук інформації в глобальній мережі Інтернет;
- прийняття фінансових рішень в електронній комерції;
- обробка та аналіз аудіо та відео інформації;
- розробка різноманітних програмних продуктів автоматизації діяльності людини, підприємств та організацій;
- розробка та впровадження Інтернет-речей;
- розробка та впровадження сучасних технологій вимірювання та представлення інформації.

Цей список можна довго продовжувати, але ми представили його для того, щоб було зрозуміло – все починається з постановки задачі та алгоритмів її реалізації. Крім того, після появи книги Д. Кнута «Мистецтво програмування», вже не має сумнівів, що алгоритмізація і програмування – це захопливий, цікавий процес створення нових програмних продуктів. Може, хтось з читачів нашого посібника буде автором нових алгоритмів. Алгоритми завжди базуються на обмеженнях та цільових настановах. Але золоте правило розробника представлено гаслом: «Можливо найбільш важливим принципом для гарного розробника алгоритмів є відмова від того, щоб бути

4

задоволеним результатом» [2]. Теорія та практика створення і використання алгоритмів сучасна дисципліна, яка постійно вдосконалюється, знання з якої активно використовуються в практичній та науково-дослідній діяльності.

І, як останній аргумент щодо необхідності знань базових алгоритмів та структур даних – моніторинг питань на співбесідах при відборі фахівців для роботи в престижних ІТ-компаніях показав, що питання з алгоритмів, логічного мислення та математичних залежностей займають третину з усіх питань співбесіди.

Навчальний посібник відповідає програмі з нормативної дисципліни «Алгоритми та структури даних» галузі знань «Інформаційні технології» спеціальності «Інженерія програмного забезпечення». В посібнику розглянуто 14 основних тем, сформовано контрольні питання для перевірки знань. Посібник представлений як електронний та ресурс. При написанні навчальних матеріалів було використано відкриті навчальні ресурси університетів світу, книги та підручники, які стали вже класикою в теорії та практиці в алгоритмізації, матеріали відкритих дистанційних курсів, форумів про професійних фахівців з програмної інженерії, результати науково-дослідної роботи авторів та студентів, навчальні проекти студентів

1 ОСНОВНІ ПОНЯТТЯ АЛГОРИТМІВ ТА ПРИКЛАД ПЕРШОГО АЛГОРИТМУ

1.1 Поняття алгоритму

Поняття алгоритму є одним з фундаментальних понять і тому не існує єдиного строгого визначення, що таке алгоритм. Наприклад, в [3] автори наводять таке неформальне визначення алгоритму:

Алгоритм (algorithm) — це будь-яка коректно визначена обчислювальна процедура, на вхід (input) якої подається певна величина чи набір величин, і результат виконання якої є вихідна (output) величина чи набір значень. Таким чином, алгоритм є послідовністю обчислювальних кроків, що перетворюють вхідні величини у вихідні.

Алгоритм також можна розглядати як інструмент, призначений на вирішення коректно поставленого обчислювального завдання (computational problem). У постановці завдання загалом задаються відно

сини між входом і виходом. У алгоритмі описується конкретна обчислювальна процедура, з допомогою якої вдається домогтися виконання зазначених відносин.

Дещо інше визначення алгоритму можна зустріти в [4].

Під алгоритмом будемо розуміти скінченну чітко сформульовану сукупність вказівок, які визначають послідовність дій виконавця, спрямованих на досягнення мети або розв'язання задач певного класу за скінченний проміжок часу.

У будь-якому випадку нас більше цікавить не формальне визначення алгоритму, а відповідь на питання, чи може деяка обчислювальна процедура вважатися алгоритмом. А для цього необхідно перевірити, чи задовольняє згадана процедура властивостям, притаманним алгоритмам. Отже розглянемо ці властивості більш детально.

1.2 Властивості та форми подання алгоритмів

По-перше, що слід визначити в будь-якому алгоритмі – це те, що він застосовується до вхідних даних і видає результати. У звичних технічних термінах це означає, що алгоритм має **входи й виходи**. Крім того, в процесі роботи алгоритму з'являються проміжні результати, які використовуються надалі. Таким чином, кожний алгоритм має справи з даними – вхідними, проміжними та вихідними.

По-друге, дані для свого розміщення вимагають **пам'яті**. Пам'ять, за звичай, вважається однорідною й дискретною, тобто складається з однакових комірок, причому кожна комірка може містити один символ алфавіту даних. Таким чином, одиниці вимірювання обсягу даних і пам'яті узгод-

6

жені. При цьому пам'ять може бути нескінченною. Питання про те, чи потрібна спільна або окрема пам'ять для кожного із трьох видів даних (вхідних, вихідних і проміжних), вирішується по-різному.

По-третє, алгоритм складається з окремих елементарних кроків або дій. І кожна така дія повинна бути виконаною ще до виконання наступної дії. Дана властивість алгоритму отримала назву **дискретність**. Типовий приклад множини елементарних дій – система команд комп'ютера. Звичайно, елементарний крок має справу з фіксованим числом символів (це зручно, наприклад, для вимірювання часу роботи алгоритму числом виконаних кроків).

Властивість **зрозумілість** означає, що алгоритм може бути виконаний, якщо цілком зрозуміла кожна дія (команда) і вона може бути виконана у повній відповідності до її призначення.

По-четверте, послідовність кроків алгоритму повинна бути такою, що

після кожного кроку або вказується який крок робити далі, або дається команда зупинки, після чого робота алгоритму вважається завершеною. Така властивість алгоритму отримала назву **точність**.

Часто під одночасним виконанням властивостей зрозумілість та точність розуміють таку властивість як **визначеність або детермінованість**. По-п'яте, логічно від алгоритму чекати **результативності**, тобто зупинки після кінцевого числа кроків (що залежить від даних) з отриманням результату. Дана властивість і отримала назву результативність. Зокрема, будь-хто, хто розробляє алгоритм розв'язання деякого завдання, наприклад, обчислення функції $f(x)$, зобов'язаний показати, що алгоритм зупиняється після кінцевого числа кроків (як кажуть, збігається) для будь-якого x з області завдання. Однак перевірити збіжність набагато складніше. Збіжність, звичайно, не вдається встановити простим перегляданням опису алгоритму, а загального методу перевірки збіжності, придатного для будь-якого алгоритму A і будь-яких даних x , взагалі не існує.

По-шосте, кожен алгоритм повинен мати властивість **масовості**. Ця властивість передбачає, що даний алгоритм придатний для розв'язання будь-якої задачі з деякого класу задач. Проте дану властивість не слід розуміти як можливість розв'язання багатьох задач, оскільки у деяких випадках клас може складатися лише з однієї задачі.

По-сьоме, слід розрізняти: а) опис алгоритму (інструкції або програми); б) механізм реалізації алгоритму (наприклад, за допомогою комп'ютера), що включає засіб пуску, зупинки, реалізації елементарних кроків, видачі результатів і забезпечення управління ходом обчислення; в) процес реалізації алгоритму, тобто послідовність кроків, що буде породжена при застосуванні алгоритму до конкретних даних [5].

На практиці найбільш поширені такі форми подання алгоритмів [6]: - **словесна** (записи природною мовою);
- **графічна** (зображення у вигляді графічних символів);

7

- у вигляді **псевдокоду** (напівформалізовані описи алгоритмів умовною алгоритмічною мовою, що включає як елементи мови програмування, так і фрази природної мови, загальноприйняті математичні позначення тощо); - **програмна** (тексти мовою програмування)

1.3 Перший алгоритм. Сортування вставкою

Наш перший алгоритм, алгоритм сортування методом вставок, призначений для вирішення задачі сортування (sorting problem). Наведемо її формулювання [7].

Вхід: послідовність із n чисел $a_1, a_2, \dots, a_n$.

Вихід: перестановка (зміна порядку) $a'_1, a'_2, \dots, a'_n$ вхідної послідов

ності таким чином, що для її членів виконується співвідношення $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

Числа, що сортуються, називають ключами.

Наше вивчення алгоритмів починається із розгляду сортування вставкою (insertion sort). Цей алгоритм ефективно працює при сортуванні невеликої кількості елементів. Сортування вставкою нагадує спосіб, до якого прибігають гравці для сортування наявних на руках карт. Нехай спочатку в лівій

руці немає жодної карти, і всі вони лежать на столі сорочкою вгору. Далі зі столу береться по одній карті, кожна з яких поміщається у потрібне місце серед карток, що знаходяться у лівій руці. Щоб визначити, куди потрібно помістити чергову карту, її масть та значення порівнюється з мастю та значенням карт у руці. Припустимо, порівняння проводиться в напрямку зліва направо. В будь-який момент часу карти в лівій руці буде відсортовано, та це будуть ті карти, які спочатку лежали в колоді на столі.

Псевдокод сортування методом вставок представлений нижче за назвою INSERTION_SORT. На його вхід подається масив $A[1..n]$, що містить послідовність з n чисел, що сортуються (кількість елементів масиву A позначено в цьому коді як $\text{length}[A]$.) Вхідні числа сортуються без використання додаткової пам'яті: їх перестановка проводиться в межах масиву, та обсяг використовуваної при цьому додаткової пам'яті не перевищує деяку постійну величину. Після роботи алгоритму INSERTION_SORT вхідний масив містить відсортовану послідовність:

```
INSERTION_SORT(A)
1 for j ← 2 to length[A]
2 do key ← A[j]
3 //Вставте елемент A[j] у відсортовану послідовність A[1..j - 1]
4 i ← j - 1
5 while i > 0 та A[i] > key
6 do A[i + 1] ← A[i]
7 i ← i - 1
8 A[i + 1] ← key
```

8

Індекс j вказує "поточну карту", яка береться в руку. На початку кожної ітерації зовнішнього циклу for з індексом j масив A складається з двох частин. Елементи $A[1..j - 1]$ відповідають відсортованим картам у руці, а елементи $A[j + 1..n]$ — стопці карт, які поки що залишилися на столі. Зауважимо, що елементи $A[1..j - 1]$ спочатку теж знаходилися на позиціях від 1 до $j - 1$, але в іншому порядку, а тепер вони відсортовані. Назвемо цю властивість елементів $A[1..j - 1]$ **інваріантом** циклу (loop invariant) та сформулюємо його ще раз.

На початку кожної ітерації циклу for з рядків 1–8 підмасив $A[1..j - 1]$

містить ті ж елементи, які були в ньому з самого початку, але розташовані у відсортованому порядку.

Інваріанти циклу дозволяють зрозуміти, чи правильно працює алгоритм. Необхідно показати, що інваріанти циклів мають такі три властивості. **Ініціалізація.** Вони справедливі перед першою ініціалізацією циклу. **Збереження.** Якщо вони є істинними перед черговою ітерацією циклу, то залишаються істинними і після неї.

Завершення. Після завершення циклу інваріанти дозволяють переконатися в правильності алгоритму.

Якщо виконуються перші дві властивості, інваріанти циклу залишаються дійсними перед кожною черговою ітерацією циклу. Зверніть увагу на схожість з математичною індукцією, коли для доказу певної властивості всіх елементів упорядкованої послідовності потрібно довести його справедливості для початкового елемента цієї послідовності, а потім обґрунтувати крок індукції. У цьому випадку першій частині доказу відповідає обґрунтування того, що інваріант циклу виконується перед першою ітерацією, а другий частини - доказ того, що інваріант циклу виконується після чергової ітерації (крок індукції).

Для наших цілей третя властивість найважливіша, тому що нам потрібно за допомогою інваріант циклу продемонструвати коректність алгоритму. Вона також відрізняється розглянутим нами методом від звичайної математичної індукції, в якій крок індукції використовується в нескінченних послідовностях. В даному випадку у разі закінчення циклу “індукція” завершується.

Розглянемо, чи виконуються ці властивості для сортування методом вставок.

Ініціалізація. Почнемо з того, що покажемо справедливості інваріанту циклу перед першою ітерацією, тобто при $j = 2$. Таким чином, підмножина елементів $A[1..j - 1]$ складається тільки з одного елемента $A[1]$, що зберігає початкове значення. Більше того, у цій підмножині елементи відсортовані (тривіальне твердження). Усе сказане вище підтверджує, що інваріант циклу витримується перед першою ітерацією циклу.

Збереження. Далі, обґрунтуємо другу властивість: покажемо, що інваріант циклу зберігається після кожної ітерації. Висловлюючись неформально,

можно сказати, що в тілі внутрішнього циклу `for` відбувається зсув елементів $A[j - 1]$, $A[j - 2]$, $A[j - 3]$, на одну позицію вправо доти, доки не звільниться відповідне місце для елемента $A[j]$ (рядки 4–7), куди він і поміщається (рядок 8). За більш формального підходу до розгляду другої властивості потрібно було б сформулювати та обґрунтувати інваріант для внутрішнього циклу `while`. Однак на даному етапі краще не вдаватися в такі формальні подробиці, тому задовольнятимемося неформальним аналізом, щоб

показати, що у зовнішньому циклі дотримується друга властивість.

Завершення. Зрештою, подивимося, що відбувається після завершення роботи циклу. При сортуванні методом включень зовнішній цикл `for` завершується, коли j перевищує n , тобто. коли $j = n + 1$. Підставивши у формулювання інваріанта циклу значення $n + 1$, отримаємо таке твердження: у підмножині елементів $A[1..n]$ знаходяться ті ж елементи, що були в ньому до початку роботи алгоритму, але розташовані у відсортованому порядку. Однак підмножина $A[1..n]$ і є сам масив A ! Таким чином, весь масив відсортовано, що і підтверджує коректність алгоритму.

1.4 Контрольні питання

Дайте визначення алгоритму.

Що таке дискретність роботи алгоритму?

Що таке детермінованість алгоритму?

Що таке зрозумілість алгоритму?

Що таке точність алгоритму?

Що таке результативність алгоритму?

Що таке масовість алгоритму?

Які форми подання алгоритму Ви знаєте?

2 АНАЛІЗ АЛГОРИТМІВ

Аналіз алгоритму полягає в тому, щоб передбачити потрібні для його виконання ресурси. Іноді оцінюється потреба в таких ресурсах, як пам'ять, пропускна здатність мережі або необхідне апаратне забезпечення, але най

частіше визначається час обчислення. Шляхом аналізу деяких алгоритмів, призначених для розв'язку однієї та тієї самої задачі, можна легко обрати найбільш ефективний з них. В процесі такого аналізу може також виявитись, що декілька алгоритмів приблизно рівноцінні, а всі інші варто відкинути.

З урахуванням того, що алгоритми реалізуються у вигляді комп'ютерних програм, в більшості випадків як технологія аналізу алгоритмів буде використовуватись модель узагальненої однопроцесорної машини з довільним доступом до пам'яті (RAM – Random Access Memory). В цій моделі команди процесора виконуються послідовно; операції, які виконуються одночасно, відсутні. У цю модель входять ті команди, які зазвичай можна знайти в реальних комп'ютерах: арифметичні (додавання, віднімання, добуток, ділення, обчислення остачі від ділення, наближення дійсного числа найближчим більшим чи найближчим меншим цілим числом), операції переміщення даних (завантаження значення в пам'ять, копіювання) та керуючі операції (умовне та безумовне розгалуження, виклик підпрограми і повернення з неї). Для виконання кожної такої інструкції потрібно певний фіксований проміжок часу.

В моделі RAM є цілочисельний тип даних та тип чисел з плаваючою комою. Також передбачається, що є верхня межа розміру одного слова даних. У моделі RAM, яка розглядається, не моделюється ієрархія пристроїв пам'яті, які сьогодні розповсюджені у звичайних комп'ютерах. Таким чином, кеш та віртуальна пам'ять відсутні у RAM. Моделі, які включають в себе таку ієрархію, набагато складніше моделі RAM, тому вони можуть ускладнити роботу. Окрім цього, аналіз, який заснований на моделі RAM, зазвичай чудово прогнозує продуктивність алгоритмів, які виконуються на реальних машинах [8].

2.1 Аналіз алгоритму, що працює за методом вставок

Час роботи процедури INSERTION_SORT залежить від набору вхідних значень: для сортування тисячі чисел потрібно більше часу, ніж для сортування трьох чисел. Крім того, час сортування за допомогою цієї процедури може бути різним для послідовностей, що складаються з однієї і тієї ж кількості елементів, залежно від ступеня впорядкованості цих послідовностей до початку сортування. Загалом час роботи алгоритму збільшується зі збільшенням кількості вхідних даних, тому загальноприйнята практика — представляти час роботи програми як функцію, яка залежить від кількості вхідних елементів. Для цього поняття "час роботи алгоритму" та "розмір вхідних даних" потрібно визначити точніше.

Найбільш адекватне поняття розміру вхідних даних (input size) залежить від розглянутої задачі. У багатьох задачах, таких як сортування або

дискретні перетворення Фур'є, — це кількість вхідних елементів, наприклад, розмір n масиву, що сортується.

Час роботи алгоритму для тих чи інших даних вимірюється у кількості елементарних операцій, або “кроків”, які необхідно виконати. Тут зручно ввести поняття кроку, щоб міркування були якомога більш машинно незалежними. На даному етапі ми виходитимемо з точки зору, згідно з якою для виконання кожного рядка псеводкоду потрібен фіксований час. Час виконання різних рядків може відрізнятися, але ми припустимо, що він є одним для будь-якого рядка.

Почнемо з того, що введемо для процедури INSERTION_SORT час виконання кожної інструкції та кількість їх повторень. До кожного $j = 2, 3, \dots, n$, де $n = \text{length}[A]$, позначимо через t_j кількість перевірок умови у циклі **while** (рядок 5). При нормальному завершенні циклів **for** або **while** (тобто коли перестає виконуватися умова, задана в заголовку циклу) умова перевіряється на один раз більше, ніж виконується тіло циклу.

```

INSERTION_SORT(A) кількість повторів 1 for  $j \leftarrow 2$  to  $\text{length}[A]$   $n-2$ 
do key  $\leftarrow A[j]$   $n-1$  3 //Вставте елемент  $A[j]$  в  $A[1..j-1]$ 
4  $i \leftarrow j-1$   $n-1$   $\diamond\diamond\diamond-2$ 
5 while  $i > 0$  та  $A[i] > \text{key}$   $\sum \diamond\diamond\diamond\diamond\diamond-2$ 
6 do  $A[i+1] \leftarrow A[i]$   $\sum (\diamond\diamond\diamond-1) \diamond\diamond\diamond-2$ 
7  $i \leftarrow i-1$   $\sum (\diamond\diamond\diamond-1)$  8  $A[i+1] \leftarrow \text{key}$   $n-1$ 

```

Час роботи алгоритму - це сума проміжків часу, необхідних для виконання кожної інструкції, що входить до його складу. Якщо виконання інструкції триває протягом часу c і вона повторюється в алгоритмі n разів, її внесок у повний час роботи алгоритму дорівнює cn .

Щоб обчислити час роботи алгоритму INSERTION_SORT (позначимо його через $T(n)$), потрібно підсумувати значення, що стоять у стовпці «кількість повторів», внаслідок чого отримаємо:

$$T(n) = c(n + (n-1) + (n-1) + \sum_{j=2}^n (\sum_{i=j-1}^0 (\sum_{k=0}^i (A[k] > \text{key}) + 1)) + (n-1))$$

Навіть якщо розмір вхідних даних є фіксованою величиною, час роботи алгоритму може залежати від ступеня впорядкованості сортованих величин, який вони мали до введення. Наприклад, найсприятливіший випадок для алгоритму INSERTION_SORT — коли всі елементи масиву вже відсортовані. Тоді кожного $j = 2, 3, \dots, n$ виходить, що $A[i] \leq \text{key}$ в рядку 5, коли i дорівнює своєму початковому значенню $j-1$. Таким чином, при $j = 2, 3, \dots, n$

кількість перевірок умови $t_j = 1$, і час роботи алгоритму у найсприятливішому випадку обчислюється так:

$$T(n) = c(n + (n - 1) + (n - 1) + (n - 1) + (n - 1)) = c(5n - 4)$$

Цей час роботи можна записати як $an + b$, де a та b — константи, що залежать від величини c ; тобто. цей час є **лінійною функцією** від n . Якщо елементи масиву відсортовані в порядку, зворотному необхідному (в даному випадку в порядку зменшення), то це найгірший випадок. Кожен елемент $A[j]$ необхідно порівнювати з усіма елементами вже відсортованої підмножини $A[1..j - 1]$, так що для $j = 2, 3, \dots, n$ значення $t_j = j$. З урахуванням того, що

$$\sum_{j=2}^n j = \frac{n(n+1)}{2} - 1$$

та

$$\sum_{j=2}^n (j - 1) = \frac{n(n-1)}{2}$$

отримуємо, що час роботи алгоритму INSERTION_SORT у найгіршому випадку визначається співвідношенням

$$T(n) = c(n + (n - 1) + (n - 1) + (n - 1) + (n - 1)) = c(4n - 3 + \frac{n(n+1)}{2} - 1 + \frac{n(n-1)}{2})$$

2)

Цей час роботи можна записати як $an^2 + bn + c$, де константи a , b і c залежать від c . Таким чином, це **квадратична функція** від n .

2.2 Найгірший і середній час роботи

Аналізуючи алгоритм, який працює за методом вставок, ми розглядали як найкращий, так і найгірший випадок, коли елементи масиву були відсортовані у порядку, зворотному необхідному. Далі ми приділятимемо основну увагу визначенню лише часу роботи у найгіршому разі, тобто, максимальному часу роботи з усіх вхідних даних розміру n . Тому є три причини [9].

- Час роботи алгоритму у найгіршому випадку – це верхня межа цієї величини для будь-яких вхідних даних. Маючи це значення, ми точно знаємо, що для виконання алгоритму не потрібно більше часу. Не треба буде робити якихось складних припущень про час роботи та сподіватися, що насправді ця величина не буде перевищена.

- У деяких алгоритмах найгірший випадок трапляється досить часто. Наприклад, якщо у базі даних відбувається пошук інформації, то найгіршому випадку відповідає ситуація, коли потрібна інформація у базі даних відсутня. У деяких пошукових програмах пошук відсутності інформації може відбуватися досить часто.

- Характер поведінки “усередненого” часу роботи часто нічим не кращий за поведінку часу роботи для найгіршого випадку. Наприклад, якщо

13

оцінити середній час роботи алгоритму як середнє арифметичне між найкращим і найгіршим випадком, то характер залежності буде квадратичним, як і в найгіршому випадку.

2.3 Порядок зростання

Для полегшення аналізу процедури INSERTION_SORT було зроблено деякі спрощувальні припущення. По-перше, ми проігнорували фактичний час виконання кожної інструкції, представивши цю величину як деяку константу c , однакову для всіх інструкцій. Далі ми побачили, що врахування цих констант дає зайву інформацію: час роботи алгоритму у найгіршому разі виражається формулою $an^2 + bn + c$. Отже, ми ігноруємо як фактичні вартості команд, так й їх абстрактні вартості c [10].

Тепер введемо ще одне абстрактне поняття, яке спрощує аналіз. Це швидкість зростання (rate of growth), або порядок зростання (order of growth), часу роботи, який і цікавить нас насправді. Таким чином, до уваги буде братися лише головний член формули (тобто у нашому випадку an^2), оскільки за великих n членами меншого порядку можна знехтувати. Крім того, постійні множники при головному члені також ігноруватимуться, оскільки для оцінки обчислювальної ефективності алгоритму з вхідними даними великого обсягу вони менш важливі, ніж порядок зростання. Таким чином, час роботи алгоритму, який працює за методом вставок, у найгіршому випадку дорівнює $\Theta(n^2)$ (вимовляється “тета від n у квадраті”). Зазвичай один алгоритм вважається ефективнішим за інший, якщо час його роботи у найгіршому разі має нижчий порядок зростання. Через наявність постійних множників та другорядних членів ця оцінка може бути помилковою, якщо вхідні дані невеликі. Однак якщо обсяг вхідних даних значний, то, наприклад, алгоритм $\Theta(n^2)$ у найгіршому випадку працює швидше, ніж алгоритм $\Theta(n^3)$. Різні позначення, що використовуються для

оцінювання складності алгоритмів, більш детально буде розглянуто у розділі 7.

2.4 Контрольні питання

Які припущення передбачає використання моделі RAM?

Охарактеризуйте поняття «складність алгоритму».

Охарактеризуйте поняття «ефективність алгоритму».

Що таке інваріанти циклу та для чого вони використовуються?

Як оцінюються найгірший та середній час роботи алгоритму?

Чому найчастіше оцінюється найгірший час роботи алгоритму?

3 ОСНОВНІ СТРУКТУРИ ДАНИХ

3.1 Стеки

Стеком називається множина деякої змінної кількості даних, над якою виконуються такі операції:

- поповнення стеку новими даними;
- перевірка, яка визначає чи стек порожній;
- перегляд останніх доданих даних;
- знищення останніх доданих даних [11].

На основі такого функціонального опису можна сформувати логічний опис. Стек – це такий послідовний список із змінною довжиною, включення і виключення елементів з якого виконуються тільки з одного боку списку. Застосовуються і інші назви стеку – магазин, пам'ять що функціонує за принципом LIFO (Last – In – First – Out – „останнім прийшов – першим вийшов”).

Самий „верхній” елемент стеку, тобто останній доданий і ще не знищений, відіграє особливу роль: саме його можна модифікувати й знищувати. Цей елемент називають вершиною стеку. Іншу частину стеку називають тілом стеку. Тіло стеку, само собою, є стеком: якщо виключити зі стеку його вершину, то тіло перетворюється в стек.

Основні операції над стеком – включення нового елемента (push – заштовхувати) і виключення елемента зі стеку (pop – вискакувати).

Корисними можуть бути також допоміжні операції:

- визначення поточної кількості елементів в стеку;
- очищення стеку;
- „неруйнуюче” читання елемента з вершини стека, яке може бути реалізоване, як комбінація основних операцій – виключити елемент зі стеку та включити його знову в стек.

При представленні стеку в статичній пам’яті для стеку виділяється пам’ять, як для вектора. В описі цього вектора окрім звичайних для вектора параметрів повинен знаходитися також показчик стеку – адреса вершини стека. Обмеження даного представлення полягає в тому, що розмір стеку обмежений розмірами вектора.

Показчик стеку може вказувати або на перший вільний елемент стеку, або на останній записаний в стек елемент. Однаково, який з цих двох варіантів вибрати, важливо надалі строго дотримуватися його при обробці стеку.

При занесенні елементу в стек елемент записується на місце, яке визначається показчиком стеку, потім показчик модифікується так, щоб він вказував на наступний вільний елемент (якщо показчик вказує на останній записаний елемент, то спочатку модифікується показчик, а потім проводиться запис елемента). Модифікація показчика полягає в надбавці до

15

нього або у відніманні від нього одиниці (стек росте у бік збільшення адреси).

Операція виключення елемента полягає в модифікації показчика стеку (в напрямку, зворотному модифікації при включенні) і вибірці значення, на яке вказує показчик стеку. Після вибірки комірка, в якій розміщувався вибраний елемент, вважається вільною.

Операція очищення стеку зводиться до запису в показчик стеку початкового значення – адреси початку виділеної ділянки пам’яті. Визначення розміру стека зводиться до обчислення різниці показчиків: показчика стеку й адреси початку ділянки.

При зв’язному представленні стеку кожен елемент стеку складається із значення і показчика, який вказує на попередньо занесений у стек елемент. Зв’язне представлення викликає втрату пам’яті, що зумовлено наявністю показчика в кожному елементі стеку, і представляє інтерес тільки у випадку, коли важко визначити максимальний розмір стеку.

Отже, для зв’язного представлення стеку потрібно, щоб кожен його елемент описувався структурою, яка поєднує дані і показчик на наступний елемент.

Для виконання операцій над стеком потрібен один показчик на вершину стеку. Створення порожнього стеку полягатиме у присвоєнні показчика на вершину нульового значення, що означатиме, що стек порожній.

Послідовність кроків для додавання елемента в стек складається з

декількох кроків:

1. Виділити пам'ять під новий елемент стеку.
 2. Занесення значення в інформаційне поле.
 3. Встановлення зв'язку між ним і „старою” вершиною стеку.
 4. Переміщення вершини стеку на новий елемент.
- Вилучення елемента зі стеку також проводять за кілька кроків:
1. Зчитування інформації з інформаційного поля вершини стеку.
 2. Встановлення на вершину стеку допоміжного покажчика.
 3. Переміщення покажчика вершини стеку на наступний елемент.
 4. Звільнення пам'яті, яку займає „стара” вершина стеку.

3.2 Черги

Чергою називається множина змінної кількості даних, над якою можна виконувати такі операції:

- поповнення черги новими даними;
- перевірка, яка визначає чи пуста черга;
- перегляд перших доданих даних;
- знищення перших доданих даних.

На основі такого функціонального опису можна сформувати логічний опис. Чергою FIFO (First – In – First – Out – „першим прийшов – першим вийшов”) називається такий послідовний список із змінної довжиною, в

якому включення елементів виконується тільки з одного боку списку (хвіст черги), а виключення – з другого боку (голова черги).

Основні операції над чергою – ті ж, що і над стеком – включення, виключення, визначення розміру, очищення, „неруйнуюче” читання. При представленні черги вектором в статичній пам'яті на додаток до звичайних для опису вектора параметрів в ньому повинні знаходитися два покажчики: на голову і на хвіст черги. При включенні елемента в чергу елемент записується за адресою, яка визначається покажчиком на хвіст, після чого цей покажчик збільшується на одиницю. При виключенні елемента з черги вибирається елемент, що адресується покажчиком на голову, після чого цей покажчик зменшується на одиницю.

Очевидно, що з часом покажчик на хвіст при черговому включенні елемента досягне верхньої межі тієї ділянки пам'яті, яка виділена для черги. Проте, якщо операції включення чергувати з операціями виключення елементів, то в початковій частині відведеної під чергу пам'яті є вільне місце. Для того, щоб місця, які займалися виключеними елементами, могли бути повторно використані, черга замикається в кільце: покажчики (на початок і на кінець), досягнувши кінця виділеної області пам'яті, перемикаються на її початок. Така організація черги в пам'яті називається кільцевою чергою. Можливі, звичайно, і інші варіанти організації: наприклад, кожний раз,

коли покажчик кінця досягне верхньої межі пам'яті, зсовувати всі непорожні елементи черги до початку ділянки пам'яті, але як цей, так і інші варіанти вимагають переміщення в пам'яті елементів черги і менш ефективні, ніж кільцева черга.

У початковому стані покажчики на голову і хвіст вказують на початок ділянки пам'яті. Рівність цих двох покажчиків є ознакою порожньої черги. Якщо в процесі роботи з кільцевою чергою кількість операцій включення перевищує кількість операцій виключення, то може виникнути ситуація, в якій покажчик кінця „наздожене” покажчик початку. Це ситуація заповненої черги, але якщо в цій ситуації покажчики порівнюються, ця ситуація буде такою ж як при порожній черзі. Для розрізнення цих двох ситуацій до кільцевої черги висувається вимога, щоб між покажчиком кінця і покажчиком початку залишався „проміжок” з вільних елементів. Коли цей „проміжок” скопичується до одного елемента, черга вважається заповненою і подальші спроби запису в неї блокуються. Очищення черги зводиться до запису одного і того ж (не обов'язково початкового) значення в обидва покажчики. Визначення розміру полягає в обчисленні різниці покажчиків з урахуванням кільцевої природи черги.

При зв'язному представленні черги кожен елемент черги складається із значення і покажчика, який вказує на попередньо занесений у чергу елемент.

Зв'язне представлення викликає втрату пам'яті, що зумовлено на явність покажчика в кожному елементі черги, і представляє інтерес тільки у випадку, коли важко визначити максимальний розмір черги. Для зв'язного

17

представлення черги потрібно, щоб кожен його елемент описувався структурою, яка поєднує дані і покажчик на наступний елемент. Для виконання операцій над чергою потрібно два покажчики: на голову і хвіст черги. Створення порожньої черги полягатиме у присвоєнні покажчиків на голову і хвіст черги нульових значень, що означатиме, що черга порожня.

Послідовність кроків для додавання елемента в кінець черги складається з декількох кроків:

1. Виділити пам'ять під новий елемент черги.
2. Занесення значення в інформаційне поле.
3. Занесення нульового значення в покажчик.
4. Встановлення зв'язку між ним і останнім елементом черги і новим, враховуючи випадок пустої черги.
5. Переміщення покажчика кінця черги на новий елемент. Вилучення елемента з черги також проводять за кілька кроків: 1. Зчитування інформації з інформаційного поля голови черги. 2. Встановлення на голову черги допоміжного покажчика. 3. Переміщення покажчика початку черги на наступний елемент. 4. Звільнення пам'яті, яку займав перший елемент черги. В реальних задачах іноді виникає необхідність

у формуванні черг, відмінних від приведених структур. Порядок вибірки елементів з таких черг визначається пріоритетами елементів. Пріоритет в загальному випадку може бути представлений числовим значенням, яке обчислюється або на підставі значень деяких полів елемента, або на підставі зовнішніх чинників. Так попередньо наведені структури стек і черги можна трактувати як пріоритетні черги, в яких пріоритет елемента залежить від часу його включення в структуру. При вибірці елемента всякий раз вибирається елемент з щонайбільшим пріоритетом.

Черги з пріоритетами можуть бути реалізовані на лінійних структурах – в суміжному або зв'язному представленні. Можливі черги з пріоритетним включенням – в яких послідовність елементів черги весь час підтримується впорядкованою, тобто кожний новий елемент включається на те місце в послідовності, яке визначається його пріоритетом, а при виключенні завжди вибирається елемент з голови. Можливі і черги з пріоритетним виключенням – новий елемент включається завжди в кінець черги, а при виключенні в черзі шукається (цей пошук може бути тільки лінійним) елемент з максимальним пріоритетом і після вибірки вилучається з послідовності. І в тому, і в іншому варіанті потрібний пошук, а якщо черга розміщується в статичній пам'яті – ще і переміщення елементів.

3.3 Деки

Дек – особливий вид черги. Дек (deq – double ended queue, тобто черга з двома кінцями) – це такий послідовний список, в якому як включення, так

18

і виключення елементів, може здійснюватися з будь-якого з двох кінців списку. Так само можна сформулювати поняття деку, як стек, в якому включення і виключення елементів може здійснюватися з обох кінців.

Деки рідко зустрічаються у своєму первісному визначенні. Окремий випадок деку – дек з обмеженим входом і дек з обмеженим виходом. Логічна і фізична структури деку аналогічні логічній і фізичній структурі кільцевої черги. Проте, стосовно деку доцільно говорити не про голову і хвіст, а про лівий і правий кінець.

Над деком доступні такі операції:

- включення елемента праворуч;
- включення елемента ліворуч;
- виключення елемента праворуч;
- виключення елемента ліворуч;
- визначення розміру;
- очищення.

Фізична структура деку в статичній пам'яті ідентична структурі кільцевої черги.

3.4 Лінійні списки

Лінійні списки є узагальненням попередніх структур; вони дозволяють представити множину так, щоб кожний елемент був доступний і при цьому не потрібно було б зачіпати деякі інші [12].

Списки є досить гнучкою структурою даних, оскільки їх легко зробити більшими або меншими, і їх елементи доступні для вставки або вилучення в будь-якій позиції списку. Списки також можна об'єднувати або розділяти на менші списки.

Лінійний список – це скінчена послідовність однотипних елементів (вузлів), можливо, з повторенням. Кількість елементів у послідовності називається довжиною списку. Вона в процесі роботи програми може змінюватися. Лінійний список L , що складається з елементів $d_1, d_2, \dots, d_n$ які мають однаковий тип, записують у вигляді $L = \langle d_1, d_2, \dots, d_n \rangle$ або зображають графічно (рис. 3.1).

d1 d2 d3 dn.....

Рис. 3.1. Графічне представлення лінійного списку

Важливою властивістю лінійного списку є те, що його елементи можна лінійно впорядкувати відповідно до їх позиції в списку.

Для формування абстрактного типу даних на основі математичного визначення списку потрібно задати множину операторів, які виконуються над об'єктами типу список. Проте не існує однієї множини операторів, що

виконуються над списками, які задовольняють відразу всі можливі застосування.

Найчастіше зі списками доводиться виконувати такі операції:

- знайти елемент із заданою властивістю;
- визначити i -й елемент у лінійному списку;
- внести додатковий елемент до або після вказаного вузла;
- вилучити певний елемент списку;
- впорядкувати вузли лінійного списку в певному порядку.

У реальних мовах програмування не існує якої-небудь структури даних для зображення лінійного списку так, щоб усі операції над ним виконувалися в однаковій мірі ефективно. Тому при роботі з лінійними списками важливе значення має подання лінійних списків, які використовуються в програмі, таким чином, щоб була забезпечена максимальна ефективність і за часом

виконання програми, і за обсягом потрібної їй пам'яті. Лінійний список є послідовність об'єктів. Позиція елемента в списку має інший тип даних, відмінний від типу даних елемента списку, і цей тип залежить від конкретної фізичної реалізації. Над лінійним списком допустимі такі операції.

Операція вставки – вставляє елемент в конкретну позицію в списку, переміщуючи елементи від цієї позиції і далі в наступну, більш вищу позицію. Операція локалізації – повертає позицію об'єкта в списку. Якщо в списку об'єкт зустрічається декілька разів, то повертається позиція першого від початку списку об'єкта. Якщо об'єкта немає в списку, то повертається значення, яке дорівнює довжині списку, збільшене на одиницю. Операція вибірки елемента з списку – повертає елемент, який знаходиться в конкретній позиції списку. Результат не визначений, якщо в списку немає такої позиції.

Операція вилучення – вилучає елемент в конкретній позиції зі списку. Результат невизначений, якщо в списку немає вказаної позиції. Операції вибірки попереднього і наступного елемента – повертають відповідно наступний і попередній елемент списку відносно конкретної позиції в списку.

Функція очищення списку робить список порожнім. Основні методи зберігання лінійних списків поділяються на методи послідовного і зв'язного зберігання. При виборі способу зберігання в конкретній програмі слід враховувати, які операції і з якою частотою будуть виконуватися над лінійними списками, вартість їх виконання та обсяг потрібної пам'яті для зберігання списку.

Найпростіша форма представлення лінійного списку — це вектор. Визначивши таким чином список можна потім звертатися до них в циклі і виконувати необхідні дії. Однак при такому представленні лінійного списку не вдасться уникнути фізичного переміщення елементів, якщо по-

трібно додавати нові елементи, або вилучати існуючі. Набагато швидше вилучати елементи можна за допомогою простої схеми чистки пам'яті. Замість вилучення елементів із списку, їх помічають як невикористані.

Більш складною організацією при роботі зі списками є розміщення в масиві декількох списків або розміщення списку без прив'язки його початку до першого елемента масиву.

При зв'язному представленні лінійного списку кожен його елемент складається із значення і покажчика, який вказує на наступний елемент у списку.

На рисунку 3.2 наведено структуру однозв'язного списку. Кожний список повинен мати особливий елемент, який називається покажчиком на по-

чаток списку, або головою списку.

List Head Inf Next Inf Next Inf Null

Рис. 3.2. Структура однозв'язного списку

Проте, обробка однозв'язного списку не завжди зручна, оскільки відсутня можливість просування в протилежний бік. Таку можливість забезпечує двохзв'язний список, кожний елемент якого містить два покажчики: на наступний і попередній елементи списку (рис. 3.3).

List Head List EndNull Inf Next Prev Inf Next Prev Inf Null

Рис. 3.3. Структура двохзв'язного списку

Для зручності обробки списку додають ще один особливий елемент – покажчик кінця списку. Наявність двох покажчиків в кожному елементі ускладнює список і приводить до додаткових витрат пам'яті, але разом з тим забезпечує більш ефективне виконання деяких операцій над списком.

Різновидом розглянутих видів лінійних списків є кільцевий список, який може бути організований на основі як однозв'язного, так і двохзв'язного списків. При цьому в однозв'язному списку покажчик останнього елемента повинен вказувати на перший елемент; в двохзв'язному списку в першому і останньому елементах відповідні покажчики змінюються.

Підсумовуючи, відзначимо основні переваги зв'язних списків. А саме:

- легкість додавання та видалення елементів;
- розмір обмежений лише обсягом пам'яті комп'ютера та розрядністю вказівників;

21

- динамічне додавання та видалення елементів.

До недоліків цієї технології належить:

- складність визначення адреси елемента за його індексом (номером) у списку;
- на поля-вказівники (вказівники на наступний та попередній елемент) витрачається додаткова пам'ять (у масивах, наприклад, покажчики не потрібні);
- робота зі списком повільніша, ніж з масивами, оскільки до будь-якого елемента списку можна звернутися, тільки пройшовши всі попередні йому елементи;
- елементи списку можуть бути розташовані в пам'яті розріджено, що

негативно вплине на кешування процесора;

- над зв'язними списками набагато важче (хоча й у принципі можливо) проводити паралельні векторні операції, такі як обчислення суми;
- кеш-промахи під час обходу списку.

3.5 Контрольні питання

Поясніть, у чому полягають особливості стека.

Як можна реалізувати стек програмно?

Назвіть основні операції, які можна виконувати зі стеком.

Дайте означення черги.

Які різновиди черг Ви знаєте?

З яких кроків складається додавання/вилучення елемента в чергу? З яких кроків складається вилучення елемента з черги?

Що таке деки?

Дайте означення лінійного списку.

Які операції можна виконувати зі списками?

Які переваги та недоліки списків порівняно з масивами?

4 МЕТОД ДЕКОМПОЗИЦІЇ. РЕКУРЕНТНІ СПІВВІДНОШЕННЯ

4.1 Приклад алгоритму, що реалізує метод декомпозиції

Багато корисних алгоритмів мають рекурсивну структуру: для вирішення даного завдання вони рекурсивно викликають самі себе один або кілька разів, щоб вирішити допоміжне завдання, що має безпосереднє відношення до поставленого завдання. Такі алгоритми часто розробляються за допомогою **методу декомпозиції**, або **розбиття**: складне завдання розбива

ється на декілька більше простих, які подібні до вихідної задачі, але мають менший обсяг; далі ці допоміжні завдання вирішуються рекурсивним методом, після чого отримані рішення комбінуються з метою одержання вирішення вихідного завдання [13].

Ця рекурсивна парадигма отримала назву «розділяй та володарюй» (divide and conquer). Сама парадигма «розділяй та володарюй» складається з трьох етапів:

Розділення задачі на декілька підзадач.

Підкорення - рекурсивний розв'язок цих підзадач. Коли об'єм підзадач достатньо малий, вони розв'язуються безпосередньо.

Комбінування розв'язку початкової задачі з розв'язків допоміжних підзадач.

Розгляд цієї парадигми ми почнемо з алгоритму сортування методом злиття (merge sort). В рамках підходу «розділяй та володарюй» цей метод можна описати так:

Розділення: послідовність для сортування, яка складається з n елементів, розбивається на дві менші послідовності, кожна з яких містить $n/2$ елементів.

Рекурсивний розв'язок: сортування обох створених послідовностей методом злиття у випадку, якщо їх розмір перевищує один.

Комбінування: злиття двох відсортованих послідовностей для отримання кінцевого результату.

Рекурсія досягає своєї нижньої межі, коли довжина послідовності для сортування дорівнює 1. В цьому випадку всю роботу вже зроблено, оскільки будь-яку таку послідовність можна вважати впорядкованою.

Основна операція, яка виконується в процесі сортування за методом злиття, - це об'єднання двох відсортованих послідовностей під час комбінування (останній етап). Це робиться за допомогою додаткової процедури Merge(A, p, q, r), де A - це масив, а p, q та r - індекси, які нумерують елементи масиву, такі що $p < q < r$. В цій процедурі припускається, що елементи підмасивів $A[p..q]$ та $A[q+1..r]$ впорядковані. Вона зливає ці два підмасиви в один відсортований, елементи якого замінюють поточні елементи підмасиву $A[p..r]$. Як і в попередньому прикладі, вважаємо, що сортуються карти.

З обчислювальної точки зору виконання кожного основного кроку займає однакові проміжки часу, тоді як все зводиться до порівняння двох верхніх карт. Оскільки необхідно виконати принаймні n основних кроків, час роботи процедури Merge дорівнює $\Theta(n)$. Приклад виконання процедури наведено на рис. 4.1, а) – і).

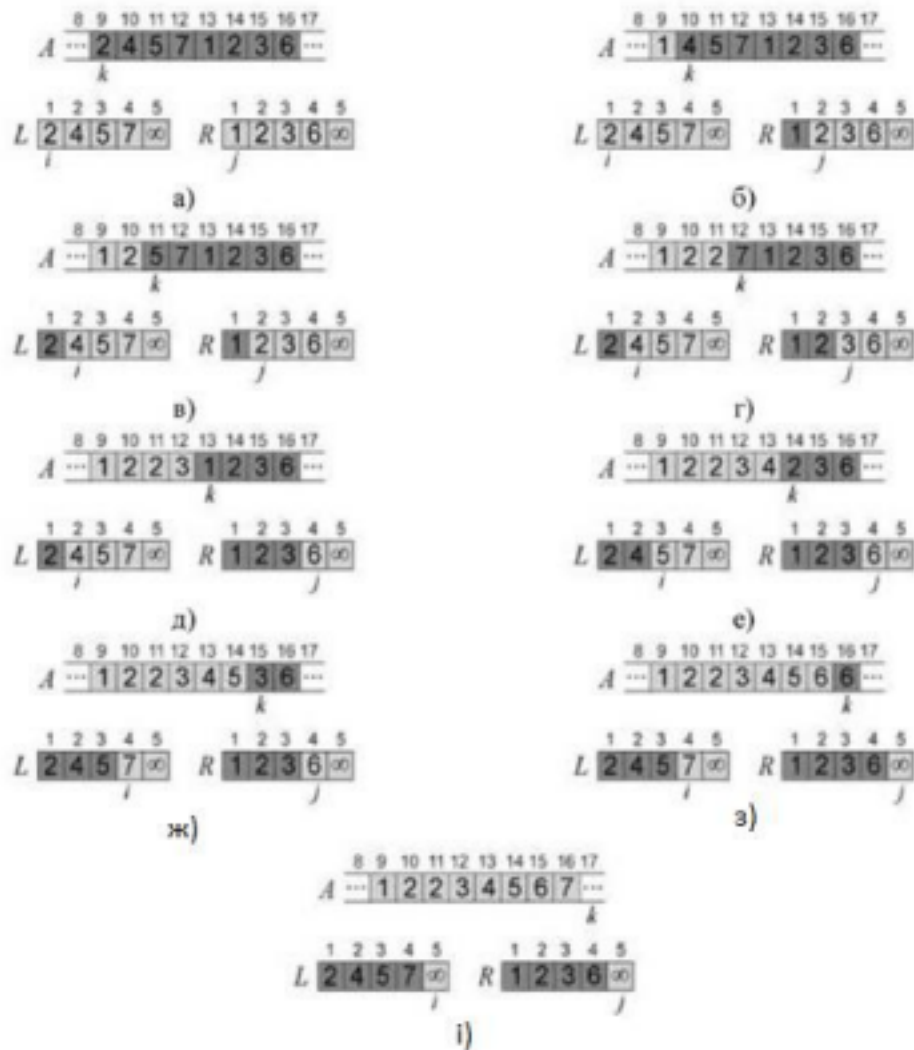


Рис. 4.1. Приклад виконання процедури сортування

Описану ідею реалізовано у нижченаведеному псевдокоді, проте в ньому також є додаткові програмістські хитрощі, завдяки яким в ході кожного основного кроку не доводиться перевіряти, чи кожний з двох стосів порожній. Ідея полягає в тому, щоб помістити в самий низ обох колод, що об'єднуються, так звану сигнальну карту особливого значення, що дозволяє спростити код. Для позначення сигнальної картки використовується символ ∞ . У колоді немає карт, вищих за сигнальну карту. Процес триває доти, доки карти, що перевіряються, в обох стосах не виявляються сигнальними. Як тільки це станеться, це означатиме, що всі несигнальні карти вже розміщено у вихідній стос. Оскільки заздалегідь відомо, що у вихідному стосі має бути

рівно $r - p + 1$ карта, виконавши таку кількість основних кроків, можна зупинитися:

MERGE(A, p, q, r)

```

1  $n_1 \leftarrow q - p + 1$ 
2  $n_2 \leftarrow r - q$ 
3 // Створюємо масиви  $L[1..n_1 + 1]$  и  $R[1..n_2 + 1]$ 
4 for  $i \leftarrow 1$  to  $n_1$ 
5 do  $L[i] \leftarrow A[p + i - 1]$ 
6 for  $j \leftarrow 1$  to  $n_2$ 
7 do  $R[j] \leftarrow A[q + j]$ 
8  $L[n_1 + 1] \leftarrow \infty$ 
9  $R[n_2 + 1] \leftarrow \infty$ 
10  $i \leftarrow 1$ 
11  $j \leftarrow 1$ 
12 for  $k \leftarrow p$  to  $r$ 
13 do if  $L[i] \leq R[j]$ 
14 then  $A[k] \leftarrow L[i]$ 
15  $i \leftarrow i + 1$ 
16 else  $A[k] \leftarrow R[j]$ 
17  $j \leftarrow j + 1$ 

```

Перед кожною ітерацією циклу for у рядках 12–17, підмасив $A[p..k - 1]$ містить $k - p$ найменших елементів масивів $L[1..n_1 + 1]$ та $R[1..n_2 + 1]$ в відсортованому порядку. Крім того, елементи $L[i]$ та $R[i]$ є найменшими елементами масивів L та R , які ще не скопійовано в масив A .

Необхідно показати, що цей інваріант циклу витримується перед першою ітерацією аналізованого циклу for, що кожна ітерація циклу не порушує його, і що з його допомогою вдається продемонструвати коректність алгоритму, коли цикл закінчує свою роботу.

Ініціалізація. Перед першою ітерацією циклу $k = p$, тому підмасив $A[p..k - 1]$ порожній. Він містить $k - p = 0$ найменших елементів масивів L та R . Оскільки $i = j = 1$, елементи $L[i]$ та $R[j]$ — найменші елементи масивів L та R , не скопійовані назад у масив A .

Збереження. Щоб переконатися, що інваріант циклу зберігається після кожної ітерації спочатку припустимо, що $L[i] \leq R[j]$. Тоді $L[i]$ найменший елемент, не скопійований до масиву A . Оскільки в підмасиві $A[p..k - 1]$ міститься $k - p$ найменших елементів, після виконання рядка 14, в якому значення елемента $L[i]$ присвоюється елементу $A[k]$, в підмасиві $A[p..k]$ буде міститись $k - p + 1$ найменший елемент. В результаті збільшення параметра k циклу for і значення змінної i (рядок 15), інваріант циклу відновлюється перед наступною ітерацією. Якщо ж виконується нерівність $L[i] > R[j]$, то в рядках 16 та 17 виконуються відповідні дії, під час яких також зберігається інваріант циклу.

Завершення. Алгоритм завершується, коли $k = r + 1$. Відповідно до інваріанту циклу, підмасив $A[p..k - 1]$ (тобто підмасив $A[p..r]$) містить

$k - p = r - p + 1$ найменших елементів масивів $L[1..n_1 + 1]$ та $R[1..n_2 + 1]$ у відсортованому порядку. Сумарна кількість елементів у масивах L та R дорів

ное $n_1 + n_2 + 2 = r - p + 3$. Всіх їх, крім двох найбільших, скопійовано назад в масив A , а два елементи, що залишилися, є сигнальними. Щоб показати, що час роботи процедури MERGE дорівнює $\Theta(n)$, де $n = r - p + 1$, зауважимо, що кожен із рядків 1–3 та 8–11 виконується протягом фіксованого часу; тривалість циклів for у рядках 4–7 дорівнює $\Theta(n_1 + n_2) = \Theta(n)$, а в циклі for у рядках 12–17 виконується n ітерацій, на кожену з яких витрачається фіксований час.

Тепер процедуру MERGE можна використовувати як підпрограму в алгоритмі сортування злиттям. Процедура MERGE_SORT(A, p, r) виконує сортування елементів підмасиву $A[p..r]$. Якщо справедлива нерівність $p \geq r$, то в цьому підмасиві міститься не більше одного елемента, і, отже, він відсортований. Інакше проводиться розбиття, під час якого обчислюється індекс q , що розділяє масив $A[p..r]$ на два підмасиви: $A[p..q]$ з $n/2$ елементами та $A[q..r]$ з $n/2$ елементами.

```

MERGE_SORT( $A, p, r$ )
1 if  $p < r$ 
2 then  $q \leftarrow (p + r)/2$ 
3 MERGE_SORT( $A, p, q$ )
4 MERGE_SORT( $A, q + 1, r$ )
5 MERGE( $A, p, q, r$ )

```

Щоб відсортувати послідовність $A = (A[1], A[2], \dots, A[n])$ викликається процедура MERGE_SORT($A, 1, \text{length}[A]$), де $\text{length}[A] = n$. На рис. 4.2 проілюстровано роботу цієї процедури у висхідному напрямку, якщо n – це ступінь двійки. У ході роботи алгоритму відбувається попарне об'єднання одноелементних послідовностей у відсортовані послідовності довжини 2, потім попарне об'єднання двоелементних послідовностей у відсортовані послідовності довжини 4 і т.д., поки не буде отримано дві послідовності, що складаються з $n/2$ елементів, які об'єднуються в кінцеву відсортовану послідовність довжини n .

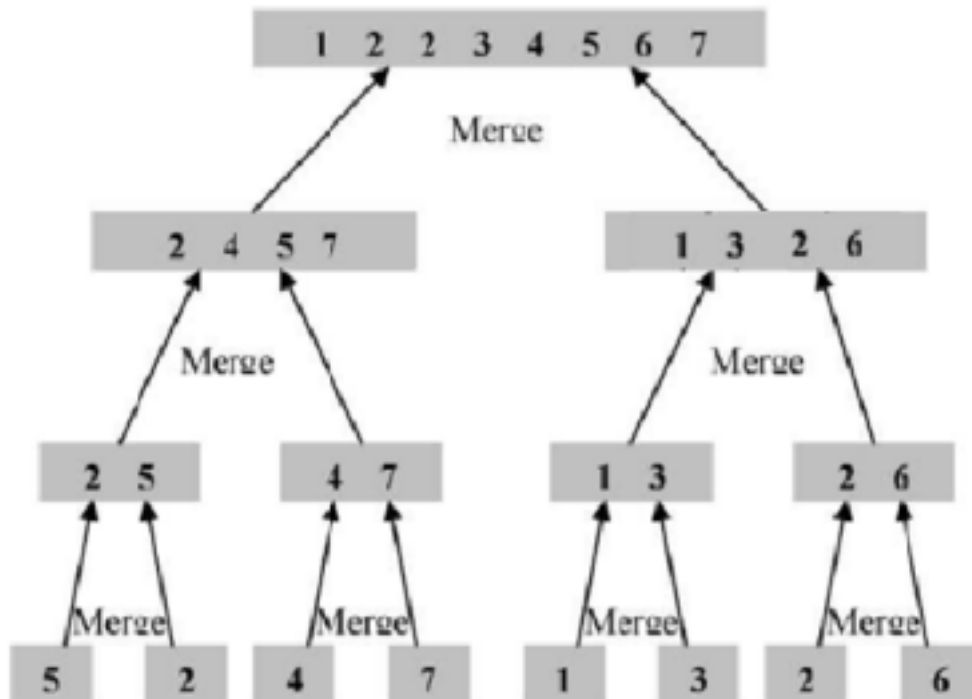


Рис. 4.2. Процес сортування масиву $A = \langle 5, 2, 4, 7, 1, 3, 2, 6 \rangle$

4.2 Аналіз алгоритмів, що реалізують метод декомозиції

Якщо алгоритм рекурсивно звертається до себе, час його роботи часто описується за допомогою рекурентного рівняння, або рекурентного співвідношення, в якому повний час, необхідний для вирішення всього завдання з обсягом введення n , виражається через час вирішення допоміжних підзадач. Потім дане рекурентне рівняння вирішується за допомогою певних математичних методів, і встановлюються межі продуктивності алгоритму.

Отримання рекурентного співвідношення для часу роботи алгоритму, заснованого на принципі "розділай і володарюй", базується на трьох етапах, що відповідають парадигмі цього принципу. Позначимо через $T(n)$ час розв'язання задачі, розмір якої дорівнює n . Якщо розмір завдання досить малий, скажімо, $n \leq c$ де c — деяка заздалегідь відома константа, то завдання вирішується безпосередньо протягом певного фіксованого часу, який ми позначимо через $\Theta(1)$. Припустимо, що завдання ділиться на b підзадач, обсяг кожної у тому числі дорівнює $1/b$ від обсягу вихідної задачі. Якщо розбиття завдання на допоміжні підзавдання відбувається протягом часу $D(n)$, а об'єднання рішень підзавдань у вирішення вихідної задачі - протягом часу $C(n)$, то ми отримаємо таке рекурентне співвідношення:

$$T(n) = \Theta(1) \text{ при } n \leq c,$$

$$T(n) = aT(n/b) + D(n) + C(n) \text{ в іншому випадку}$$

4.3 Аналіз алгоритму сортування злиттям

Щоб отримати рекурентне рівняння для верхньої оцінки часу роботи $T(n)$ алгоритму, що виконує сортування n чисел методом злиття, будемо міркувати так. Сортування одного елемента шляхом злиття триває протягом фіксованого часу. Якщо $n > 1$, час роботи розподіляється в такий спосіб.

Розбиття. У ході розбиття визначається, де знаходиться середина під масиву. Ця операція триває фіксований час, тому $D(n) = \Theta(1)$. **Підкорення.** Рекурсивно вирішуються дві підзадачі, обсяг кожної з яких становить $n/2$. Час розв'язання цих підзадач дорівнює $2T(n/2)$. **Комбінування.** Як уже згадувалося, процедура MERGE у n -елементному підмасиві виконується протягом часу $\Theta(n)$, тому $C(n) = \Theta(n)$. Склавши функції $D(n)$ і $C(n)$, отримаємо суму величин $\Theta(n)$ та $\Theta(1)$, яка є лінійною функцією від n , тобто $\Theta(n)$. Додаючи до цієї величини доданок $2T(n/2)$, що відповідає етапу "підкорення", отримаємо рекурентне співвідношення для часу роботи $T(n)$ алгоритму сортування за методом злиття у найгіршому випадку:

$$\begin{aligned} T(n) &= \Theta(1) \text{ при } n = 1, \\ T(n) &= 2T(n/2) + \Theta(n) \text{ при } n > 1 \end{aligned}$$

В [3] формально показано, що $T(n) = \Theta(n \lg n)$. Щоправда, можна інтуїтивно зрозуміти, що рішенням попереднього рекурентного співвідношення є вираз $T(n) = \Theta(n \lg n)$. Перепишемо його у такому вигляді:

$$\begin{aligned} T(n) &= c \text{ при } n = 1, \\ T(n) &= 2T(n/2) + cn \text{ при } n > 1, \end{aligned}$$

де константа c позначає час, який потрібен для вирішення задачі, розмір якої дорівнює 1, а також питомий (припадає на один елемент) час, необхідний для поділу та поєднання.

Процес вирішення рекурентного співвідношення проілюстровано на рис. 4.3, а) – г).

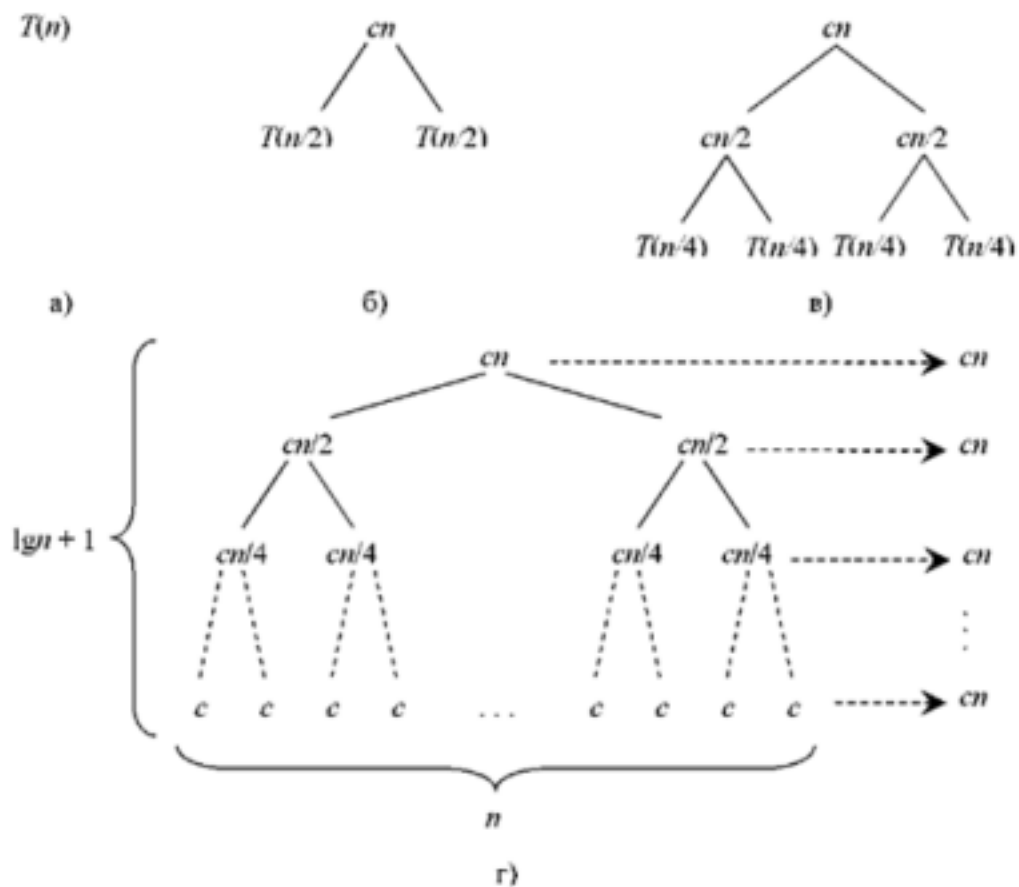


Рис. 4.3. Побудова дерева рекурсії для рівняння $T(n) = 2T(n/2) + cn$

Для зручності припустимо, що n дорівнює ступеню двійки. У частині а) згаданого малюнка показано час $T(n)$, представлений у частині б) у вигляді еквівалентного дерева, яке уособлює рекурентне рівняння. Коренем цього дерева є доданок cn (вартість верхнього рівня рекурсії), а два піддерева, що беруть початок від кореня, представляють дві менші рекурентні послідовності $T(n/2)$. У частині в) показаний черговий крок рекурсії. Час виконання кожного з двох підвузлів, що знаходяться на другому рівні рекурсії, дорівнює $cn/2$. Далі триває розкладання кожного вузла, що входить до складу дерева, шляхом розбиття їх на складові частини, визначені в рекурентній послідовності. Так відбувається доти, доки розмір задачі не стає рів

ним 1, а час його виконання — константі c . Дерево, що вийшло в результаті, показано в частині г). Дерево складається з $\lg n + 1$ рівнів (тобто його висота дорівнює $\lg n$), а кожен рівень дає вклад в повний час роботи, рівний cn . Таким чином, повний час роботи алгоритму дорівнює $cn \lg n + cn$, що відпо- відає $\Theta(n \lg n)$.

Після того як дерево збудовано, тривалості виконання всіх його вузлів підсумовуються за всіма рівнями. Повний час виконання верхнього рівня дорівнює cn , наступний рівень дає внесок, що дорівнює $c(n/2) + c(n/2) = cn$.

29

Ту ж величину внеску дають і всі наступні рівні. У загальному випадку рі- вень i (якщо вести відлік зверху) має 2^i вузлів, кожен з яких дає вклад у за- гальний час роботи алгоритму, рівний $c(n/2^i)$, тому повний час виконання всіх вузлів, що належать певному рівню, дорівнює $2^i c(n/2^i) = cn$. На ниж- ньому рівні є n вузлів, кожен із яких дає вклад c , що у сумі дає час, рівний cn .

Повна кількість рівнів дерева дорівнює $\lg n + 1$. Це легко зрозуміти із неформальних індуктивних міркувань. У найпростішому випадку, коли $n=1$, є лише один рівень. Оскільки $\lg 1 = 0$, вираз $\lg n + 1$ дає правильну кількість рівнів. Тепер як індуктивне припущення приймемо, що кількість рівнів ре- курсивного дерева з 2^i вузлами дорівнює $\lg 2^i + 1 = i + 1$ (оскільки для будь- якого i виконується співвідношення $\lg 2^i = i$). Оскільки ми припустили, що кількість вхідних елементів дорівнює ступеню двійки, тепер потрібно розг- лянути випадок для 2^{i+1} елементів. Дерево з 2^{i+1} вузлами має на один рівень більше, ніж дерево з 2^i вузлами, тому повна кількість рівнів дорівнює $(i+1) + 1 = \lg 2^{i+1} + 1$.

Щоб знайти повний час, що є рішенням рекурентного співвідношення, потрібно просто скласти внески від усіх рівнів. Усього є $\lg n + 1$ рівнів, ко- жен з яких виконується протягом часу cn , тому повний час дорівнює $cn(\lg n + 1) = cn \lg n + cn$. Нехтуючи членами нижчих порядків та константою c отримуємо $\Theta(n \lg n)$.

4.4 Рекурентні співвідношення

Рекурентні співвідношення мають надзвичайно важливе значення для програмування. Вони використовуються в аналізі алгоритмів, наближених обчисленнях, динамічному програмуванні, тощо [14].

Рекурентне співвідношення першого порядку

Нехай $\{a_k: k \geq 0\}$ - деяка послідовність дійсних чисел.

Означення 1. Послідовність $\{a_k: k \geq 0\}$ називається заданою рекурентним співвідношенням першого порядку, якщо явно задано її

перший член a_0 , а кожен наступний член a_k цієї послідовності визначається деякою залежністю через її попередній член a_{k-1} , тобто

$$a_0 = u, a_k = f(n, p, a_{k-1}), k \geq 0,$$

де u - задане (початкове) числове значення, p – деякий сталий параметр або набір параметрів, f – функція, задана аналітично у вигляді арифметичного виразу, що складається з операцій, доступних для виконання за допомогою мови програмування.

У рекурентних співвідношеннях m -го порядку k -й член послідовності визначається через m попередніх.

Рекурсія

Означення 2. Рекурсія – спосіб визначення об'єкту (або методу) попереднім описом одного чи кількох його базових випадків, з наступним описом на їхній основі загального правила побудови об'єкту.

30

Означення 3. Рекурсивна функція – метод визначення функції, при якому функція прямо або неявно викликає сама себе.

Як відомо, послідовність чисел Фібоначчі визначається рекурентним співвідношенням другого порядку

$$\begin{aligned} F_0 &= 1, F_1 = 1, \\ F_n &= F_{n-1} + F_{n-2}, n \geq 2. \end{aligned}$$

Отже, якщо покласти, що функція F_n визначає n -те число послідовності Фібоначчі, то з вищенаведеного рекурентного співвідношення отримаємо, що послідовність Фібоначчі може бути визначена рекурсивною функцією:

$$\begin{aligned} F(0) &= 1, F(1) = 1, \\ F(n) &= F(n-1) + F(n-2), n \geq 2. \end{aligned}$$

Початкові значення $F(0) = 1, F(1) = 1$, дуже важливі при визначенні рекурсивної функції. Якби їх не було, то рекурсія б стала нескінченною! Тому, описуючи рекурсивну функцію завжди треба переконуватися, що для всіх припустимих значень аргументів виклик рекурсивної функції завершиться, тобто що рекурсія буде скінченною.

Можна звернути увагу на те, що, на відміну від обчислення елементів послідовності заданої рекурентним співвідношенням, де обчислення відбувається від тривіального (початкового) елементу до шуканого, рекурсивна функція починає обчислення від шуканого.

Кількість вкладених викликів функції називається глибиною рекурсії. Наприклад, глибина рекурсії при знаходженні $F(5)$ буде 4. Наведемо стандартний підхід до опису рекурсивної функції. Структурно рекурсивна функція на верхньому рівні завжди є розгалуженням, що складається з двох або більше альтернатив, з яких

• принаймні одна є термінальною (припинення рекурсії); • принаймні одна є рекурсивною (тобто здійснює виклик себе з іншими аргументами).

4.5 Контрольні питання

У чому полягає основна ідея методу декмпозиції?

Назвіть основні етапи парадигми «розділяй та

володарюй». Що являє собою послідовність $a_0=1$; $a_k=a_{k-1} \cdot k$?

Дайте означення рекурентного співвідношення першого порядку.

Наведіть приклад рекурентного співвідношення другого порядку.

Дайте означення рекурсивної функції.

5 МЕТОД ЗАМІТАЮЧОЇ ПРЯМОЇ

5.1 Задача пошуку перетину відрізків

Проілюструємо цей метод на прикладі конкретної задачі: задано множину відрізків на площині, треба знайти всі їх перетини. Візьмемо вертикальну пряму l , яка розбиває дану площину на ліву та праву півплощини. Припустимо, що кожна з цих півплощин містить кінці заданих відрізків. Зрозуміло, що розв'язком задачі є об'єднання розв'язків для кожної з двох півплощин, тому, якщо припустити, що у нас вже є множина точок перетину ліворуч від l , то на цю множину не будуть впливати відрізки, розташовані праворуч від l [15].

Тепер зазначимо, що перетин може відбутися тільки між такими двома відрізками, чії перетини з деякою вертикаллю суміжні; отже, якщо побудувати всі вертикальні перерізи відрізків заданої множини, то будуть знайдені і всі шукані перетини. Але цієї задачі (що не має розв'язку) побудови (континуально) нескінченної множини всіх вертикалей, що перерізають множину відрізків, можна уникнути, якщо помітити, що площа розбивається на вертикальні смуги, обмежені або кінцями відрізків, або точками їх перетинів, а також, що вертикальний порядок точок перетину вздовж будь-якого перерізу в такій смузі не змінюється. Таким чином, все, що треба зробити – це стрибок з лівого кінця смуги на її правий кінець з оновленням порядку точок перетину вертикалі та пошуком нових перетинів серед «сусідніх» в цьому порядку відрізків.

Відзначимо суттєві риси метода плоского замітання. Є вертикаль, яка замітає площину зліва направо, зупиняючись в особливих точках, як на зведено «точками подій». Перетин замітаючої прямої з вхідними даними за

дачі містить все корисну для продовження пошуку інформацію. Отже, маємо дві основні структури:

1. Список точок подій – послідовність абсцис, впорядованих за неспаданням, які визначають позиції зупинок замітаючої прямої. 2. Статус замітаючої прямої - адекватний опис перетину цієї замітаючої прямої з геометричною структурою, що «замітається». «Адекватний» означає, що цей перетин містить інформацію, корисну для поставленої задачі. Статус замітаючої прямої оновлюється в кожній точці подій. Перед тим, як розглядати задачу пошуку перетину відрізків на площині, розглянемо більш просту (одновимірну) задачу. Нехай задано N інтервалів на дійсній осі. Треба визначити, чи перекриваються будь-які два з них. Інтервали задано значеннями своїх лівого та правого кінців.

Відповідь можна отримати за час $O(N^2)$, якщо перебрати всі пари інтервалів, але є кращий алгоритм, який базується на сортуванні. Якщо впорядкувати $2N$ кінцевих точок цих інтервалів і позначити їх як лівий (+1) або

32

правий (-1), то ці інтервали не перекриваються тоді і тільки тоді, коли кінці в послідовності йдуть строго по черзі: Л П Л П ... Л П (рис. 5.1).

Л П Л П Л П Л П

Рис. 5.1 Інтервали не перекриваються

Натомість, якщо два одноіменних кінці відрізків будуть йти підряд (Л Л або П П), це означатиме, що інтервали перекриваються (рис. 5.2).

Л Л П П Л Л П П Л Л П П

Рис. 5.2 Інтервали перекриваються

Цю перевірку можна провести за час $O(n \log n)$, якщо використати замітаючу пряму (тобто переглянути масив (вектор) відсортованих інтервалів в якому лівий кінець помічено як +1, а правий – як -1 і сумірюючи ці значення. Поява в сумі значення 2 означає, що інтервали перетинаються).

Задача пошуку відрізків, що перетинаються, по суті, є узгальненням розглянутої задачі пошуку відрізків, що перетинаються, на двовимірний випадок.

5.2 Застосування методу замітаючої прямої для перевірки

перетину відрізків

Дано n відрізків на площині. Потрібно перевірити, чи перетинаються один з одним хоча б два з них. (Якщо відповідь позитивна — то вивести, що перетинаються; інакше - ні)

Наївний алгоритм рішення — перебрати за $O(n^2)$ всі пари відрізків та перевірити для кожної пари перетинаються вони чи ні. Далі наводиться алгоритм з часом роботи $O(n \log n)$, який заснований на принципі скануючої (замітаючої) прямої (англійською: "sweep line").

Алгоритм

Проведемо подумки вертикальну пряму $x = -\infty$ і почнемо рухати цю пряму праворуч. По ходу свого руху ця пряма буде зустрічатися з відрізками, причому у будь-який момент часу кожен відрізок буде перетинатися з нашою прямою в одній точці (ми поки вважатимемо, що вертикальних відрізків немає).

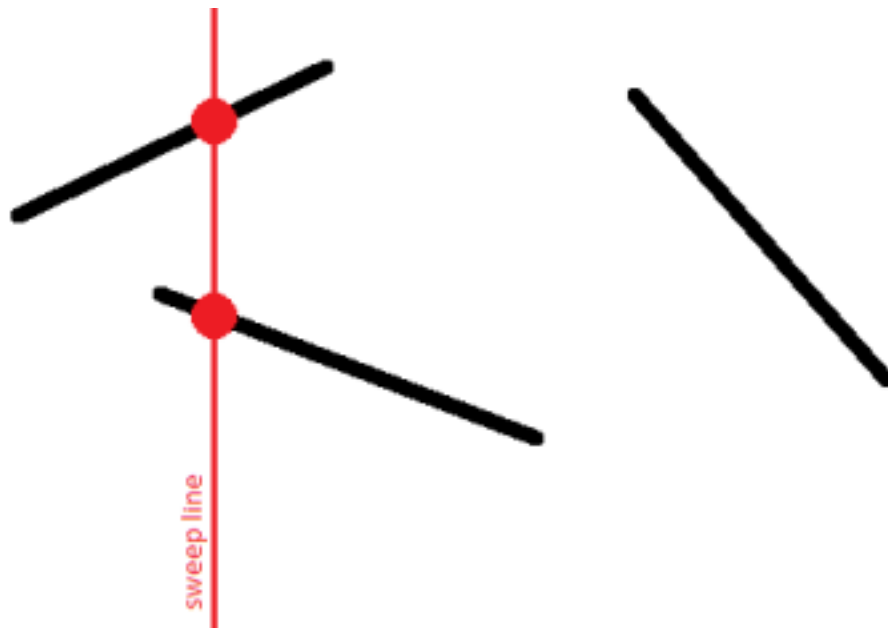


Рис. 5.3. Графічна ілюстрація метода замітаючої прямої

Таким чином, для кожного відрізка в якийсь момент часу його точка з'явиться на прямій, що сканує, потім з рухом прямої буде рухатися і ця точка, і, нарешті, в якийсь момент відрізок зникне з прямої.

Нас цікавить відносний порядок відрізків по вертикалі. А саме, ми зберігатимемо список відрізків, що перетинають скануючу пряму в даний момент часу, де відрізки будуть відсортовані за їх y -координатою на скануючій прямій (рис. 5.4).

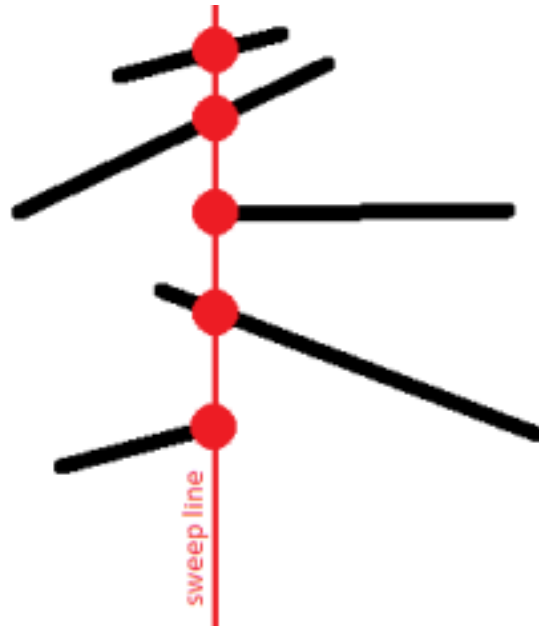


Рис. 5.4. Відрізки не перетинаються

34

Цей порядок цікавий тим, що відрізки, що перетинаються, матимуть однакову x -координату хоча б в один момент часу (рис. 5.5):

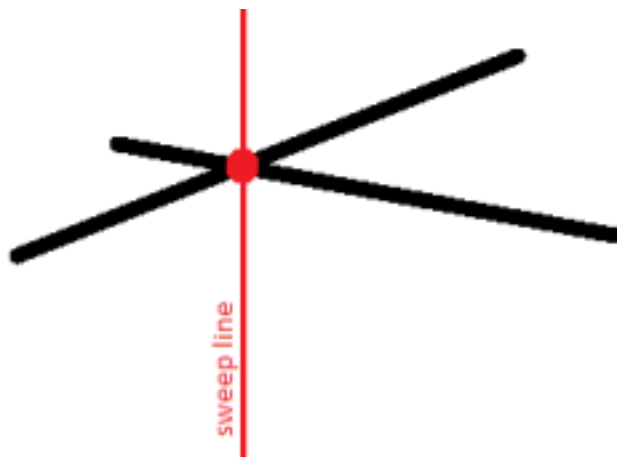


Рис. 5.5. Відрізки перетинаються

Сформулюємо ключові твердження:

- Для пошуку пари, що перетинається, достатньо розглядати при кожному фіксованому положенні скануючої прямої **тільки сусідні відрізки**.
- Достатньо розглядати скануючу пряму не у всіх можливих дійсних позиціях $(-\infty \dots +\infty)$, а **лише в тих позиціях, коли з'являються нові відрізки або зникають старі**. Інакше кажучи, досить обмежитися лише положеннями, рівними абсцисам точок-кінців відрізків.

- При появі нового відрізка достатньо **вставити** його в потрібне місце

в список, отриманий для попередньої прямої скануючої. Перевіряти на перетин треба **тільки відрізок, що додається, з його безпосередніми сусідами в списку зверху і знизу.**

- При зникненні відрізка достатньо **видалити** його зі списку. Після цього треба **перевірити на перетин між собою відрізки, які були його верхнім та нижнім сусідами** у списку.

- Інших змін у порядку проходження відрізків у списку, крім описаних, не існує. Інших перевірок на перетин робити не треба.

Для розуміння істинності цих тверджень достатньо таких зауважень: • Два відрізки, що не перетинаються, ніколи не змінюють свого відносного порядку.

Справді, якщо один відрізок спочатку був вищим за інший, а потім став нижчим, то між двома цими моментами стався перетин цих двох відрізків. • Мати однакові -координати два відрізки, що не перетинаються, та **У** кож не можуть.

- З цього випливає, що в момент появи відрізка ми можемо знайти в черзі позицію для цього відрізка, і більше цей відрізок переставляти в черзі не доведеться: його **порядок щодо інших відрізків у черзі не змінювати меться.**

35

- Два відрізки, що перетинаються, в момент точки свого перетину ви являться **сусідами** один одного в черзі.

- Отже, для знаходження пари відрізків, що перетинаються, достатньо перевірити на перетин лише ті пари відрізків, які коли-небудь за час руху скануючої прямої хоча б раз **були сусідами один одного.**

Легко помітити, що для цього достатньо лише перевіряти відрізок, що додається, зі своїми верхнім і нижнім сусідами, а також при видаленні відрізка - його верхнього і нижнього сусідів (які після видалення стануть сусідами один одного).

- Слід звернути увагу, що при фіксованому положенні скануючої прямої ми **спочатку** повинні зробити **додавання** всіх відрізків, що з'являються тут, і лише **потім - видалення** всіх відрізків, що зникають тут.

Тим самим ми не пропустимо перетину відрізків по вершині: тобто. такі випадки, коли два відрізки мають загальну вершину.

- Зауважимо, що вертикальні відрізки насправді не впливають на коректність алгоритму. Ці відрізки виділяються тим, що вони з'являються і зникають в той самий момент часу. Однак, за рахунок попереднього зауваження, ми знаємо, що спочатку всі відрізки будуть додані в чергу, і потім будуть видалені. Отже, якщо вертикальний відрізок перетинається з якимось іншим відкритим у цей момент відрізком (зокрема вертикальним), це буде виявлено.

У яке місце черги поміщати вертикальні відрізки? Адже вертикальний

відрізок не має однієї певної x -координати, він простягається на цілий відрізок по x -координаті. Однак легко зрозуміти, що як x -координату можна взяти будь-яку координату з цього відрізка.

Таким чином, весь алгоритм здійснить не більше $O(n)$ тестів на перетин пари відрізків, і здійснить $O(n^2)$ операцій з чергою відрізків (по $O(n)$ операцій в моменти появи та зникнення кожного відрізка).

Підсумкова асимптотика алгоритму становить $O(n^2)$.

5.3 Контрольні питання

Які основні структури даних потребує реалізація алгоритму перевірки перетину відрізків за методом замітаючої прямої?

Порівняйте ефективність алгоритму перевірки перетину відрізків за методом замітаючої прямої та «наївного» алгоритму.

Придумайте можливі застосування методу замітаючої прямої.

6 ХЕШУВАННЯ ДАНИХ

6.1 Загальні поняття про хешування даних

Для прискорення доступу до даних можна використовувати попереднє їх впорядкування згідно зі значеннями ключів. При цьому можуть використовуватися методи пошуку в упорядкованих структурах даних, наприклад, метод дихотомічного пошуку, що суттєво скорочує час пошуку даних за значенням ключа. Проте при додаванні нового запису потрібно дані знову впорядкувати. Втрати часу на повторне впорядкування можуть значно перевищувати виграш від скорочення часу пошуку. Тому для скорочення часу доступу до даних використовується так зване випадкове впорядкування або хешування. При цьому дані організуються у вигляді таблиці за допомогою хеш-функції $h()$, яка використовується для „обчислення” адреси за значенням ключа, як це представлено на рис. 6.1.

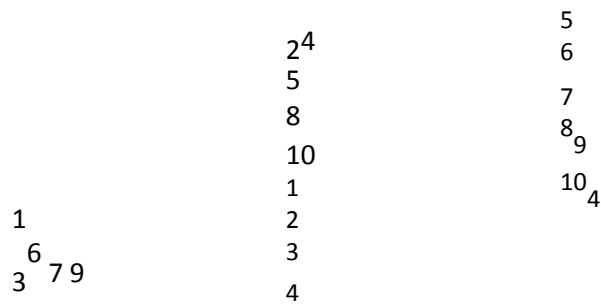


Рис. 6.1. Ілюстрація принципу хешування даних

Ідеальною хеш-функцією є така хеш-функція, яка для будь-яких двох неоднакових ключів дає неоднакові адреси. Підібрати таку функцію можна у випадку, якщо всі можливі значення ключів відомі наперед. Така організація даних носить назву „досконале хешування”.

У випадку наперед невизначеної множини значень ключів і обмеженні розміру таблиці підбір досконалої функції складний. Тому часто використовують хеш-функції, які не гарантують виконання умови [16].

6.2 Колізії та методи їх розв’язання

Очевидно, що, оскільки існує більше можливих значень ключа, ніж комірок в таблиці, то деякі значення ключів можуть відповідати одним і тим

37

самим коміркам таблиці. Даний випадок носить назву „колізія”, а такі ключі називаються „ключі-синоніми”.

Щоб уникнути цієї потенційної проблеми, схема хешування повинна включати в себе алгоритм розв’язання колізій, який визначає послідовність дій у випадку, якщо ключ відповідає позиції в таблиці, яка вже зайнята іншим записом.

Як працюють методи розв’язання колізій? Спочатку встановлюють відповідність між ключем запису і розміщенням в хеш-таблиці. Якщо ця комірка вже зайнята, вони відображають ключ на іншу комірку таблиці. Якщо вона також вже зайнята, процес повторюється знову до тих пір, поки нарешті алгоритм не знайде незайняту комірку в таблиці.

В результаті, для реалізації хешування необхідні три речі:

- структура даних (хеш-таблиця) для зберігання даних;
- функція хешування, яка встановлює відповідність між значеннями ключа і розміщенням в таблиці;
- алгоритм вирішення колізій, який визначає послідовність дій, якщо декілька ключів відповідають одній комірці таблиці.

Для розв’язання колізій використовуються різноманітні методи, ми обмежимося описом методів „ланцюжків” і „відкритої адресації”. **Методом ланцюжків** називається метод, в якому для розв’язання колізій в усі записи вводяться вказівники, які використовуються для організації списків – „ланцюжків переповнення”. У випадку виникнення колізій при заповненні таблиці в список для потрібної адреси хеш-таблиці додається ще один елемент (рис. 6.2).



Рис. 6.2. Ілюстрація розв’язання колізій методом ланцюжків

Пошук в хеш-таблиці з ланцюжками переповнення здійснюється таким чином. Спочатку обчислюється адреса за значенням ключа. Потім здійснюється послідовний пошук в списку, який зв’язаний з обчисленою адресою.

Процедура вилучення з таблиці зводиться до пошуку елемента та його вилучення з ланцюжка переповнення.

Якщо ланцюжки приблизно однакові за розміром, то у цьому випадку списки усіх ланцюжків повинні бути найбільш короткими при даній кількості ланцюжків. Якщо вихідна множина складається з N елементів, а кількість комірок в таблиці — M , тоді середня довжина списків буде дорівнювати $\alpha = N/M$ елементів. Якщо можна оцінити N і вибрати M якомога ближчим до цієї величини, то в списку буде один-два елементи. Тоді час доступу до елемента множини буде малою постійною величиною, яка залежить від N . В [3] показано, що середній час пошуку елемента в хеш-таблиці складає $\Theta(1+\alpha)$.

Одна з переваг цього методу хешування полягає в тому, що при його використанні хеш-таблиці ніколи не переповнюються. При цьому вставка і пошук елементів завжди виконується дуже просто, навіть якщо елементів у

таблиці дуже багато. Із хеш-таблиці, яка використовує ланцюжки, також просто вилучати елементи, при цьому елемент просто вилучається з відповідного зв'язного списку.

Один із недоліків зв'язування полягає в тому, що якщо кількість зв'язних списків недостатньо велика, то розмір списків може стати великим, при цьому для вставки чи пошуку елемента необхідно буде перевірити велику кількість елементів списку.

Метод відкритої адресації полягає в тому, щоб, користуючись якимось алгоритмом, який забезпечує перебір елементів таблиці, переглядати їх в пошуках вільного місця для нового запису.

З використанням методу відкритої адресації всі елементи зберігаються у хеш-таблиці, тобто. кожен запис таблиці містить або елемент динамічної множини, або значення NIL. При пошуку елемента ми систематично перевіряємо комірки таблиці до того часу, доки знайдемо шуканий елемент або переконаємося у його відсутності у таблиці. Тут, на відміну від методу ланцюжків, немає ані списків, ані елементів, що зберігаються поза таблицею. Таким чином, у методі відкритої адресації хеш-таблиця може бути заповненою, унеможливаючи вставку нових елементів; коефіцієнт заповнення не може перевищувати 1.

Звичайно, при хешуванні з розв'язанням колізій методом ланцюжків можна використовувати вільні місця в хеш-таблиці для зберігання пов'язаних списків, але перевага відкритої адресації полягає в тому, що вона дозволяє повністю відмовитися від вказівників. Замість слідування за вказівниками, ми обчислюємо послідовність комірок, що перевіряються. Додаткова пам'ять, що звільняється внаслідок відмови від вказівників, дозволяє використовувати хеш-таблиці більшого розміру при тій же загальній кількості пам'яті, потенційно призводячи до меншої кількості колізій та більш швидкій вибірці.

Для виконання вставки при відкритій адресації ми послідовно перевіряємо, або досліджуємо (probe), комірки хеш-таблиці до тих пір, поки не

39

знаходимо порожню комірку, в яку поміщаємо ключ, що вставляється. Замість фіксованого порядку дослідження комірок $0, 1, \dots, m-1$ (для чого потрібно $\Theta(n)$ часу), послідовність досліджуваних комірок залежить від ключа, що вставляється в таблицю. Для визначення досліджуваних комірок ми розширимо хеш-функцію, включивши до неї як другий аргумент номер дослідження (починається з 0).

В результаті хеш-функція стає такою:

$$h : U \times \{0, 1, \dots, m-1\} \rightarrow \{0, 1, \dots, m-1\}.$$

У методі відкритої адресації потрібно, щоб для кожного ключа k послідовність досліджень

$$h(k, 0), h(k, 1), \dots, h(k, m-1)$$

являла собою перестановку множини $0, 1, \dots, m-1$, щоб в результаті

могли бути переглянуті всі комірки хеш-таблиці.

Для обчислення послідовності досліджень для відкритої адресації за звичай використовуються три методи: лінійне дослідження, квадратичне дослідження та подвійне хешування. Ці методи гарантують, що для кожного ключа k $h(k, 0), h(k, 1), \dots, h(k, m - 1)$ є перестановкою $0, 1, \dots, m - 1$. Однак ці методи не задовольняють припущенню про рівномірне хешування, оскільки жоден з них не в змозі згенерувати більше m^2 різних послідовностей досліджень (замість $m!$, потрібних для рівномірного хешування). Найбільшу кількість послідовностей досліджень дає подвійне хешування і, як і слід очікувати, дає найкращі результати.

Лінійне дослідження зводиться до послідовного перебору елементів таблиці з деяким фіксованим кроком. При кроці рівному одиниці відбувається послідовний перебір усіх елементів після поточного.

Квадратичне дослідження відрізняється від лінійного тим, що крок перебору елементів нелінійно залежить від номеру спроби знайти вільний елемент. Завдяки нелінійності такої адресації зменшується кількість спроб при великій кількості ключів-синонімів. Проте навіть відносно невелика кі-

лькість спроб може швидко привести до виходу за адресний простір невеликої таблиці внаслідок квадратичної залежності адреси від номеру спроби. Ще один різновид методу відкритої адресації, яка називається **подвійним хешуванням**, базується на нелінійній адресації, яка досягається за рахунок сумування значень основної і додаткової хеш-функцій. Очевидно, що в міру заповнення хеш-таблиці будуть відбуватися колізії і в результаті їх розв'язання методами відкритої адресації чергова адреса може вийти за межі адресного простору таблиці. Щоб це явище відбувалося рідше можна піти на збільшення розмірів таблиці у порівнянні з діапазоном адрес, які обчислюються хеш-функцією.

З однієї сторони це приведе до скорочення кількості колізій і прискоренню роботи з хеш-таблицею, а з іншої – до нераціональних витрат адресного простору. Навіть при збільшенні таблиці в два рази у порівнянні з областю значень хеш-функції нема гарантій того, що в результаті колізій адреса не перевищить розмір таблиці. При цьому в початковій частині таблиці

може залишатися достатньо вільних елементів. Тому на практиці використовують циклічний перехід на початок таблиці.

Розглядаючи можливість виходу за межі адресного простору таблиці, ми не враховували фактори наповненості таблиці й вдалого вибору хеш-функції. При великій наповненості таблиці виникають часті колізії і циклічні переходи на початок таблиці. При невдалому виборі хеш-функції відбуваються аналогічні явища. В найгіршому випадку при повному заповненні таблиці алгоритми циклічного пошуку вільного місця приведуть до зациклю

вання. Тому при використанні хеш-таблиць необхідно старатися уникати дуже густого заповнення таблиць. Звичайно довжину таблиці вибирають із розрахунку дворазового перевищення передбачуваної максимальної кількості записів. Не завжди при організації хешування можна правильно оцінити потрібну довжину таблиці, тому у випадку великої наповненості таблиці може знадобитися рехешування. У цьому випадку збільшують довжину таблиці, змінюють хеш-функцію і впорядковують дані.

Проводити окрему оцінку густини заповнення таблиці після кожної операції вставки недоцільно, тому можна проводити таку оцінку непрямым способом – за кількістю колізій під час однієї вставки. Достатньо визначити деякий поріг кількості колізій, при перевищенні якого потрібно провести рехешування. Крім того, така перевірка гарантує неможливість зациклювання алгоритму у випадку повторного перегляду елементів таблиці.

6.3 Побудова хеш-функції

Як вже відзначалося, дуже важливим є правильний вибір хеш-функції. При вдалій побудові хеш-функції таблиця заповнюється більш рівномірно, зменшується кількість колізій і зменшується час виконання операцій пошуку, вставки і вилучення. Для того щоб попередньо оцінити якість хеш-функції можна провести імітаційне моделювання [17].

Якісна хеш-функція задовольняє (приблизно) припущенню простого рівномірного хешування: для кожного ключа рівноймовірно розміщення в будь-яку з m комірок, незалежно від хешування інших ключів. На жаль, цю умову зазвичай неможливо перевірити, оскільки, як правило, розподіл ймовірностей, відповідно до якого надходять ключі, що вносяться до таблиці, невідомий; крім того, ключі, що вставляються, можуть не бути незалежними.

На практиці при побудові якісних хеш-функцій найчастіше використовуються різні евристичні методики. У процесі побудови велику допомогу надає інформація про розподіл ключів. Розглянемо, наприклад, таблицю символів компілятора, у якій ключами служать символні рядки, які мають ідентифікатори у програмі. Найчастіше в одній програмі зустрічаються схожі ідентифікатори, наприклад `pt` та `pts`. Хороша хеш-функція повинна мінімізувати шанси попадання цих ідентифікаторів до одної комірки хеш-таблиці.

При побудові хеш-функції хорошим підходом є вибір функції таким чином, щоб вона ніяк не корелювала із закономірностями, яким можуть відповідати наявні дані. Наприклад, метод поділу обчислює хеш-значення як залишок від поділу ключа на деяке просте число. Якщо це просте число не пов'язане з розподілом вихідних даних, метод часто дає хороші результати. На закінчення зауважимо, деякі застосування хеш-функцій можуть накла

дати додаткові вимоги, крім вимог простого рівномірного хешування. Наприклад, ми можемо вимагати, щоб “близькі” в деякому сенс ключі давали далекі хеш-значення.

Побудова хеш-функції методом ділення

Побудова хеш-функції методом ділення полягає у відображенні ключа k в одну з комірок шляхом отримання залишку від ділення k на m , тобто. хеш-функція має вигляд $h(k) = k \bmod m$.

Наприклад, якщо хеш-таблиця має розмір $m = 12$, а значення ключа $k = 100$ то $h(k) = 4$. Оскільки для обчислення хеш-функції потрібна лише одна операція ділення, хешування шляхом ділення вважається досить швидким.

При використанні даного методу ми зазвичай намагаємося уникати деяких значень m . Наприклад, m не повинно бути ступенем 2, оскільки якщо $m = 2^p$, то $h(k)$ є просто p молодших бітів числа k . Якщо тільки заздалегідь не відомо, що всі набори молодших p бітів ключів рівномірні, краще будувати хеш-функцію таким чином, щоб її результат залежав від усіх бітів ключа.

Найчастіше хороші результати можна отримати, вибираючи як значення m просте число, досить далеке від ступеня двійки. Припустимо, наприклад, що ми хочемо створити хеш-таблицю з розв’язанням колізій методом ланцюжків для зберігання $n = 2000$ символьних рядків, розмір символів яких дорівнює 8 бітам. Нас влаштовує перевірка в середньому трьох елементів при невдалому пошуку, так що ми вибираємо розмір таблиці рівним $m=701$. Число 701 вибрано як просте число, близьке до величини $2000/3$ і не є ступенем 2. Розглядаючи кожен ключ k як ціле число, ми отримуємо шу кану хеш-функцію:

$$h(k) = k \bmod 701.$$

6.4 Універсальне хешування

Якщо зловмисник буде навмисно вибирати ключі для хешування за допомогою конкретної хеш-функції, то він зможе підібрати n значень, які будуть хешуватися в одну і ту саму комірку таблиці, приводячи до середнього часу вибірки $\Theta(n)$. Таким чином, будь-яка фіксована хеш-функція стає вразливою, і єдиний ефективний вихід із ситуації — випадковий вибір хеш-функції, який залежить від того, з якими саме ключами їй належить працювати. Такий підхід, який називається універсальним хешуванням, гарантує хоча б продуктивність у середньому, незалежно від того, які дані будуть обрані зловмисником.

Головна ідея універсального хешування полягає у випадковому виборі хеш-функції з деякого ретельно відібраного класу функцій на початку роботи програми. Як і у разі швидкого сортування, рандомізація гарантує, що одні й ті самі вхідні дані неспроможні постійно давати найгіршу поведінку

алгоритму. В силу рандомізації алгоритм працюватиме щоразу по-різному, навіть для тих самих вхідних даних, що гарантує високу середню продуктивність для будь-яких вхідних даних.

Нехай H — кінцева множина хеш-функцій, що відображають простір ключів U в діапазон $\{0, 1, 2, \dots, m-1\}$. Така множина називається універсальною, якщо для кожної пари різних ключів $k, l \in U$ кількість хеш-функцій $h \in H$, для яких $h(k) = h(l)$, не перевищує $|H|/m$. Іншими словами, при випадковому виборі хеш-функції H ймовірність колізії між різними ключами k і l не перевищує ймовірності збігу двох випадковим чином обраних хеш-значень з множини $\{0, 1, 2, \dots, m-1\}$, що дорівнює $1/m$.

Побудова універсальної множини хеш-функцій

Побудувати таку множину досить просто, що впливає з теорії чисел. Почнемо з вибору простого числа p , досить великого, щоб усі можливі ключі перебували в діапазоні від 0 до $p-1$ включно. Нехай Z_p позначає множину $\{0, 1, \dots, p-1\}$, а Z_p^* — множину $\{1, 2, \dots, p-1\}$. Оскільки p — просте число, ми можемо розв'язувати рівняння по модулю p . З припущення, що простір ключів більше, ніж кількість комірок у хеш-таблиці, впливає, що $p > m$.

Тепер визначимо хеш-функцію $h_{a,b}$ для будь-яких $a \in Z_p^*$ і $b \in Z_p$ таким чином:

$$h_{a,b}(k) = ((ak + b) \bmod p) \bmod m.$$

Наприклад, при $p = 17$ та $m = 6$ $h_{3,4}(8) = 5$. Сімейство всіх таких функцій утворює множину

$$H_{p,m} = \{h_{a,b} : a \in Z_p^* \text{ і } b \in Z_p\}$$

Кожна хеш-функція $h_{a,b}$ відображає Z_p на Z_m . Цей клас хеш-функцій має ту властивість, що розмір вихідного діапазону m довільний і не обов'язково являє собою просте число. Оскільки число a можна вибрати $p-1$ способом, і p способами — число b , всього у множині $H_{p,m}$ міститься $p(p-1)$ хеш-функцій.

6.5 Ідеальне хешування

Хоча найчастіше хешування використовується через високу середню продуктивність, можлива ситуація, коли реально отримати високу продуктивність хешування в найгіршому випадку. Такою ситуацією є статична множина ключів, тобто після того, як всі ключі збережені в таблиці, їх множина ніколи не змінюється. Ряд застосунків у силу своєї природи працює зі статичними множинами ключів. Як приклад можна навести множину зарезервованих слів мови програмування або множину імен файлів на компакт-

падку виконує пошук за $O(1)$ звернень до пам'яті.

Основна ідея ідеального хешування досить проста. Ми використовуємо дворівневу схему хешування з універсальним хешуванням на кожному рівні. Перший рівень по суті той же, що і у разі хешування з ланцюжками: n ключів хешуються в m комірок з використанням хеш-функції h , ретельно обраної із сімейства універсальних хеш-функцій. Однак замість того, щоб створювати список ключів, хешованих в комірку j , ми використовуємо маленьку вторинну хеш-таблицю S_j зі своєю хеш-функцією h_j . Шляхом точного вибору хеш-функції h_j ми можемо гарантувати відсутність колізій на другому рівні.

6.4 Контрольні питання

Що являє собою ключ у хеш-таблиці?

Дайте визначення поняття колізії.

У чому полягає розв'язання колізій методом ланцюжків?

Які різновиди методу відкритої адресації Ви знаєте?

Які вимоги можуть висуватися до хеш-функції?

Охарактеризуйте побудову хеш-функції методом ділення.

Що зумовлює необхідність універсального хешування?

Як відбувається побудова універсальної множини хеш-функцій?

Що таке ідеальне хешування?

У яких випадках можливе ідеальне хешування?

7 БІНАРНІ ДЕРЕВА ПОШУКУ

7.1 Поняття про бінарні дерева пошуку

Дерева пошуку являють собою структури даних, які підтримують багато операцій з динамічними множинами, включаючи пошук елемента, мінімального та максимального значення, попереднього та наступного елемента, вставку та видалення. Таким чином, дерево пошуку може використовуватися і як словник, і як черга з пріоритетами.

Основні операції в бінарному дереві пошуку виконуються за час, пропорційний його висоті. Для повного бінарного дерева з n вузлами ці операції виконуються за час $\Theta(\lg n)$ у найгіршому випадку. Математичне очікування висоти побудованого випадковим чином бінарного дерева дорівнює $O(\lg n)$, так що всі основні операції над динамічною множиною в такому дереві виконуються в середньому за час $\Theta(\lg n)$.

Як випливає з назви, бінарне дерево пошуку в першу чергу є бінарним деревом, як показано на рис.7.1. Таке дерево може бути представлено за допомогою зв'язаної структури даних, в якій кожен вузол є об'єктом. На додаток до полів ключа `key` та супутніх даних, кожен вузол містить поля `left`, `right` та `p`, які вказують на лівий та правий дочірні вузли та на батьківський вузол відповідно. Якщо дочірній або батьківський вузол відсутній, поле має значення `NULL`. Єдиний вузол, показник `p` якого дорівнює `NULL`, — це ко

ренивий вузол дерева. Ключі в бінарному дереві пошуку зберігаються таким чином, щоб будь-якої миті задовольняти таку властивість бінарного дерева пошуку:

Якщо x — вузол бінарного дерева пошуку, а вузол y знаходиться у лівому піддереві x , то $key[y] \leq key[x]$. Якщо вузол y знаходиться у правому піддереві x , то $key[x] \leq key[y]$.

Так, на рис. 7.1 а) ключ кореня дорівнює 5, ключі 2, 3 та 5, які не перевищують значення ключа в корені, знаходяться в його лівому піддереві, а ключі 7 і 8, які не менше, ніж ключ 5, — в його правому піддереві. Та ж властивість, як легко переконатись, виконується для кожного іншого вузла дерева. На рис. 7. 1 б) показано дерево з тими ж вузлами і тими ж властивостями, проте менш ефективне в роботі, оскільки його висота дорівнює 4, на відміну від дерева на рис. 7.1 а), висота якого дорівнює 2.

45

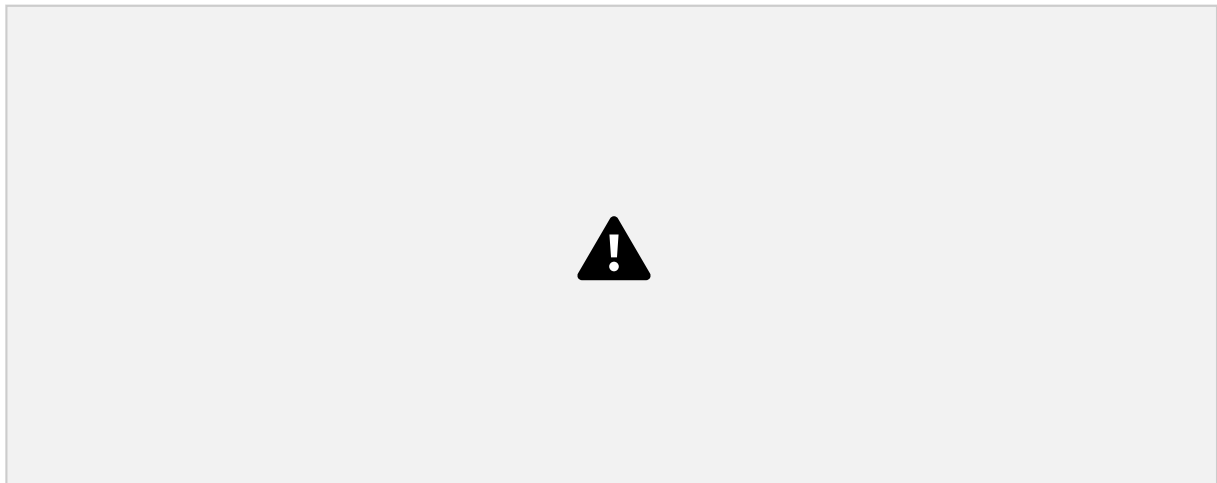


Рис. 7.1 Бінарні дерева пошуку

Властивість бінарного дерева пошуку дозволяє нам вивести всі ключі, що знаходяться у дереві, у відсортованому порядку за допомогою простого рекурсивного алгоритму, що називається центрованим (симетричним) обходом дерева (inorder tree walk). Цей алгоритм отримав дану назву у зв'язку з тим, що ключ у корені піддерева виводиться між значеннями ключів лівого піддерева та правого піддерева. Є й інші способи обходу, саме — обхід у порядку (preorder tree walk), у якому спочатку виводиться корінь, а потім — значення лівого і правого піддерева, і обхід у зворотному порядку

(postorder tree walk), коли першими виводяться значення лівого і правого піддерева, а вже потім кореня. Центрований обхід дерева T реалізується процедурою INORDER_TREE_WALK(root [T]):

```
INORDER_TREE_WALK(x)
1 if x != NULL
2 then INORDER_TREE_WALK(left[x])
3 print key[x]
4 INORDER_TREE_WALK(right[x])
```

7.2 Робота з бінарним деревом пошуку

Найбільш поширеною операцією, що виконується з бінарним деревом пошуку, є пошук у ньому певного ключа. Крім того, бінарні дерева пошуку підтримують такі запити, як пошук мінімального та максимального елемента, а також попереднього та наступного. Всі вони можуть бути виконані в бінарному дереві пошуку заввишки h за час $O(h)$.

Пошук

Для пошуку вузла із заданим ключем у бінарному дереві пошуку використовується наступна процедура TREE_SEARCH, яка отримує як параметр покажчик на корінь бінарного дерева x і ключ k , а повертає покажчик на вузол з цим ключем (якщо такий існує; інакше повертається значення NULL).

46

```
TREE_SEARCH(x, k)
1 if x == NULL чи k == key[x]
2 then return x
3 If k < key[x]
4 then return TREE_SEARCH(left[x], k)
5 else return TREE_SEARCH(right[x], k)
```

Процедура пошуку починається з кореня дерева та проходить вниз по дереву. Для кожного вузла x на шляху вниз його ключ $key[x]$ порівнюється з переданим як параметр ключом k . Якщо ключі однакові, пошук буде завершено. Якщо k менше $key[x]$, пошук продовжується у лівому піддереві x ; якщо більше - то пошук переходить у праве піддерево. Так, на рис. 2 для пошуку ключа 13 ми повинні пройти наступний шлях від кореня: $15 \rightarrow 6 \rightarrow 7 \rightarrow 13$. Вузли, які ми відвідуємо при рекурсивному пошуку, утворюють низхідний шлях від кореня дерева, тому час роботи процедури TREE_SEARCH дорівнює $O(h)$, де h — висота дерева. Ту ж процедуру можна записати ітеративно, "розвертаючи" кінцеву рекурсію в цикл while. На більшості комп'ютерів така версія виявляється більш ефективною. Графічну ілюстрацію процедури пошуку у бінарному дереві наведено на рис. 7.2.

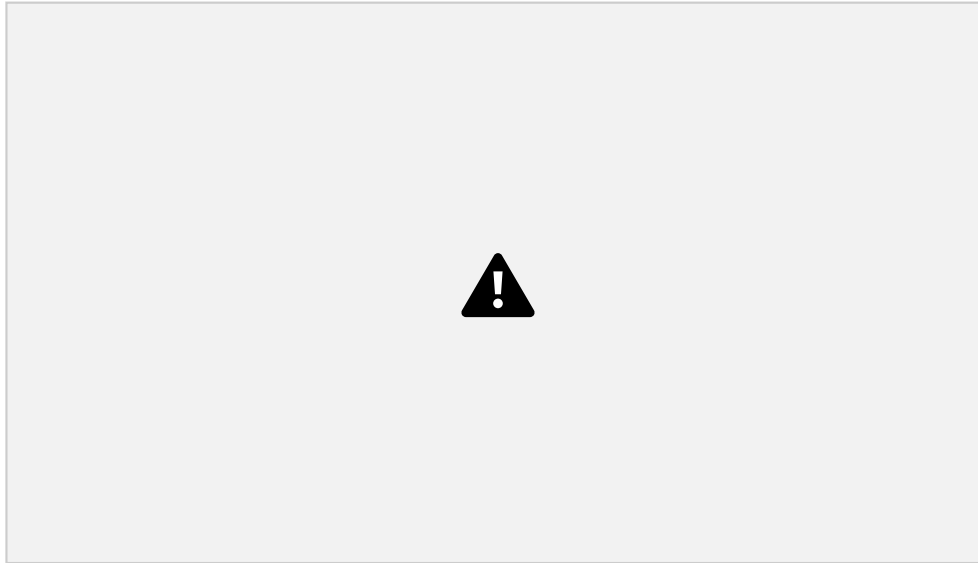


Рис. 7.2. Запити в бінарному дереві пошуку

Так виглядає псевдокод рекурсивної процедури пошуку:

```
ITERATIVE_TREE_SEARCH(x, k)
1 while x != NULL та k != key[x]
2 do if k < key[x]
3 then x ← left[x]
4 else x ← right[x]
5 return x
```

47

Пошук мінімуму та максимуму

Елемент з мінімальним значенням ключа легко знайти, слідуючи за вказівниками left від кореневого вузла до того часу, доки не зустрінеться значення NULL. Так, на рис.2, слідуючи за вказівниками left, ми пройдемо шлях $15 \rightarrow 6 \rightarrow 3 \rightarrow 2$ до мінімального ключа в дереві, що дорівнює 2. Ось як виглядає реалізація описаного алгоритму:

```
TREE_MINIMUM(x)
1 while left[x] != NULL
2 do x ← left[x]
3 return x
```

Властивість бінарного дерева пошуку гарантує правильність процедури TREE_MINIMUM. Якщо у вузла x немає лівого піддерева, то, оскільки всі ключі в правому піддереві x не менше ключа key [x], мінімальний ключ піддерева з коренем у вузлі x знаходиться у цьому вузлі. Якщо ж у вузла є ліве піддерево, то, оскільки в правому піддереві не може бути вузла з ключем, меншим key[x], а всі ключі у вузлах лівого піддерева не перевищують

key[x], вузол з мінімальним значенням ключа знаходиться в піддереві, коренем якого є вузол left[x].

Алгоритм пошуку максимального елемента дерева симетричний алгоритму пошуку мінімального елемента:

```
TREE_MAXIMUM(x)
1 while right[x] != NГДЛ
2 do x ← right[x]
3 return x
```

Представлені процедури знаходять мінімальний (максимальний) елемент дерева за час $O(h)$, де h — висота дерева, оскільки, як і в процедурі TREE_SEARCH, послідовність вузлів, що перевіряються, утворює низхідний шлях від кореня дерева.

Вставка

Для вставки нового значення v у бінарне дерево пошуку T ми скористаємося процедурою TREE_INSERT. Процедура отримує як параметр вузол z , у якого $\text{key}[z] = v$, $\text{left}[z] = \text{NULL}$ і $\text{right}[z] = \text{NULL}$, після чого вона таким чином змінює T та деякі поля z , що z виявляється вставленим у відповідну позицію у дереві.

```
TREE_INSERT(T, z)
1 y ← NULL
2 x ← root[T]
3 while x != NULL
4 do y ← x
5 if key[z] < key[x]
6 then x ← left[x]
7 else x ← right[x]
8 p[z] ← y
9 if y = NULL
10 then root[T] ← z // Дерево T — пусте
11 else if key[z] < key[y]
12 then left[y] ← z
13 else right[y] ← z
```

48

На рис. 7.3 показано роботу процедури TREE_INSERT. Подібно до процедур TREE_SEARCH і ITERATIVE_TREE_SEARCH, процедура TREE_INSERT починає роботу з кореневого вузла дерева і проходить низхідним шляхом. Показчик x відзначає прохідний шлях, а показчик y вказує на батьківський по відношенню до x вузол. Після ініціалізації цикл while у рядках 3–7 переміщує ці вказівники вниз по дереву, переміщаючись ліворуч чи праворуч залежно від результату порівняння ключів $\text{key}[x]$ і $\text{key}[z]$, доки x не стане рівним NULL. Це значення знаходиться саме у тій позиції, куди слід помістити елемент z . У рядках 8–13 виконується встановлення значень показчиків

ків для вставки z . Так само, як і інші примітивні операції над бінарним деревом пошуку, процедура `TREE_INSERT` виконується за час $O(h)$ у дереві заввишки h .

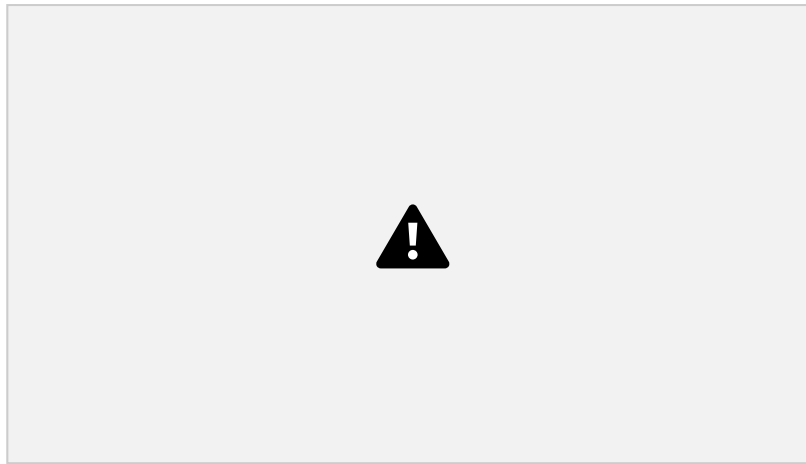


Рис.7.3. Вставка елемента з ключем 13 в бінарне дерево пошуку. Світлі вузли вказують шлях від кореня до позиції вставки; пунктиром вказано зв'язок, що додається при вставці нового елемента

Видалення

Процедура видалення даного вузла z з бінарного дерева пошуку отри має як аргумент покажчик на z . Процедура розглядає три можливі ситуації.. Якщо у вузла z немає дочірніх вузлів, ми просто змінюємо його батьківський вузол $p[z]$, замінюючи у ньому покажчик на z значенням `NIL`. Якщо у вузла z лише один дочірній вузол, ми видаляємо вузол z , створюючи новий зв'язок між батьківським і дочірнім вузлом вузла z . І нарешті, якщо у вузла

49

z два дочірні вузли, то ми знаходимо наступний за ним вузол y , у якого не має лівого дочірнього вузла, прибираємо його з позиції, де він знаходився раніше, шляхом створення нового зв'язку між його батьком і нащадком, і замінюємо їм вузол z .

Псевдокод процедури `TREE_DELETE` реалізує видалення вузла з бінарного дерева пошуку.

```
TREE_DELETE(T, z)
1 if left[z] = NIL или right[z] = NIL
2 then y ← z
3 else y ← TREE_SUCCESSOR(z)
4 if left[y] ≠ NIL
5 then x ← left[y]
6 else x ← right[y]
7 if x ≠ NIL
8 then p[x] ← p[y]
```

```

9 if p[y] = NIL
10 then root[T] ← x
11 else if y = left[p[y]]
12 then left[p[y]] ← x
13 else right[p[y]] ← x
14 if y ≠ z
15 then key[z] ← key[y]
16 // Копіювання супутніх даних в z
17 return y

```

У рядках 1–3 алгоритм визначає шляхом “склейки” батька та нащадка вузол y , що видаляється з дерева. Цей вузол є або вузол z (якщо у вузла z не більше одного дочірнього вузла), або вузол, що йде за вузлом z (якщо у z два дочірніх вузла). Потім у рядках 4–6 x надається покажчик на дочірній вузол вузла y або значення NIL, якщо у y немає дочірніх вузлів. Потім вузол y забирається з дерева в рядках 7–13 шляхом зміни покажчиків у $p[y]$ та x . Це видалення ускладнюється необхідністю коректного відпрацювання граничних умов (коли x дорівнює NIL або коли y кореневий вузол). І нарешті, у рядках 14–16, якщо видалений вузол y був наступним за z , ми перезаписуємо ключ z та супутні дані ключем та супутніми даними y . Видалений вузол y повертається в рядку 17, щоб викликаюча процедура могла за необхідності звільнити або використовувати пам'ять, яку він займає. Час роботи описаної процедури з деревом заввишки h становить $O(h)$.

Випадкова побудова бінарного дерева пошуку

Всі базові операції з бінарними деревами пошуку мають час виконання $O(h)$, де h – висота дерева. Однак при вставці та видаленні елементів висота дерева змінюється. Якщо, наприклад, всі елементи вставляються у дерево в строго зростаючій послідовності, то таке дерево вироджується в ланцюжок висоти $n - 1$. З іншого боку, $h \geq \lg n$. Як і у разі швидкого сортування, можна

50

показати, що поведінка алгоритму в середньому набагато ближче до кращого випадку, ніж найгіршого.

У ситуації, коли при формуванні бінарного дерева пошуку використовуються і вставки, і видалення, про середню висоту дерев, що утворюються відомо мало, тому обмежимося аналізом ситуації, коли дерево будується лише з використанням вставок, без видалень. Визначимо випадкове бінарне дерево пошуку (randomly built binary search tree) з n ключами як дерево, що виникає при вставці ключів у порожнє дерево у випадковому порядку, коли всі $n!$ перестановок входних ключів рівноймовірні. В [3] показано, що математичне очікування висоти випадкового бінарного дерева пошуку з n ключами дорівнює $O(\log n)$.

7.3 Порівняння хеш-таблиць і бінарних дерев пошуку

Швидкість хеш-таблиці в основному залежить від вибору функції хешування, оскільки колізії сповільнюють доступ до необхідних елементів. Операції з хеш-таблицею, такі як вставка, видалення та пошук, надзвичайно швидкі та займають у середньому $O(1)$.

Однак, якщо у нас дуже погана функція хешування, яка завжди вказує на одну й ту саму комірку таблиці, найгірше, що ми можемо отримати, є $O(n)$. У цьому випадку ми перебираємо всі елементи, як у масиві або зв'язаному списку. Ключові показники порівняння збалансованого бінарного дерева пошуку і хеш-таблиці наведено на рис. 7.4 [18].



Рис. 7.4. Порівняння бінарного дерева пошуку і хеш-таблиці

Інша ситуація, коли для хеш-таблиці потрібен лінійний час $O(n)$, якщо розмір таблиці перевищує початково виділений розмір. Тоді рехешування має створити більшу таблицю та переробити кожен елемент у ній. Це робить хеш-таблицю не дуже придатною для роботи з постійно зростаючими “живими” даними.

Часова складність для тих самих операцій вставки, пошуку та видалення в самобалансуючому бінарному дереві пошуку в середньому становить $O(\log(n))$. Це значно повільніше порівняно з хеш-таблицею, якщо розглядати лише ці три операції.

Тим не менш, якщо програма вимагає більш складних завдань, таких як пошук найближчих менших або більших елементів до числа або виконання запитів діапазону, тоді відсортована структура двійкового дерева пошуку виконає пошук набагато швидше порівняно з хеш-таблицею.

Двійкові дерева пошуку, як правило, ефективно використовують **пам'ять**, оскільки вони не резервують більше пам'яті, ніж потрібно. З іншого боку, хеш-таблиці можуть бути дещо більш вимогливими, якщо ми не знаємо точну кількість елементів, які хочемо зберегти.

Зазвичай хеш-таблиця виділяє масив і уникає колізій шляхом рівномірного хешування за розміром цього масиву. Отже, якщо ми маємо хеш-таблицю з початково зарезервованою пам'яттю для 100 елементів, але використовуємо лише 20, решта пам'яті буде просто витрачено.

7.4 Контрольні питання

Поясніть відмінність між бінарними деревами та бінарними деревами пошуку.

У якому порядку відбувається обхід бінарного дерева пошуку?

Які операції у бінарних деревах пошуку Ви знаєте?

Яка складність операції пошуку вузла у бінарному дереві пошуку?

Яка складність операції вставки вузла у бінарне дерево пошуку? Яка складність операції видалення вузла з бінарного дерева пошуку? У чому полягають переваги бінарних дерев пошуку порівняно з хеш таблицями?

У чому полягають недоліки бінарних дерев пошуку порівняно з хеш таблицями?

8 ОЦІНЮВАННЯ СКЛАДНОСТІ АЛГОРИТМІВ. АЛГОРИТМИ СОРТУВАННЯ

Розглядаючи вхідні дані досить великих розмірів для оцінювання тільки такої величини, як порядок зростання часу роботи алгоритму, ми тим самим вивчаємо **асимптотичну** ефективність алгоритмів. Це означає, що нас цікавить лише те, як час роботи алгоритму зростає зі збільшенням роз

міру вхідних даних у межі, коли цей розмір прагне нескінченності. Зазвичай алгоритм, ефективніший в асимптотичному сенсі, буде більш продуктивним для всіх вхідних даних, крім дуже маленьких (хоча бувають і винятки).

Хоча в багатьох випадках ці позначення використовуються неформально, корисно почати з точних визначень.

8.1 Θ -позначення

Нехай час $T(n)$ роботи деякого алгоритму на входах довжини $n \in \Theta(n^2)$. Точний зміст цього твердження такий: знайдуться такі константи $c_1, c_2 > 0$ і таке число n_0 , що $c_1 n^2 \leq T(n) \leq c_2 n^2$ для всіх $n \geq n_0$. Взагалі, якщо $g(n)$ – деяка функція, то запис $f(n) = \Theta(g(n))$ означає, що знайдуться такі $c_1, c_2 > 0$ і таке n_0 , що $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$ для всіх $n \geq n_0$.

Функція $f(n)$ належить множині $\Theta(g(n))$, якщо існують позитивні константи c_1 і c_2 , що дозволяють укласти цю функцію в межі між функціями $c_1 g(n)$ і $c_2 g(n)$ для досить великих n . Оскільки $\Theta(g(n))$ — це нескінченність, то можна написати “ $f(n) \in \Theta(g(n))$ ”. Це означає, що функція $f(n)$ належить множині $\Theta(g(n))$ (іншими словами, є його елементом). Але найчастіше використовують еквівалентний запис “ $f(n) = \Theta(g(n))$ ”. Але таке позначення варто вживати з обережністю: встановивши, що $f_1(n) = \Theta(g(n))$ і $f_2(n) = \Theta(g(n))$, не слід робити висновок, що $f_1(n) = f_2(n)$.

На рис.8.1 показано інтуїтивне зображення функцій $f(n)$ і $g(n)$, таких що $f(n) = \Theta(g(n))$. Для всіх значень n , що лежать праворуч від n_0 , функція $f(n)$ більше або дорівнює функції $c_1 g(n)$, але не перевищує функції $c_2 g(n)$. Іншими словами, для всіх $n \geq n_0$ функція $f(n)$ дорівнює функції $g(n)$ з точністю до постійного множника. Говорять, що функція $g(n)$ є асимптотично точною оцінкою функції $f(n)$.



Рис. 8.1. Ілюстрація оцінювання складності алгоритмів

Покажемо для прикладу, що $(1/2)n^2 - 3n = \Theta(n^2)$. Відповідно до визначення треба вказати додатні константи c_1 , c_2 і число n_0 так, щоб нерівності

$$c_1 n^2 \leq 1/2 n^2 - 3n \leq c_2 n^2$$

виконувалися для всіх $n \geq n_0$. Розділимо дані нерівності на n^2 :

$$c_1 \leq 1/2 - 3/n \leq c_2.$$

Бачимо, що для виконання другої нерівності досить покласти $c_2 = 1/2$. Перше буде виконано, якщо, наприклад, $n_0 = 7$ і $c_1 = 1/14$. Звичайно, константи можна вибрати по-іншому, проте важливо не те, як їх вибрати, а те, що така можливість існує.

Інший приклад використання формального визначення: покажемо, що $6n^3 \neq \Theta(n^2)$. Справді, нехай існують такі c_2 і n_0 , що $6n^3 \leq c_2 n^2$ для всіх $n \geq n_0$. Але тоді $n \leq c_2/6$ для всіх $n \geq n_0$ – що явно не так.

Визначаючи асимптотично точну оцінку для суми, можна відкидати члени меншого порядку, що при великих n стають малими порівняно з основним складовим. Зазначимо також, що коефіцієнт при старшому члені ролі не відіграє (він може вплинути тільки на вибір констант c_1 і c_2).

Згадаємо важливий окремий випадок використання Θ -позначень: $\Theta(1)$ – означає обмежену функцію, відокремлену від нуля деякою додатною константою при досить великих значеннях аргументу.

8.2 O - і Ω - позначення

Запис $f(n) = \Theta(g(n))$ містить дві оцінки: верхню і нижню. Їх можна розділити. Говорять, що $f(n) = O(g(n))$, якщо знайдеться така константа $c > 0$ і таке число n_0 , що $0 \leq f(n) \leq cg(n)$ для всіх $n \geq n_0$.

O-позначення застосовуються, коли потрібно вказати верхню межу функції з точністю до постійного множника. Інтуїтивне уявлення про O-позначення дозволяє отримати рис.1, б. Для всіх n , що лежать праворуч від n_0 , значення функції $f(n)$ не перевищує значення функції $cg(n)$.

Щоб вказати, що функція $f(n)$ належить множині $O(g(n))$, пишуть $f(n)=O(g(n))$. Зверніть увагу, що з $f(n) = \Theta(g(n))$ випливає $f(n) = O(g(n))$, оскільки Θ -позначення сильніші, ніж O-позначення. В значеннях теорії множин $\Theta(g(n)) \subset O(g(n))$.

Щоб записати час роботи алгоритму в O-позначеннях, часто-густо достатньо вивчити його загальну структуру. Наприклад, наявність подвійного вкладеного циклу в структурі алгоритму сортування за методом вставок свідчить про те, що верхня межа часу роботи в найгіршому випадку виражається як $O(n^2)$: “вартість” кожної ітерації у внутрішньому циклі обмежена зверху константою $O(1)$, індекси i та j — числом n , а внутрішній цикл виконується щонайбільше один раз для кожної з n^2 пар значень i та j .

Оскільки O-позначення описують верхню межу, то в ході їх використання для обмеження часу роботи алгоритму у найгіршому випадку ми отримуємо верхню межу цієї величини для будь-яких вхідних даних. Таким чином, межа $O(n^2)$ для часу роботи алгоритму в найгіршому випадку може застосовуватися для часу розв'язання задачі з будь-якими вхідними даними, чого не можна сказати про Θ -позначення. Наприклад, оцінка $\Theta(n^2)$ для часу сортування вставкою у найгіршому випадку не застосовується для довільних вхідних даних. Наприклад, якщо вхідні елементи вже відсортовані, час роботи алгоритму сортування вставкою оцінюється як $\Theta(n)$.

Кажуть, що $f(n)=\Omega(g(n))$, якщо знайдеться така константа $c>0$ і таке число n_0 , що $0 \leq cg(n) \leq f(n)$ для всіх $n \geq n_0$. Інтуїтивне уявлення про Ω -позначення дозволяє отримати рис.1, в. Для всіх n , що лежать праворуч від n_0 , значення функції $f(n)$ більші або дорівнюють $cg(n)$.

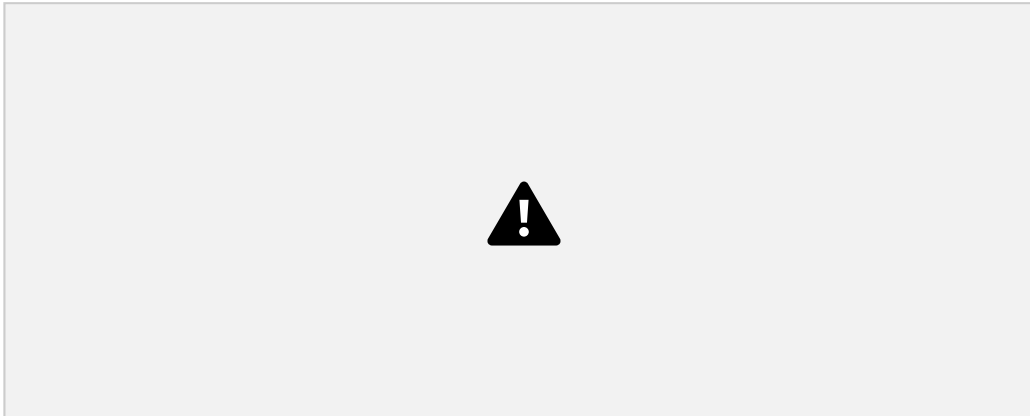
8.3 Алгоритми сортування

Сортування за квадратичний час. Варіації на тему «сортування бульбашкою»

Сортування вставкою було розглянуто раніше (див. розділ 1). Розберемо алгоритм, у якому обмін місцями двох елементів є характерною особливістю процесу. Викладений нижче алгоритм ґрунтується на порівнянні й зміні місць для пари сусідніх елементів і продовженні цього процесу доти, поки не будуть упорядковані всі елементи (тому даний алгоритм отримав ще назву „сортування прямим обміном”). І такі проходи по масиву слід повторювати, пересуваючи щоразу найменший елемент послідовності, яка залишилась, до лівого кінця масиву. Якщо розглядати масиви як вертикальні, а не горизонтальні, то елементи можна інтерпретувати як бульбашки у чані з водою, причому вага кожного відповідає його ключу.

Тут під час кожного проходу одна бульбашка „піднімається” до рівня, що відповідає її вазі (табл.8.1).

Таблиця 8.1. Сортування бульбашкою



Псевдокод, що реалізує такий підхід у найпростішому вигляді, наведено нижче:

```
BUBBLESORT(A)
1 for i ← 2 to n
2 do for j ← n downto i
3 do if A[j-1] > A[j] then
4 виконати обмін A[j] ↔ A[j-1]
```

Поліпшення цього алгоритму очевидне. На прикладі у табл. 1 бачимо, що три останніх проходи не впливають на порядок елементів, оскільки вони вже відсортовані. Отже, слід запам'ятовувати, мали місце перестановки під час деякого проходу або ні. І якщо у черговому проході перестановок не було – алгоритм можна завершувати.

Зазначимо, що це поліпшення можна знову ж поліпшити, якщо запам'ятовувати не лише факт обміну, а й індекс k останнього обміну. Цілком зрозуміло, що усі пари сусідніх елементів вище цього індексу вже відсортовані. Тому перегляд можна закінчувати на цьому індексі, а не йти до зазда легідь визначеної нижньої межі для i .

Крім того, тут можна помітити ще деяку своєрідну асиметрію: „спливає” бульбашка на потрібне місце за один прохід, а „тоне” на потрібне місце дуже повільно – лише на один крок за один прохід. Наприклад, масив 12 18 42 44 55 67 94 06

за допомогою удосконаленого бульбашкового сортування можна упорядкувати за один прохід, а для масиву

94 06 12 18 42 44 55 67

потрібно сім проходів. Така асиметрія наводить на думку про третє

поліпшення: послідовно чергувати напрямки проходів. Алгоритм, що дозволяє реалізувати таке поліпшення називають шейкерним сортуванням

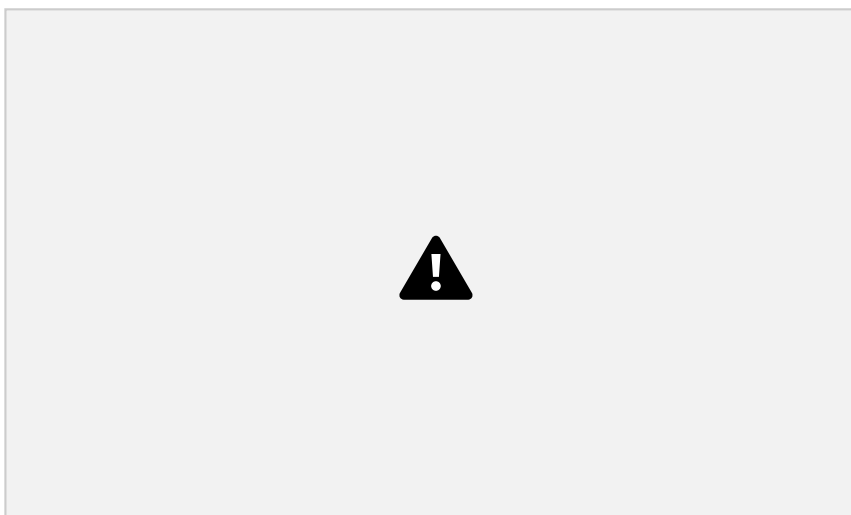
56

(ShakerSort). Псевдокод, що дозволяє реалізувати алгоритм шейкерного сортування наведено нижче (тут змінні L , R , k – позначають ліву та праву границі ділянки сортованого масиву, а також індекс останнього обміну, відповідно). А таблиця 8.2 ілюструє процес сортування тих же (табл. 8.1) восьми ключів.

SHAKERSORT(A)

```
1  $L \leftarrow 2$ 
2  $R \leftarrow n$ 
3  $k \leftarrow n$ 
4 repeat
5   for  $j \leftarrow R$  downto  $L$ 
6   do if  $A[j-1] > A[j]$  then
7     виконати обмін  $A[j] \leftrightarrow A[j-1]$ 
8    $k \leftarrow j$ 
9    $L \leftarrow k+1$ 
10  for  $j \leftarrow L$  to  $R$ 
11  do if  $A[j-1] > A[j]$  then
12    виконати обмін  $A[j] \leftrightarrow A[j-1]$ 
13   $k \leftarrow j$ 
14   $R \leftarrow k-1$ 
15 until  $L < R$ 
```

Таблиця 8.2. Приклад шейкерного сортування



Сортування за псевдолінійний час. Алгоритм Хоара

Сортування злиттям було розглянуто раніше. Розглянемо, напевно, кращий з практичної точки зору алгоритм, який носить назву швидкого сортування, або алгоритм Хоара.

Алгоритм Хоара - це «розділяй і володарюй» алгоритм. Як і всі алгоритми «розділяй і володарюй», він спочатку ділить великий масив на два менші підмасиви, а потім рекурсивно сортує підмасиви. По суті, весь процес включає три етапи:

Вибір опорного елемента: Виберіть елемент, що називається опорним, з масиву (зазвичай це найлівіший чи найправіший чи середній елемент роз ділу).

Поділ: Переупорядкуйте масив таким чином, щоб усі елементи зі значеннями менше опорного розташовувалися перед опорним. Навпаки, всі елементи зі значеннями більше ніж точка опори йдуть після неї. Рівні значення можуть у будь-якому напрямі. Після цього розбиття опорний елемент займає своє кінцеве становище.

Повторювати: Рекурсивно застосуйте описані вище кроки до підмасиву елементів із меншими значеннями, ніж у опорного, та окремо до підмасиву елементів із більшими значеннями, ніж у опорного.

Базовим випадком рекурсії є масиви розміром 1, які ніколи не потрібно сортувати. На наступній діаграмі показано, як ми вибираємо крайній правий елемент як опорний на кожному етапі алгоритму Хоара, розбиваємо масив по опорному елементу та повторюємося у двох підмасивах, які ми отримуємо після процесу поділу.

Зверніть увагу, що етапи вибору опори та розбиття можуть бути виконані декількома способами і вибір конкретних схем реалізації істотно впливає на продуктивність алгоритму. Сортування масиву $A = \langle 9, -3, 5, 2, 6, 8, -6, 1, 3 \rangle$ за алгоритмом Хоара наведено на рис. 8.2:

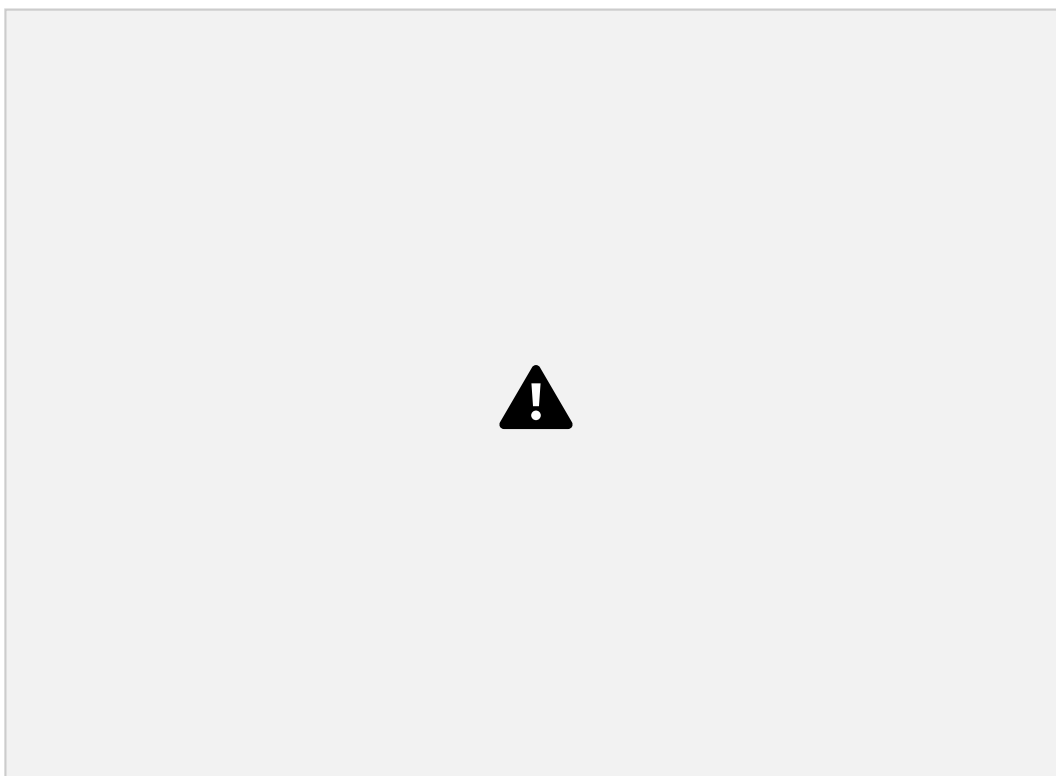
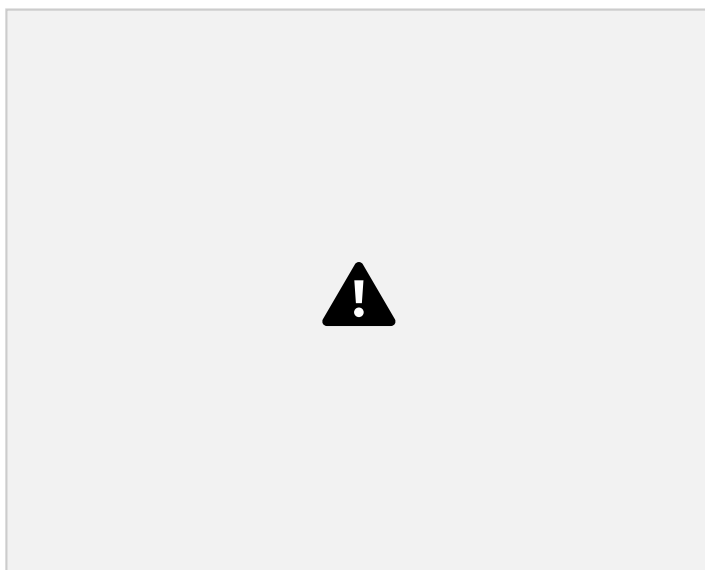
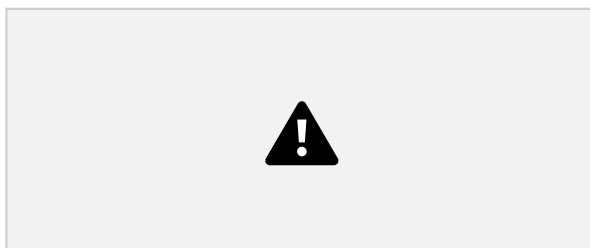


Рис. 8.2. Сортування за алгоритмом Хоара

58

Нижче наведено псевдокод для алгоритму Хоара та основної процедури сортування PARTITION.



8.4 Контрольні питання

Поясніть значення терміна «асимптотична ефективність алгоритмів».
У чому різниця між Θ -, O - і Ω -позначеннями?
Які алгоритми сортування за квадратичний час Ви знаєте? Як
можна вдосконалили алгоритм сортування бульбашкою?
Пояніть значення терміну «сортування за псевдолінійний час».
Які алгоритми сортування за псевдолінійний час Ви знаєте?

9 ГРАФИ

9.1 Подання графів

Граф – це складна нелінійна багатозв’язна динамічна структура, що ві
дображає властивості і зв’язки складного об’єкту.

Ця багатозв’язна структура має такі властивості:

- на кожний елемент (вузол, вершину) може бути довільна кількість по
силань;
- кожний елемент може мати зв’язок з будь-якою кількістю інших еле
ментів;
- кожний зв’язок (ребро, дуга) може мати напрям і вагу.

У вузлах графа міститься інформація про елементи об’єкту. Зв’язки
між вузлами задаються ребрами графа. Ребра графа можуть мати
спрямованість, тоді вони називаються орієнтованими, в іншому випадку –
неорієнтовані. Граф, усі зв’язки якого орієнтовані, називається
орієнтованим графом; граф зі всіма неорієнтованими зв’язками –
неорієнтованим графом; граф із зв’яз
ками обох типів – змішаним графом.

Існує два основні методи подання графів в пам’яті комп’ютера: мат ричний
і зв’язними нелінійними списками. Вибір методу подання залежить від
природи даних і операцій, що виконуються над ними. Якщо задача ви магає

великої кількості включень і виключень вузлів, то доцільно подавати граф зв'язними списками; інакше можна застосувати і матричне подання.

При використанні матриць суміжності їхні елементи подаються в пам'яті комп'ютера елементами масиву. При цьому, для простого графа матриця складається з нулів і одиниць, для мультиграфа – з нулів і цілих чисел, які вказують кратність відповідних ребер, для зваженого графа – з нулів і дійсних чисел, які задають вагу кожного ребра.

Орієнтований граф подається зв'язним нелінійним списком, якщо він часто змінюється або якщо півміри входу і виходу його вузлів великі.

Графом називається сукупність двох множин: непорожньої множини V (множини вершин) та множини E неупорядкованих пар різних елементів множини V (E – множина ребер).

На відміну від дерев, графи можуть містити цикли та петлі (ребра, які з'єднують вершину саму з собою).

Для практичного застосування необхідно розробити подання графів у пам'яті комп'ютера, а також забезпечити реалізацію таких дій: - обхід графа;

- пошук довжин найкоротших шляхів від заданої вершини графа (дже рела) до всіх інших вершин графа.

Є два стандартних способи подання графа $G = (V, E)$: як набору списків суміжних вершин або як матриці суміжності. Обидва способи подання за-

стосовні як орієнтованих, так неорієнтованих графів. Зазвичай краще подання з допомогою списків суміжності, оскільки воно забезпечує компактне подання розріджених (sparse) графів, тобто. таких, котрим $|E|$ набагато менше $|V|^2$.

Більшість алгоритмів припускають, що вхідний граф подається у вигляді списку суміжності. Подання за допомогою матриці суміжності краще у разі щільних (dense) графів, тобто коли значення $|E|$ близько до $|V|^2$, або коли нам треба мати можливість швидко визначити, чи існує ребро, що з'єднує дві дані вершини. Подання графа $G = (V, E)$ як списку суміжності (adjacency-list representation) використовує масив Adj з $|V|$ списків, по одному для кожної вершини з V . Для кожної вершини $u \in V$ список $Adj[u]$ містить усі вершини v , такі що $(u, v) \in E$, тобто $Adj[u]$ складається з усіх вершин, суміжних з u у графі G (список може містити не самі вершини, а покажчики на них). Вершини у кожному списку зазвичай зберігаються у до вільному порядку. На рис. 9.1 б) показано таке подання графа, зображеного на рис. 9.1 а) (на рис. 9.1 в) наведено його матрицю суміжності).

Аналогічно, на рис. 9.2 б) та 9.2 в) показано список і матриця суміжності орієнтованого графа, зображеного на рис. 9.2 а).



Рис. 9.1. Два подання неорієнтованого графа



Рис. 9.2. Два подання орієнтованого графа

Якщо G - орієнтований граф, то сума довжин всіх списків суміжності дорівнює $|E|$, оскільки ребру (u, v) однозначно відповідає елемент v списку $\text{Adj}[u]$. Якщо G — неорієнтований граф, то сума довжин усіх списків сумі-

61

жності дорівнює $2|E|$, оскільки ребро (u, v) , будучи неорієнтованим, з'являється у списку $\text{Adj}[v]$ як u , та у списку $\text{Adj}[u]$ — як v . Як для орієнтованих, так і для неорієнтованих графів подання у вигляді списків вимагає обсяг пам'яті, що дорівнює $\Theta(V + E)$.

Списки суміжності легко адаптуються для подання зважених графів (weighted graph), тобто графів, з кожним ребром яких пов'язана певна вага (weight), зазвичай визначається ваговою функцією (weight function) $w: E \rightarrow \mathbb{R}$.

Наприклад, нехай $G = (V, E)$ - зважений граф з ваговою функцією w . Вага $w(u, v)$ ребра $(u, v) \in E$ просто зберігається разом з вершиною v у списку суміжності u . Подання за допомогою списків суміжності досить гнучко у тому сенсі, що легко адаптується для підтримки багатьох інших варіантів графів.

Потенційний недолік подання за допомогою списків суміжності полягає в тому, що при цьому немає швидшого способу визначити, чи є дане ребро (u, v) у графі, ніж пошук v у списку $\text{Adj}[u]$. Цей недолік можна усунути ціною використання асимптотично більшої кількості пам'яті та подання графа за допомогою матриці суміжності.

Подання графа $G = (V, E)$ за допомогою матриці суміжності (adjacency

matrix representation) передбачає, що вершини перенумеровані в деякому по рядку числами $1, 2, \dots, |V|$. У разі подання графа G з використанням матриці суміжності є матрицю $A = (a_{ij})$ розміром $|V| \times |V|$, таку що

$$a_{ij} = \begin{cases} 1, & \text{якщо } (v_i, v_j) \in E \\ 0 & \text{в протилежному випадку.} \end{cases}$$

На рис. 9.1 в) і 9.2 в) показано подання з використанням матриці суміжності неорієнтованого та орієнтованого графів, показаних на рис. 1а та 2а відповідно. Матриця суміжності графа вимагає обсяг пам'яті, що дорівнює $\Theta(V^2)$, незалежно від кількості ребер графа.

Хоча список суміжності асимптотично, як мінімум, настільки ж ефективний в плані пам'яті, як і матриця суміжності, простота останньої робить її кращою при роботі з невеликими графами. Крім того, у випадку незважених графів для подання одного ребра достатньо одного біта, що дозволяє суттєво заощадити необхідну для подання пам'ять.

9.2 Пошук в ширину

Пошук в ширину (breadth-first search, BFS) є одним із найпростіших алгоритмів для обходу графа і є основою багатьох важливих алгоритмів для роботи з графами. Наприклад, алгоритм Прима (Prim) пошуку мінімального кістяка або алгоритм Дейкстри (Dijkstra) пошуку найкоротшого шляху з однієї вершини використовують ідеї, подібні до ідей, що використовуються при пошуку в ширину.

Нехай заданий граф $G = (V, E)$ та виділена вихідна (source) вершина s . Алгоритм пошуку в ширину систематично обходить усі ребра G для “відк-

62

риття” всіх вершин, досяжних з s , обчислюючи при цьому відстань (мінімальна кількість ребер) від s до кожної досяжної s вершини. Крім того, в процесі обходу будується “дерево пошуку в ширину” з коренем s , що містить усі досяжні вершини. Для кожної досяжної з s вершини v шлях у дереві пошуку в ширину відповідає найкоротшому (тобто містить найменшу кількість ребер) шляху від s до v G . Алгоритм працює як для орієнтованих, так і для неорієнтованих графів.

Пошук в ширину має таку назву тому, що в процесі обходу ми йдемо вшир, тобто перед тим як розпочати пошук вершин на відстані $k + 1$, виконується обхід всіх вершин з відстанню k .

Для відстеження роботи алгоритму пошук в ширину розфарбовує вершини графа в білий, сірий і чорний кольори. Спочатку всі вершини білі, і вони можуть стати сірими, а потім чорними. Коли вершина відкривається (discovered) у процесі пошуку, вона фарбується. Таким чином, сірі та чорні вершини – це вершини, які вже були відкриті, але алгоритм пошуку в ши

рину по-різному працює з ними, щоб забезпечити оголошений порядок обходу. Якщо $(u, v) \in E$ і вершина u чорного кольору, то вершина або сіра, або чорна, тобто всі вершини, суміжні із чорною, вже відкриті. Сірі вершини можуть мати білих сусідів, являючи собою межу між відкритими та невідкритими вершинами.

Пошук в ширину будує дерево пошуку в ширину, яке спочатку складається з одного кореня, яким є вихідна вершина s . Якщо в процесі сканування списку суміжності відкритої вершини u відкривається біла вершина v , то ребро (u, v) додається в дерево. Ми говоримо, що u є попередни ком (predecessor), або батьком (parent), v у дереві пошуку вшир. Оскільки вершина може бути відкрита не більше одного разу, вона має не більше од ного батька. Взаємини предків і нащадків визначаються в дереві пошуку в ширину як завжди — якщо u перебуває в шляху від кореня s до вершини v , то u є предком v , а v — нащадком u .

Нижче наведено процедуру пошуку в ширину BFS, яка передбачає, що вхідний граф $G = (V, E)$ представлений за допомогою списків суміжності. Крім того, підтримуються додаткові структури даних у кожній вершині графа. Колір кожної вершини $u \in V$ зберігається в змінній $color[u]$, а попередник - у змінній $\pi[u]$. Якщо попередника у u немає (наприклад, якщо $u=s$ чи u не відкрита), то $\pi[u] = NIL$. Відстань від s до вершини u , що обчислюється алгоритмом, зберігається в полі $d[u]$. Алгоритм використовує чергу Q для роботи з множиною сірих вершин:

BFS(G, s)

```

1 for (для) кожної вершини  $u \in V[G] - s$ 
2 do  $color[u] \leftarrow WHITE$ 
3  $d[u] \leftarrow \infty$ 
4  $\pi[u] \leftarrow NIL$ 
5  $color[s] \leftarrow GRAY$ 
```

63

```

6  $d[s] \leftarrow 0$ 
7  $\pi[s] \leftarrow NIL$ 
8  $Q \leftarrow \emptyset$ 
9 ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11 do  $u \leftarrow DEQUEUE(Q)$ 
12 for (для) кожної  $v \in Adj[u]$ 
13 do if  $color[v] = WHITE$ 
14 then  $color[v] \leftarrow GRAY$ 
15  $d[v] \leftarrow d[u] + 1$ 
16  $\pi[v] \leftarrow u$ 
17 ENQUEUE( $Q, v$ )
18  $color[u] \leftarrow BLACK$ 
```

Для кращого розуміння роботи BFS проілюструємо його роботу покроково (рис. 9.3 – 9.9) [19].

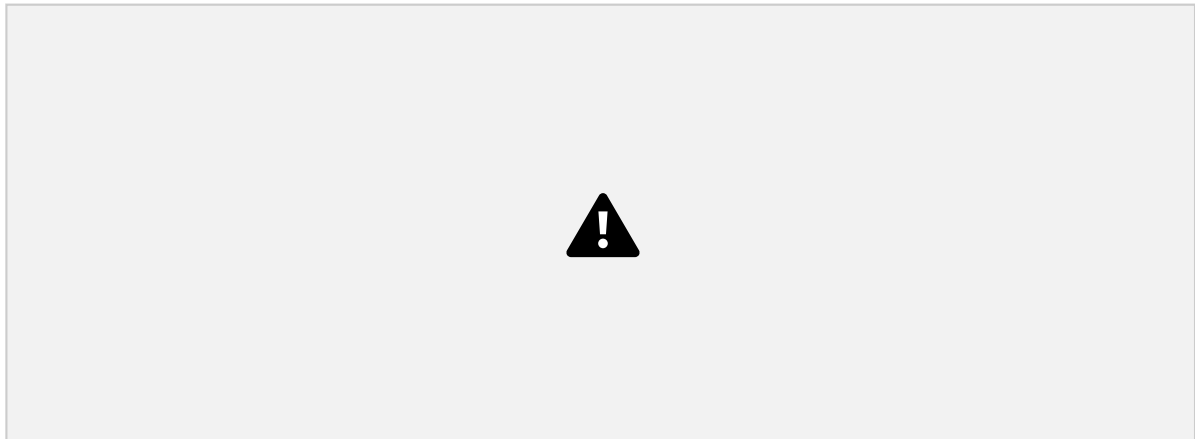


Рис. 9.3. Крок 1: черга та масив відвіданих вершин спочатку порожні

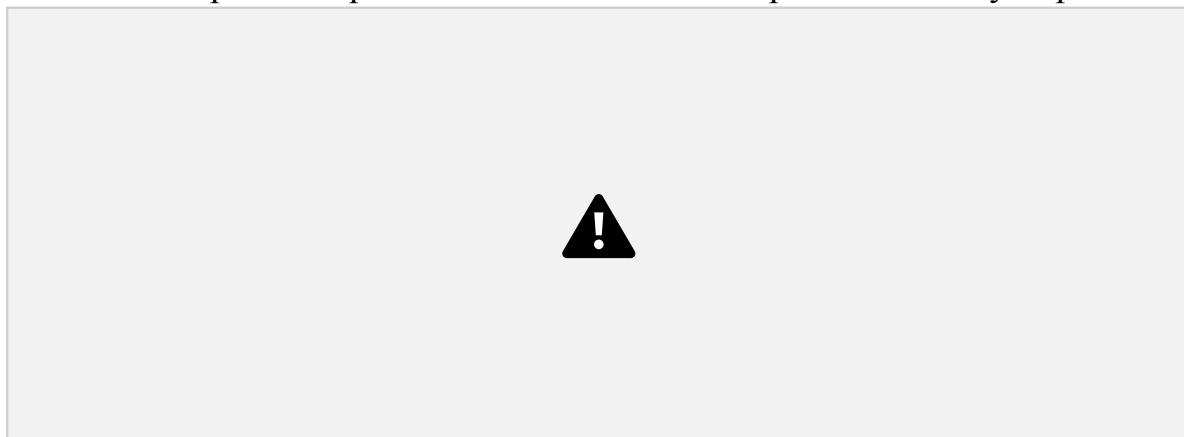


Рис. 9.4. Крок 2: занести вершину 0 в чергу та відмітити її як відвідану

64

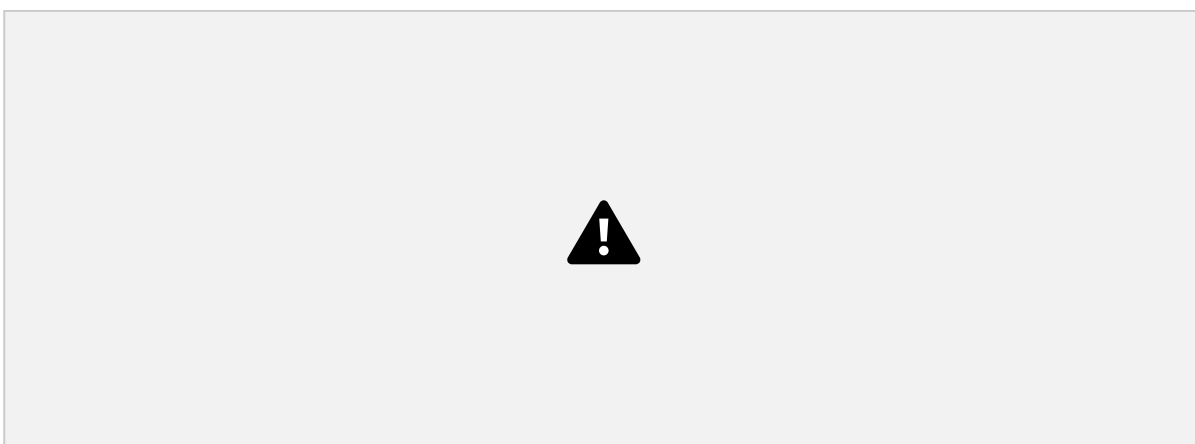


Рис. 9.5. Крок 3: видалити вершину 0 з черги; її невідвіданих сусідів 1 і 2 занести в чергу і відмітити як відвіданих

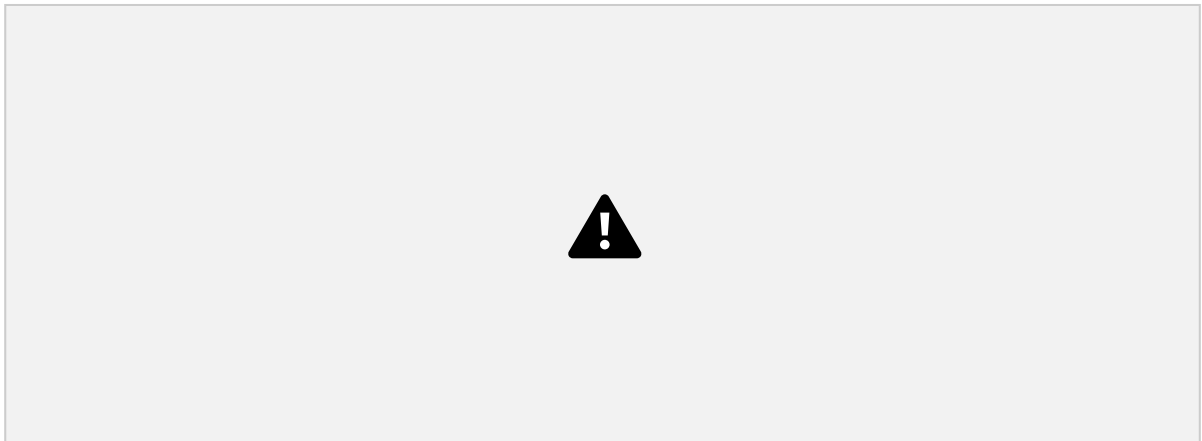


Рис. 9. 6. Крок 4: видалити вершину 1 з черги; її невідвіданого сусіда 3 за нести в чергу і відмітити як відвіданого

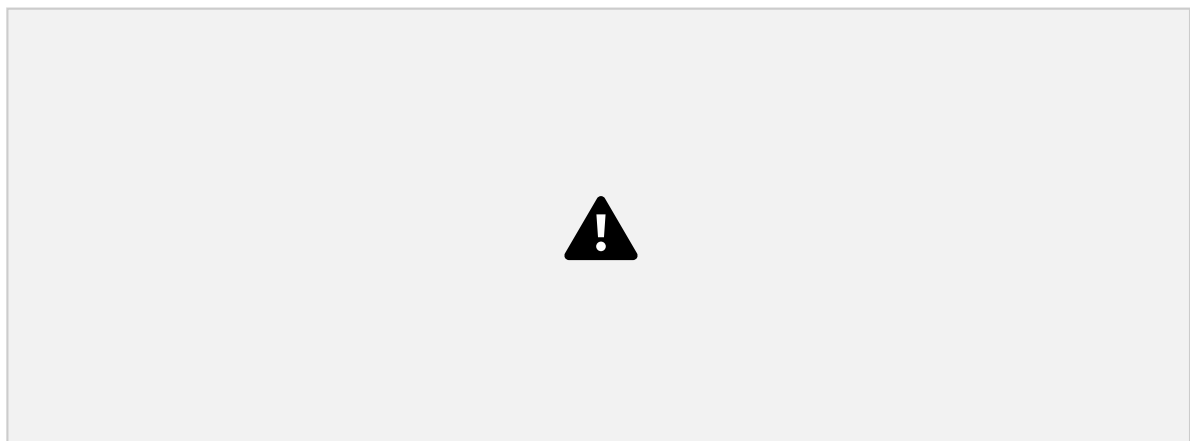


Рис. 9.7. Крок 5: видалити вершину 2 з черги; її невідвіданого сусіда 4 за нести в чергу і відмітити як відвіданого

65

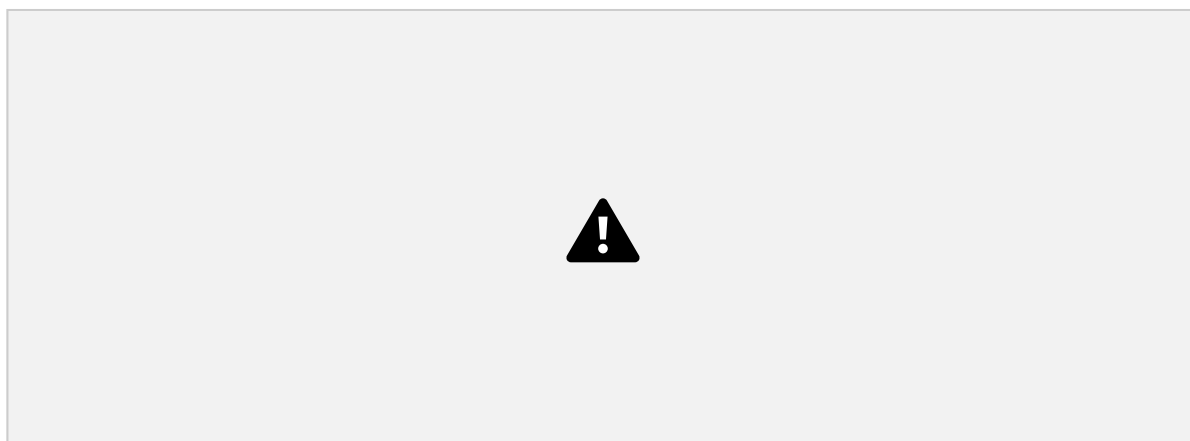


Рис. 9.8. Крок 6: видалити вершину 3 з черги; оскільки у неї немає невідвіданих сусідів у чергу нічого не заноситься



Рис. 9.9. Крок 7: видалити вершину 4 з черги; оскільки у неї немає невідвіданих сусідів у чергу нічого не заноситься

Тепер черга порожня, отже, ітераційний процес

завершено. **9.3 Пошук в глибину**

Стратегія пошуку в глибину (depth first search, DFS), як випливає з її назви, полягає в тому, щоб йти "вглиб" графа, наскільки це можливо. Під час пошуку в глибину досліджуються всі ребра, що виходять з вершини, відкритої останньою, і залишає вершину, тільки коли не залишається недосліджених ребер - при цьому відбувається повернення у вершину, з якої була відкрита вершина v . Цей процес триває доти, доки не будуть відкриті всі вершини, досяжні з вихідної. Якщо при цьому залишаються невідкриті вершини, то одна з них вибирається як нова вихідна вершина і пошук повторюється вже з неї. Цей процес повторюється доти, доки не будуть відкриті всі вершини.

Як і у разі пошуку в ширину, коли вершина v відкривається в процесі сканування списку суміжності вже відкритої вершини u , процедура пошуку

66

записує цю подію, встановлюючи поле попередника $\pi[v]$ рівним u . На відміну від пошуку в ширину, де підграф передування утворює дерево, при пошуку в глибину підграф передування може складатися з кількох дерев, оскільки пошук може виконуватися з кількох вихідних вершин.

Підграф передування (predecessor subgraph) пошуку в глибину, таким чином, дещо відрізняється від такого для пошуку в ширину. Ми визначаємо його як граф $G_\pi = (V, E_\pi)$, де

$$E_\pi = \{(\pi[v], v) : v \in V \text{ і } \pi[v] \neq \text{NIL}\}.$$

Підграф передування пошуку в глибину утворює ліс пошуку в глибину (depth first forest), який складається з кількох дерев пошуку в глибину (depth

first trees). Ребра в Ел називаються ребрами дерева (tree edges).

Як і в процесі пошуку в ширину, вершини графа розфарбовуються в різні кольори, що свідчать про їх стан. Кожна вершина спочатку біла, потім при відкритті (discover) у процесі пошуку вона забарвлюється в сірий колір, і після завершення (finish), коли її список суміжності повністю сканований, вона стає чорною. Така методика гарантує, що кожна вершина зрештою знаходиться тільки в одному дереві пошуку в глибину, так що дерева не перетинаються.

Крім побудови лісу пошуку у глибину, пошук у глибину також проста вляє у вершинах мітки часу (timestamp). Кожна вершина має дві такі мітки - першу $d[v]$, в якій вказується, коли вершина v відкривається (і забарвлюється в сірий колір), і друга - $f[v]$, яка фіксує момент, коли пошук завершує сканування списку суміжності вершини v і вона стає чорною. Ці мітки використовуються багатьма алгоритмами та корисні при розгляді поведінки пошуку у глибину.

Наведена нижче процедура DFS записує в полі $d[u]$ момент, коли вершина u відкривається, а в полі $f[u]$ - момент завершення роботи з вершиною u . Ці мітки часу є цілими числами в діапазоні від 1 до $2|V|$, оскільки у кожній з $|V|$ вершин є тільки одна подія відкриття і одна - завершення. Для кожної вершини u

$$d[u] < f[u] .$$

До моменту часу $d[u]$ вершина має колір WHITE, між $d[u]$ та $f[u]$ — колір GRAY, а після $f[u]$ - колір BLACK.

Далі наведено псевдокод алгоритму пошуку в глибину. Вхідний граф G може бути як орієнтованим, і неорієнтованим. Змінна $time$ глобальна та ви використовується для міток часу.

DFS(G)

1 for (для) кожної вершини $v \in V[G]$
2 do $color[v] \leftarrow WHITE$

67

3 $p[u] \leftarrow NIL$
4 $time \leftarrow 0$
5 for (для) кожної вершини $u \in V[G]$
6 do if $color[u] = WHITE$
7 then DFS_VISIT(u)

DFS_VISIT(u)

1 $color[u] \leftarrow GRAY$ // Відкрито білу вершину u 2 $time \leftarrow time + 1$
3 $d[u] \leftarrow time$
4 for (для) кожної вершини $v \in Adj[u]$ // Дослідження ребра (u, v) . 5
do if $color[v] = WHITE$


```

6 then  $p[v] \leftarrow u$ 
7 DFS_VISIT( $v$ )
8 color [ $u$ ]  $\leftarrow$  BLACK // Завершення 9  $f[u] \leftarrow \text{time} \leftarrow \text{time} + 1$ 

```

Для кращого розуміння роботи DFS проілюструємо його роботу покроково (рис. 9.10 – 9.15) [20].

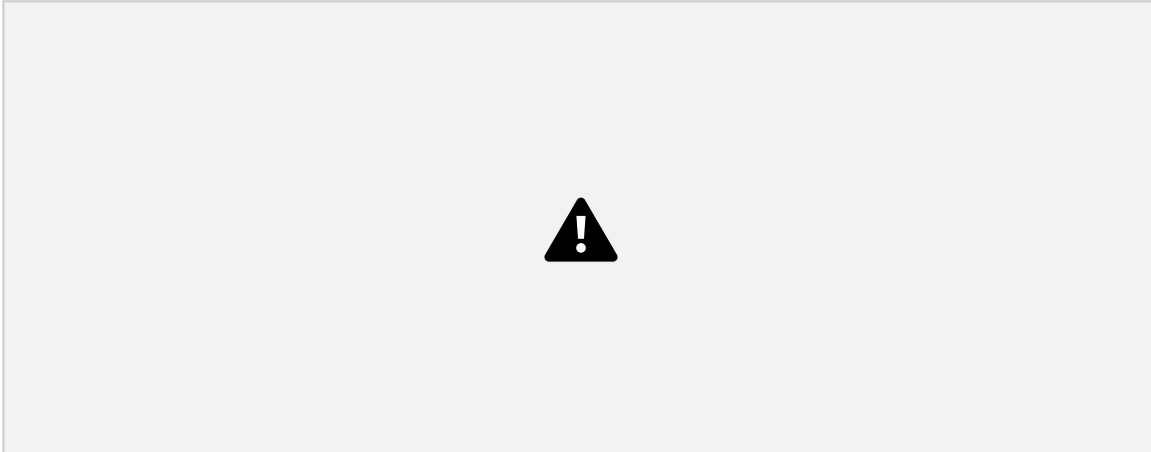


Рис. 9.10. Крок 1: спочатку стек і масив відвіданих вершин порожні

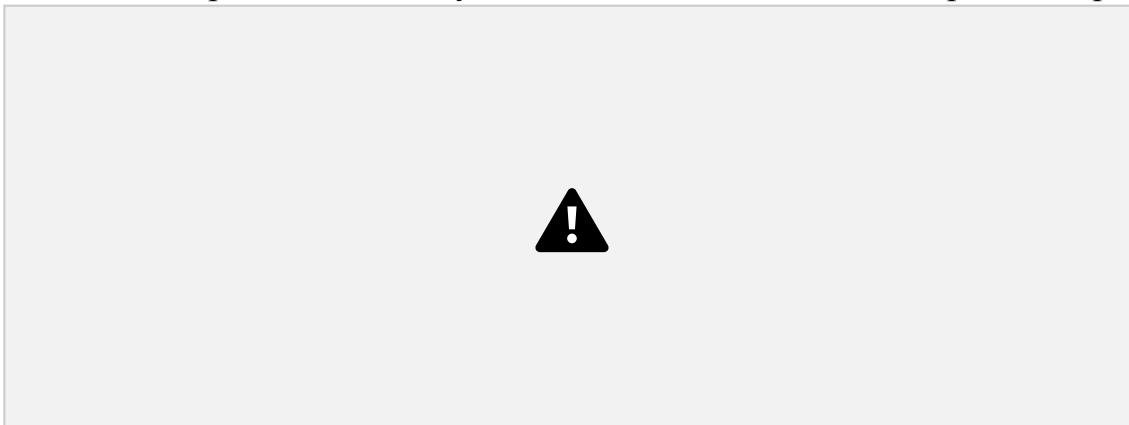


Рис. 9.11. Крок 2: відвідати вершину 0 і занести в стек невідвіданих сусідів

68

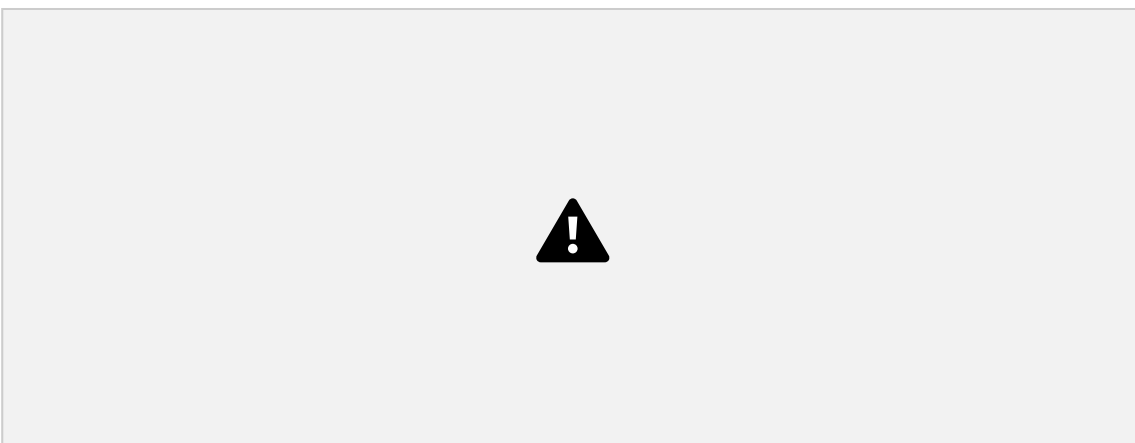


Рис. 9.12. Крок 3: виштовхнути зі стека вершину 1 і помітити її як відвідану; занести в стек її невідвіданих сусідів (яких немає)

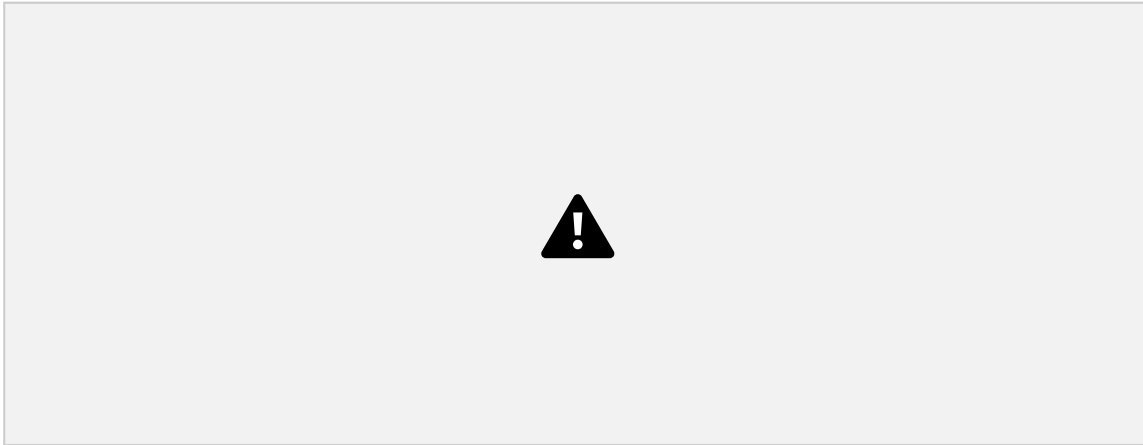


Рис. 9.13. Крок 4: виштовхнути зі стека вершину 2 і помітити її як відвідану; занести в стек її невідвіданих сусідів (тобто 4)

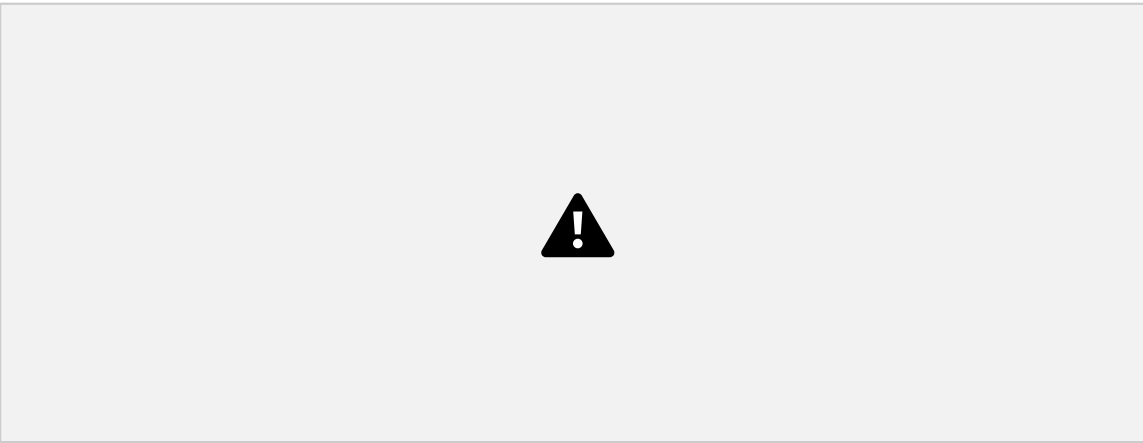


Рис. 9.14. Крок 5: виштовхнути зі стека вершину 4 і помітити її як відвідану; занести в стек її невідвіданих сусідів (яких немає)

69

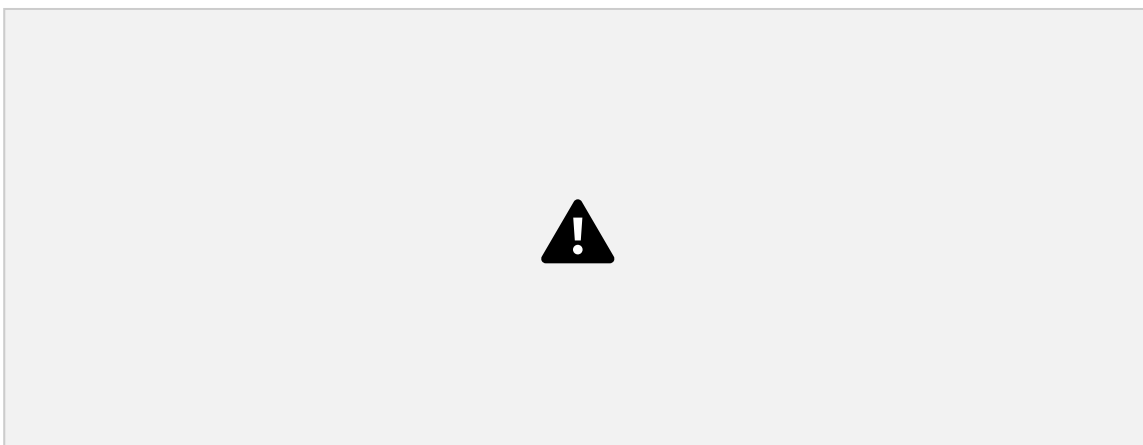


Рис. 9.15. Крок 6: виштовхнути зі стека вершину 3 і помітити її як відвідану; занести в стек її невідвіданих сусідів (яких немає)

Тепер стек порожній, отже ітераційний процес

завершено. **9.4. Вибір між BFS та DFS**

Важливе практичне питання, яке нас цікавить, полягає в тому, як зробити вибір між DFS і BFS.

Якщо ми знаємо, що цільова вершина розташована глибоко, ми повинні використовувати DFS, оскільки він досягає глибоких вузлів швидше, ніж BFS. Хорошим прикладом є задоволення обмежень. У цих задачах ми маємо призначити значення кільком змінним таким чином, щоб обмеження на змінні та їхні значення були задоволені. Кожен вузол у отриманому графі пошуку є списком призначень. Оскільки ми зацікавлені в повних рішеннях, має більше сенсу досягти повних вузлів призначення якомога швидше за допомогою DFS, ніж повільно шукати неповні призначення за допомогою BFS.

З іншого боку, якщо ми знаємо, що ціль може відображатися на низькому рівні в дереві пошуку, BFS є кращим вибором. Те саме стосується випадків, коли ми не можемо прийняти неоптимальний шлях і готові витратити більше пам'яті, щоб отримати теоретичні гарантії. Але застереження полягає в тому, що BFS може вимагати багато пам'яті та виявиться неефективним, якщо коефіцієнт розгалуження занадто великий або ціль знаходиться глибоко.

Саме витрати пам'яті роблять BFS непрактичним, навіть якщо він є повним і оптимальним у випадках, коли DFS не є таким. З цієї причини DFS став «робочою конячкою» у системах штучного інтелекту. Ми навіть можемо допустити проблеми його незавершеності та неоптимальних рішень. Ітераційне поглиблення, яке обмежує глибину DFS і запускає його з поступовими глибинами обмеження, є повним, оптимальним і має таку ж часову складність, як і BFS. Однак його витрати пам'яті такі ж, як і у DFS.

9.5 Контрольні питання

Дайте визначення графа.

Як називаються і як зображуються основні елементи графа? Які різновиди графів Ви знаєте?

Як подаються графи у пам'яті комп'ютера?

Які структури даних використовуються в процесі пошуку в ширину?

Які структури даних використовуються в процесі пошуку в глибину?

В яких випадках доцільно використовувати пошук в ширину? В яких випадках доцільно використовувати пошук в глибину?

10 АЛГОРИТМИ ПОШУКУ НАЙКОРОТШОГО ШЛЯХУ У ГРАФІ

10.1 Алгоритм Дейкстри

Алгоритм нідерландського вченого Едсгера Дейкстри знаходить всі найкоротші шляхи з однієї наперед заданій вершини графа до всіх інших. З його допомогою, за наявності всієї необхідної інформації, можна, наприклад, дізнатися яку послідовність доріг краще використовувати, щоб діста

тися з одного міста до кожного з багатьох інших, або в які країни вигідніше експортувати нафту тощо.

Мінусом даного підходу є неможливість обробки графів, в яких є ребра з негативним вагою, тобто, якщо, наприклад, деяка система передбачає збиткові для Вашої фірми маршрути, то для роботи з нею варто скористатися відмінним від алгоритму Дейкстри методом.

Для програмної реалізації алгоритму використовуємо два масиви: логічний `visited` - для зберігання інформації про відвідані вершини та числовий `distance`, в який будуть заноситися знайдені найкоротші шляхи. Отже, є граф $G=(V, E)$. Кожна з вершин, що входять в множину n , спочатку позначена як не відвідана, тобто елементам масиву `visited` присвоєно значення `false`.

Оскільки найвигідніші шляхи тільки належить знайти, в кожен елемент вектора `distance` записується таке число, яке точно більше будь-якого потенційного шляху (зазвичай це число називають нескінченністю, але в програмі використовують, наприклад максимальне значення конкретного типу даних). За вихідний пункт вибирається вершина s і їй приписується нульовий шлях: $distance[s] = 0$, тобто, немає ребра з s в s (метод не передбачає петель).

Далі, знаходяться всі сусідні вершини (в які є ребро з s) (нехай такими будуть t і u) і по черзі досліджуються, а саме обчислюється вартість маршруту з s черзі в кожен з них:

$distance[t] = distance[s] + \text{вага_інцидентного } s \text{ і } t \text{ ребра};$

$distance[u] = distance[s] + \text{вага_інцидентного } s \text{ і } u \text{ ребра}.$

Але цілком ймовірно, що в ту чи іншу вершину з s існує кілька шляхів, тому вартість шляху в таку вершину в масиві `distance` доведеться переглядати, тоді найбільше (неоптимальне) значення ігнорується, а найменше ставиться у відповідність вершині.

Після обробки суміжних з s вершин вона позначається як відвідана: `visited[s] = true`, і активною стає та вершина, шлях з s в яку мінімальний. Припустимо, шлях з s в u коротший, ніж з s в t , отже, вершина u стане активною і вище описаним способом досліджуються її сусіди, за винятком вершини s . Далі, u позначається як пройдена: `visited[u] = true`, активною стає вершина t , і вся процедура повторюється для неї. Алгоритм Дейкстри триває доти, поки всі доступні з s вершини не будуть досліджені.

Тепер на конкретному графі простежимо роботу алгоритму, знайдемо всі найкоротші шляхи між початковою та всіма іншими вершинами. Розмір (кількість ребер) зображеного на рис. 10.1 графа дорівнює 7 ($|E| = 7$), а порядок (кількість вершин) - 6 ($|n| = 6$). Це зважений граф, кожному з його ребер поставлено у відповідність деяке числове значення, тому цінність маршруту необов'язково визначається числом ребер, що лежать між парою

вершин.

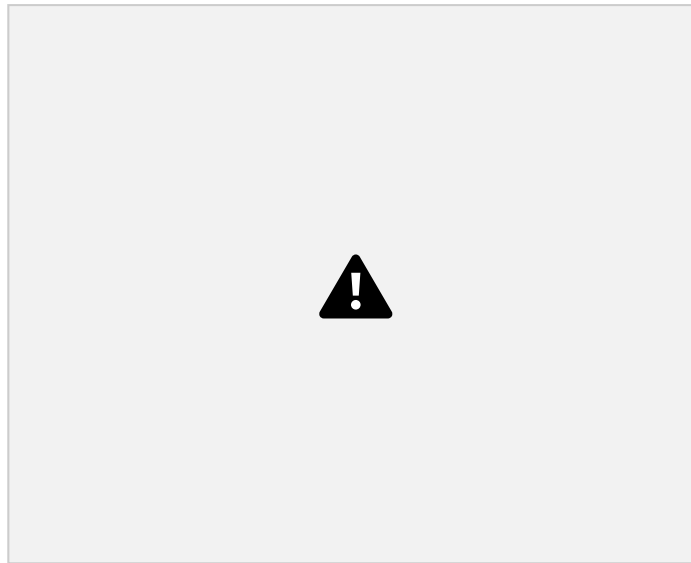


Рис. 10.1. Початковий граф для пошуку найкоротшого шляху за алгоритмом Дейкстри

З усіх вершин, що належать до множини n , виберемо одну, ту, від якої необхідно знайти найкоротші шляхи до інших доступних вершин. Нехай це буде вершина 1. Довжина шляху до всіх вершин, крім першої, спочатку дорівнює нескінченності, а до неї - 0, тобто, граф не має петель.

У вершини 1 рівно 3 сусідніх (вершини 2, 3, 5), і щоб обчислити довжину шляху до них потрібно додати вагу дуг, що лежать між вершинами: 1 і 2, 1 і 3, 1 і 5 до значення першої вершини (з нулем):

$$2 \leftarrow 1 + 0$$

$$3 \leftarrow 4 + 0$$

$$5 \leftarrow 2 + 0$$

Як уже зазначалося, отримані значення присвоюються вершинам, лише в тому випадку якщо вони «кращі» (мають менше значення), ніж інші на даний момент. А оскільки кожне з трьох чисел менше нескінченності, вони стають новими величинами, що визначають довжину шляху з вершини 1 до вершин 2, 3 і 5 (рис. 10.2).

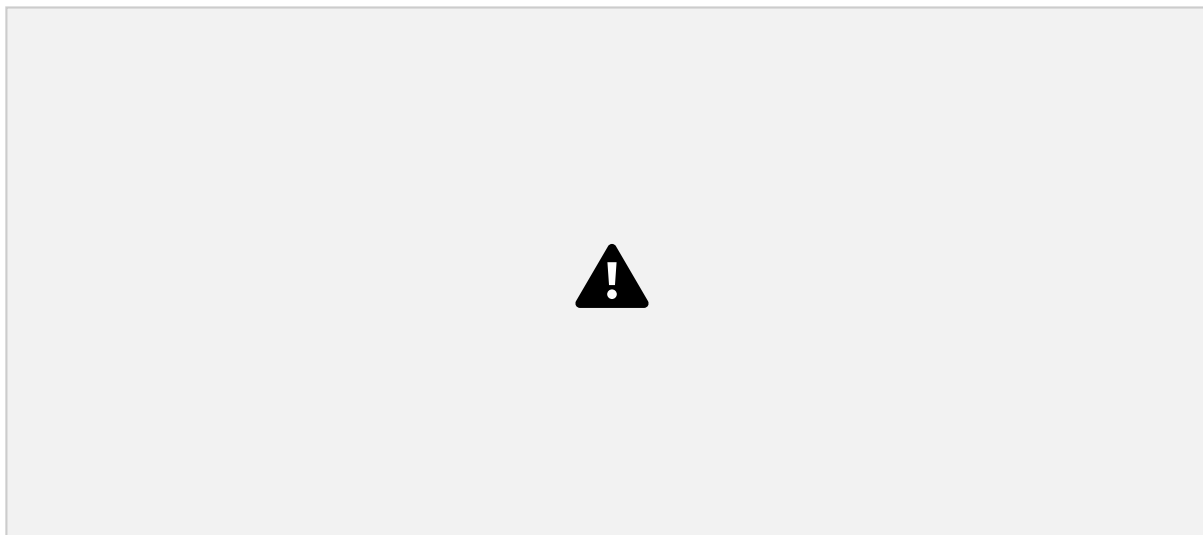


Рис. 10.2 Алгоритм Дейкстри: обробка вершини 1

Далі, активна вершина позначається як відвідана, статус «активної» (червоне коло) переходить до однієї з її сусідок, а саме до вершини 2, оскільки вона найближча до раніше активної вершини.

У вершини 2 всього один нерозглянутий сусід (вершина 1 позначена як відвідана), відстань до якого з неї дорівнює 9, але нам необхідно обчислити довжину шляху з початкової вершини, для чого потрібно додати величину, розраховану для вершини 2, до ваги дуги з неї до вершини 4: $4 \leftarrow 1 + 9$

Умова «стислості» ($10 < \infty$) виконується, отже, вершина 4 отримує нове значення довжини шляху (рис. 10.3).

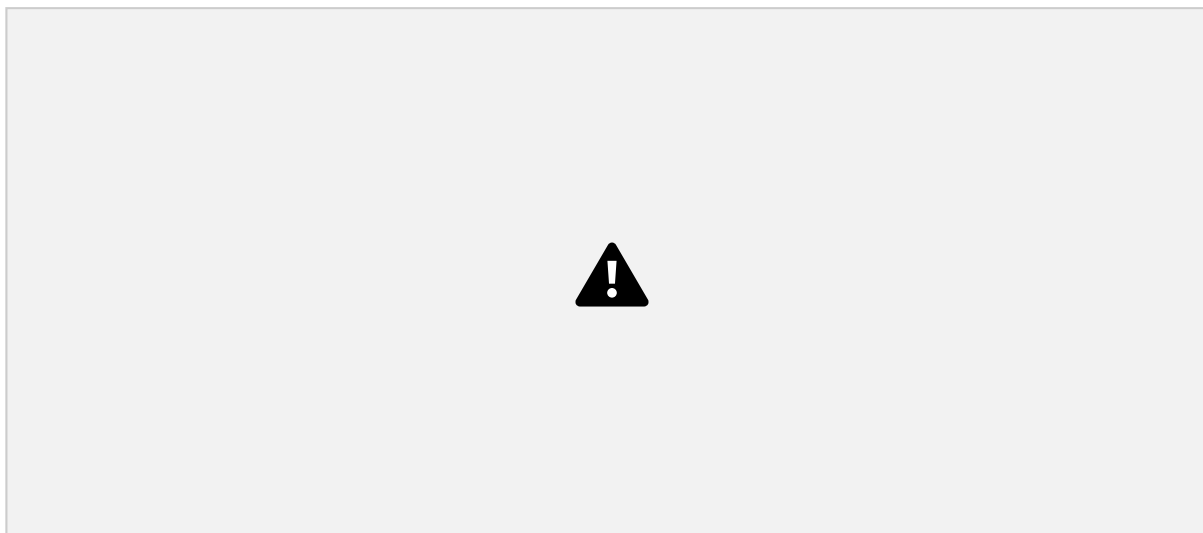


Рис. 10.3. Алгоритм Дейкстри: обробка вершини 2

Вершина 2 перестає бути активною, також як і вершина 1, і видаляється зі списку невідведаних. Тепер тим самим способом досліджуються сусіди вершини 5, і обчислюються відстані до них (рис. 10.4).

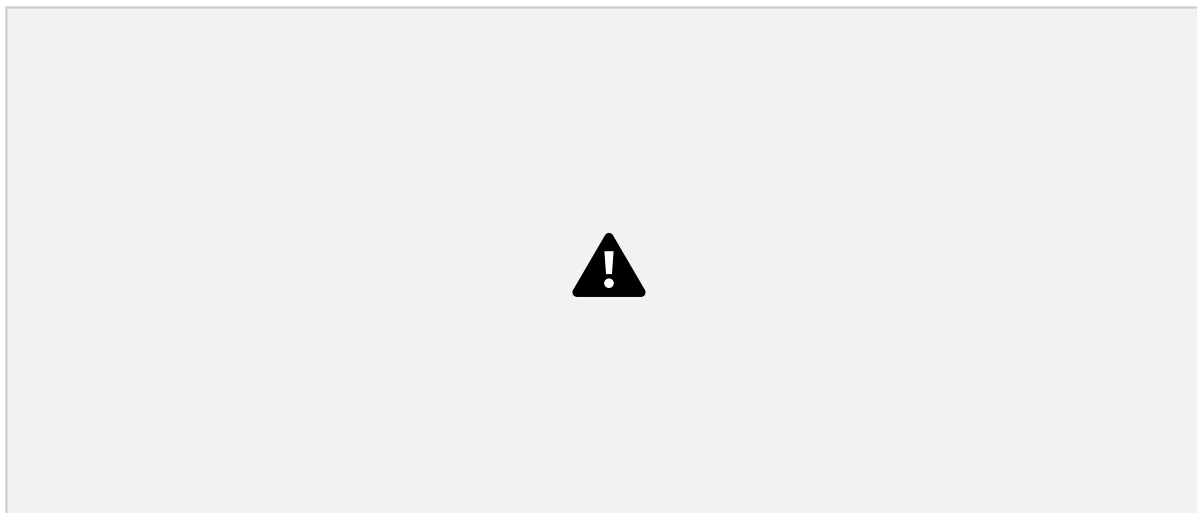


Рис. 10.4 Алгоритм Дейкстри: обробка вершини 5

Коли справа доходить до огляду сусідів вершини 3, то тут важливо не помилитися, оскільки вершину 4 вже була досліджено і довжина одного з можливих шляхів з початкової точки до неї обчислено. Якщо рухатися в неї через вершину 3, то шлях становитиме $4 + 7 = 11$, а $11 > 10$, тому нове значення ігнорується, старе залишається (рис. 10.5).

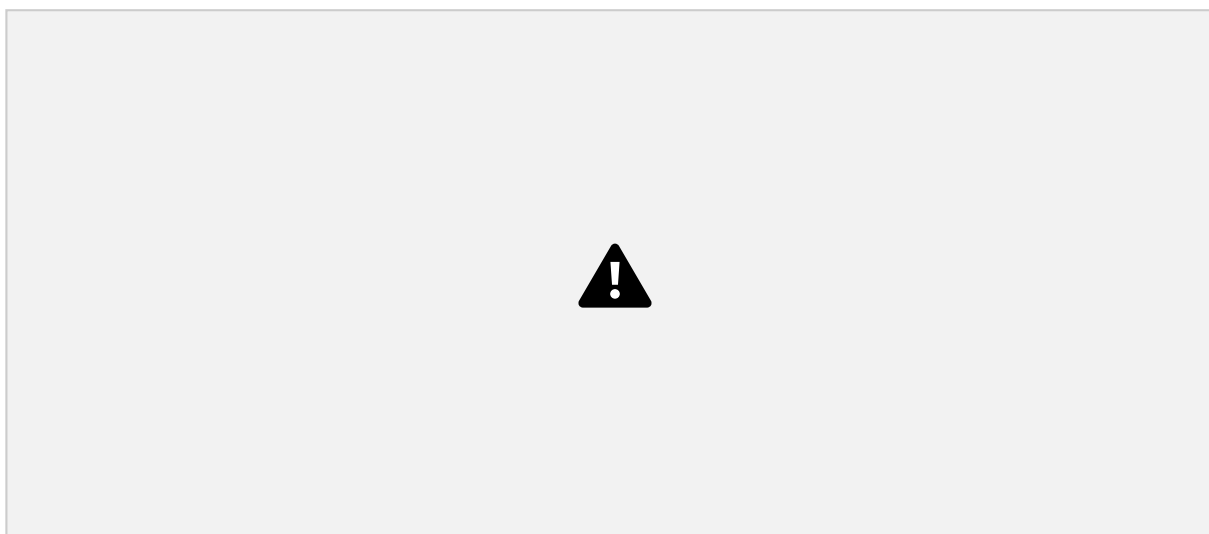


Рис. 10.5. Алгоритм Дейкстри: обробка вершини 3

Аналогічна ситуація з вершиною 6. Значення найближчого шляху до неї з вершини 1 дорівнює 10, а воно виходить тільки в тому випадку, якщо йти через вершину 5 (рис. 10.6).



Рис. 10.6. Алгоритм Дейкстри: обробка вершин 4 і 6

Коли всі вершини графа, або всі ті, що доступні з початкової точки, будуть позначені як відвідані, тоді робота алгоритму Дейкстри завершиться, і всі знайдені шляхи будуть найкоротшими. Так, наприклад, буде виглядати список найоптимальніших відстаней для шляхів, що лежать між вершиною 1 і всіма іншими вершинами розглянутого графа:

$1 \rightarrow 1 = 0$
 $1 \rightarrow 2 = 1$
 $1 \rightarrow 3 = 4$
 $1 \rightarrow 4 = 10$
 $1 \rightarrow 5 = 2$
 $1 \rightarrow 6 = 10$

10.2 Алгоритм Белмана-Форда

Завдання: Даний граф і початкова вершина src у графі. Необхідно знайти найкоротші шляхи від src до всіх вершин у даному графі. У графі можуть бути ребра з негативними вагами.

Ми вже обговорювали алгоритм Дейкстри як спосіб вирішення цього завдання. Алгоритм Дейкстри є жадібним алгоритмом, складність якого дорівнює $O(V \log V)$ (з використанням купи Фібоначчі). Однак алгоритм Дейкстри не працює для графів з негативними вагами ребер, тоді як алгоритм Белмана-Форда допускає існування у графі таких ребер. Алгоритм Белмана-Форда навіть простіший, ніж алгоритм Дейкстри, і добре підходить для розподілених систем. У той самий час складність його дорівнює $O(VE)$, що більше, ніж показник алгоритму Дейкстри.

Алгоритм

Нижче наведено докладно розписані кроки алгоритму.

Вхідні дані: Граф та початкова вершина src .

Вихідні дані: Найкоротша відстань до всіх вершин src . Якщо трапляється цикл негативної ваги, то короткі відстані не обчислюються, виводиться повідомлення про наявність такого циклу.

1. На цьому кроці ініціалізуються відстані від вихідної вершини до решти вершин, як нескінченні, а відстань до самого src приймається рівною 0. Створюється масив $dist[]$ розміру $|V|$ з усіма значеннями, рівними нескінченності, крім елемента $dist[src]$, де src — вихідна вершина.

2. Другим кроком обчислюються найкоротші відстані. Наступні кроки необхідно виконувати $|V|-1$ раз, де $|V|$ - число вершин в даному графі. Виконати таку дію для кожного ребра $u-v$:

якщо $dist[v] > dist[u] + \text{вага ребра } uv$, то оновити $dist[v]$:

$dist[v] = dist[u] + \text{вага ребра } uv$

На цьому кроці повідомляється, чи є у графі цикл негативної ваги. Для кожного ребра $u-v$ необхідно виконати таке:

якщо $dist[v] > dist[u] + \text{вага ребра } uv$,

то у графі присутній цикл негативної ваги.

3. Ідея кроку 3 полягає в тому, що крок 2 гарантує якнайкоротшу відстань, якщо граф не містить циклу негативної ваги. Якщо ми знову переберемо всі ребра і отримаємо коротший шлях для будь-якої з вершин, це буде сигналом присутності циклу негативної ваги.

Як це працює? Як і в інших завданнях динамічного програмування, алгоритм обчислює найкоротші шляхи знизу вгору. Спочатку він обчислює найкоротші відстані, тобто шляхи довжиною не більше, ніж одне ребро. Потім він обчислює найкоротші шляхи завдовжки не більше двох ребер тощо. Після i -ї ітерації зовнішнього циклу обчислюються найкоротші шляхи довжиною не більше i ребер. У кожному простому шляху може бути максимум $|V|-1$ ребер, тому зовнішній цикл виконується саме $|V|-1$ раз. Ідея полягає в тому, що якщо ми вирахували найкоротший шлях з не більш ніж i ребрами, то ітерація по всіх ребрах гарантує отримання найкоротшого шляху з не більше ніж $i + 1$ ребрами.

Приклад

Давайте розберемося в алгоритмі на прикладі графа, зображеного на рис. 10.7.

Нехай початкова вершина дорівнює 0. Візьміть всі відстані за нескінченні, крім відстані до самої src . Загальна кількість вершин у графі дорівнює 5, тому всі ребра потрібно пройти 4 рази.



Рис. 10.7. Початковий граф для пошуку алгоритма Белмана-Форда

Нехай ребра відпрацьовуються в такому порядку: (B, E), (D, B), (B, D), (A, B), (A, C), (D, C), (B, C), (E, D). Ми отримуємо такі відстані, коли прохід по ребрах був здійснений вперше. Перший рядок показує початкові відстані, другий рядок показує відстані, коли ребра (B, E), (D, B), (B, D) та (A, B) обробляються. Третій рядок показує відстань під час обробки (A, C). Четвертий рядок показує, що відбувається, коли обробляються (D, C), (B, C) та (E, D) (рис. 10.8).



Рис. 10.8. Алгоритм Белмана-Форда: перша ітерація

78

Перша ітерація гарантує, що всі найкоротші шляхи будуть не довшими за шлях в 1 ребро. Ми отримуємо наступні відстані, коли буде здійснено другий прохід по всіх ребрах (в останньому рядку показані кінцеві значення).

На рис. 10.9 показано результат роботи алгоритму після другої ітерації.

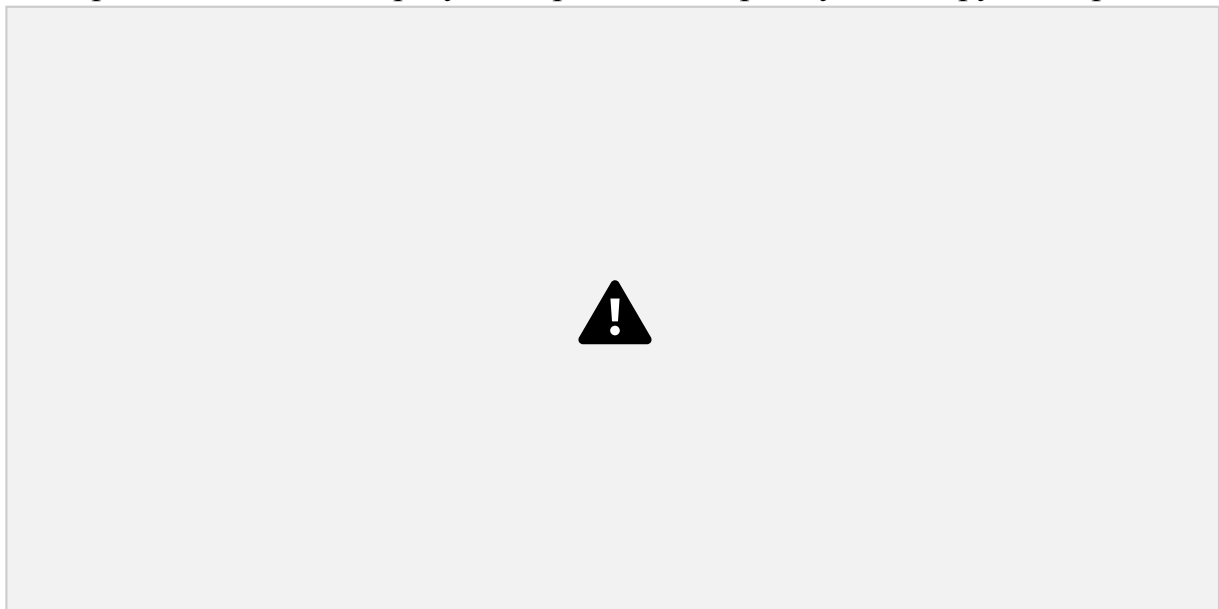


Рис. 10.8. Алгоритм Белмана-Форда: друга ітерація

Друга ітерація гарантує, що всі найкоротші шляхи матимуть довжину трохи більше 2 ребер. Алгоритм проходить по всіх ребрах ще 2 рази. Відстань мінімізується після другої ітерації, тому третя і четверта ітерації не вносять значення відстаней.

10.3. Контрольні питання

Яка часова складність алгоритму Дейкстри?

Які обмеження має застосування алгоритму Дейкстри?

Які структури даних використовуються для реалізації алгоритму Дейкстри?

У чому переваги алгоритму Белмана-Форда порівняно з алгоритмом Дейкстри?

Сформулюйте основні кроки алгоритму Белмана-Форда.

11 ДИНАМІЧНЕ ПРОГРАМУВАННЯ

11.1 Загальне поняття про динамічне програмування

Динамічне програмування - це особливий підхід до розв'язку завдань. Немає єдиного визначення динамічному програмуванню, але спробуємо його сформулювати. Ідея полягає в тому, що оптимальне рішення найчастіше можна знайти, розглянувши всі можливі шляхи вирішення завдання, і вибрати серед них найкраще.

Робота динамічного програмування дуже нагадує рекурсію із запам'ятовуванням проміжних рішень — таку рекурсію ще називають **мемоізацією**. Рекурсивні алгоритми, зазвичай, розбивають велике завдання на більш дрібні підзадачі і розв'язують їх. Динамічні алгоритми ділять завдання на шматочки та обчислюють їх по черзі, крок за кроком нарощуючи розв'язки. Тому динамічні алгоритми можна розглядати як рекурсію, працюючу знизу нагору (рис. 11.1).

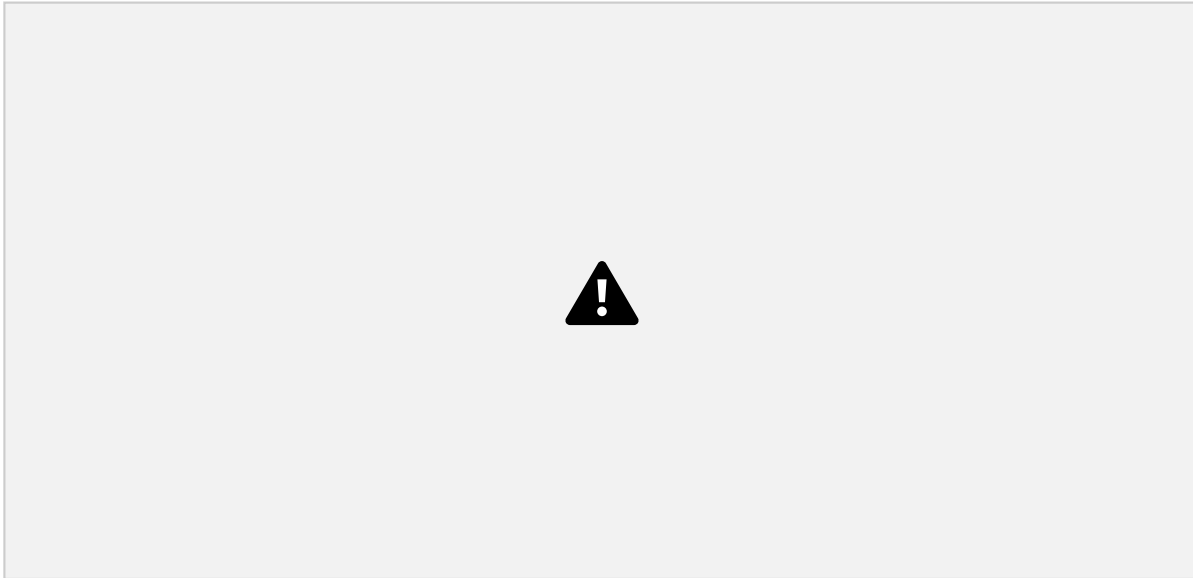


Рис. 11.1. Порівняння методів рекурсії та динамічного програмування

Магія динамічного програмування полягає у розумному поводженні з рішеннями підзавдань. «Розумний» у цьому контексті означає «що не вирішує те саме підзавдання двічі». Для цього рішення дрібних підзавдань мають десь зберігатися. Для багатьох реальних алгоритмів динамічного програмування такою структурою даних є таблиця.

У найпростіших випадках ця таблиця складатиметься лише з одного рядка - аналогічно звичайному масиву. Ці випадки будуть називатися одновимірним динамічним програмуванням і використовувати $O(n)$ пам'яті. Наприклад, алгоритм ефективного обчислення чисел Фібоначчі використо-